

Module-2

Decentralization using blockchain, Methods of decentralization, Blockchain and full ecosystem decentralization, Smart contract, Decentralized organizations, Decentralized autonomous organizations, Decentralized autonomous corporations, Decentralized autonomous societies Decentralized applications, Platforms for decentralization.

Cryptographic primitives: Symmetric cryptography, Asymmetric cryptography, Public and private keys, Hash functions: Compression of arbitrary messages into fixed length digest, Easy to compute, Pre-image resistance, Second pre-image resistance, Collision resistance, Message Digest (MD), Secure Hash Algorithms (SHAs), Merkle trees, Patricia trees, Distributed hash tables (DHTs), Digital signatures, Elliptic Curve Digital signature algorithm (ECDSA).

Chapter 2, Chapter 3: pg:56-105

Introduction

What is decentralization? [definition]

Decentralization is known as the distribution of functions among several units. It is an interconnected system where no single entity has complete authority. It is the architecture in which the workloads, both hardware, and software, are distributed among several workstations.

- The functions are distributed among several machines in a decentralized system instead of relying on a single server. They have multiple central owners. The owners can store the resources so that each user can have access.
- The system can be imagined in a graphical manner. Each user's machine can be visualized as nodes that are connected to one another. Each node has a copy of another node's data and the multiple owners have copies of all the nodes as well so as to reduce the access time.
- So, whenever an update or change is made in a node's data, the changes are reflected in the copies as well.
- Let's illustrate with the help of examples: **Bitcoin** is the latest technology and a prime example of a decentralized system. It is a blockchain where no central authority exists. Anyone and everyone can become a part of the network get involved in transactions and take part in voting. The decision is taken on the majority of votes. Dogecoin is a cryptocurrency that is decentralized and peer-to-peer that allows us to do transactions.

Importance of Decentralization

Decentralization is very important because of the following reasons:

- **Optimization of Resources:** Each user does not have to have all resources. The decentralized setup allows the user to share his burden with others at a lower level.
- **Greater output:** Since all users have the same authority, therefore each and every user work with greater efficiency so as to enhance the maximum productivity.

- **Flexibility:** Users can share their own views as there are no restrictions imposed by any central authority. They also have the flexibility to change their decisions.

Working of Decentralization

In a decentralized network, a user wants to share some data. The user doesn't have to take permission so has the full authority to publish something. In the network, each user is connected to one another. So, when the user shares the data, the data is shared among the other peers with the help of protocols. Peers approve the data. When the data is approved the protocols update the database. A database is maintained so as to keep track of all information.

Need of Decentralization

Nowadays, technology is advancing day by day and the number of users is also increasing. For the administration of the system, the centralized system is failing to meet all the criteria. So Decentralized system is becoming very useful day by day. It creates an efficient, secured, and reliable administration. It improves peer-to-peer networks. It ensures the right of each user. Each individual has the right to decision-making. Users also have access to the database. The biggest advantage of decentralization is if a part of the network crashes, the whole network will work uninterrupted. The main reason why decentralization is better than centralization is the flexibility and data to adapt to market demands quickly.

Is Decentralization secured?

Decentralization is highly secured. Since there is no central authority, there is no central server. Each user's server act as a central server. Therefore, there are multiple servers. Hacking all the servers is not a feasible option. Therefore, many organizations are now switching to a decentralized network. The prime example is Google. The products of Google from online search to mobile Android are given the freedom to work independently.

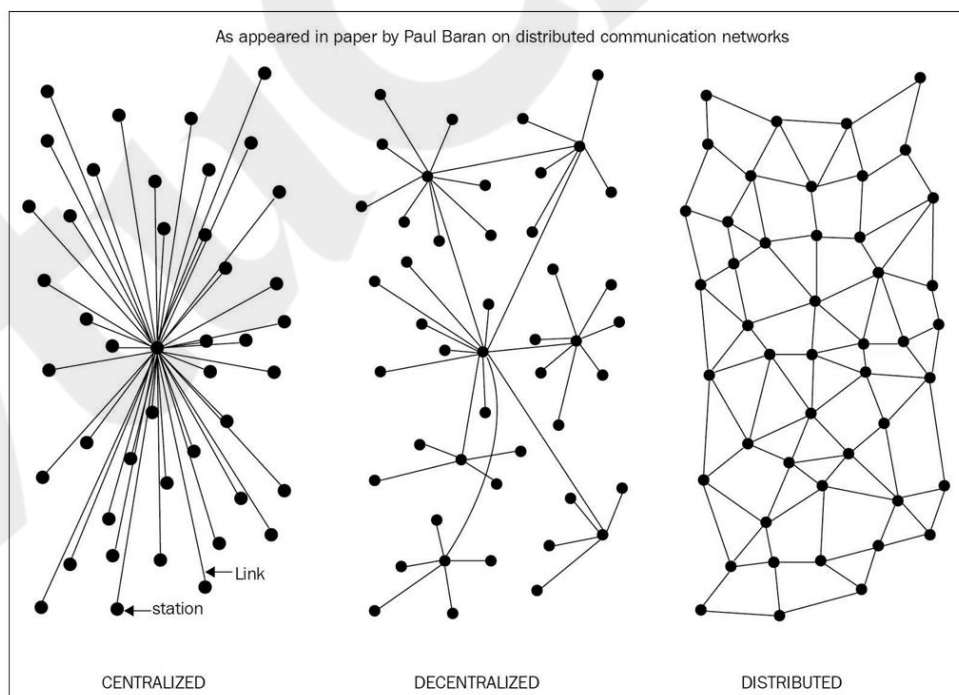
Benefits of Decentralization

- The main benefit is if some nodes fail or the main node fails, the whole system doesn't crash.
- The decision-making process is done on the basis of voting.
- They are usually open development platforms and there is less censorship.
- More machines can be added to the network.

DECENTRALIZATION USING BLOCKCHAIN

- Decentralization is a core benefit and service provided by the blockchain technology.

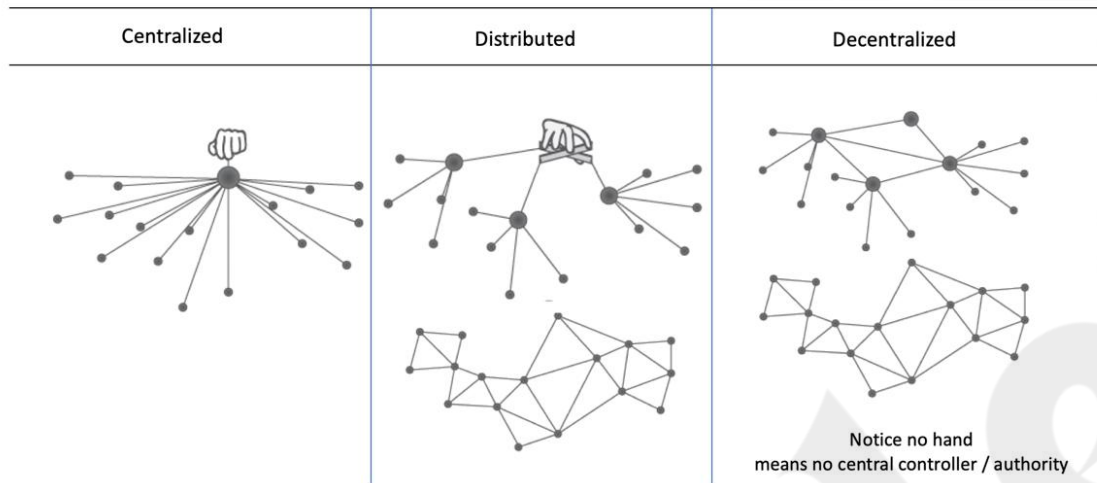
- Blockchain by design is a perfect vehicle for providing a platform that does not need any intermediaries and can function with many different leaders chosen via consensus mechanisms.
- This model allows anyone to compete to become the decision-making authority. This competition is governed by a consensus mechanism and the most commonly used method is known as Proof of Work (PoW).
- Decentralization is applied in varying degrees from semi-decentralized to fully decentralized depending on the requirements and circumstances.
- Decentralization can be viewed from a blockchain perspective as a mechanism that provides a way to remodel existing applications and paradigms or build new applications in order to give full control to users.
- Information and communication technology (ICT) has traditionally operated on a centralized model, where a central authority, such as a system administrator, controls the servers and operations.
- With the introduction of blockchain and technologies like Bitcoin, the model has shifted towards decentralization, allowing systems to operate without a single point of failure or trusted authority. These decentralized systems can run autonomously or with human intervention, depending on the governance model used.
- The concept of central, distributed, and decentralized systems, first described in a 1964 paper by Paul Baran on distributed communication networks, outlines different IT system architectures:



- **Centralized systems** are conventional (client-server) IT systems in which there is a single authority that controls the system, and who is solely in charge of all operations

on the system. All users of a centralized system are dependent on a single source of service. The majority of online service providers, including Google, Amazon, eBay, and Apple's App Store, use this conventional model to deliver services.

- In a **distributed system**, data and computation are spread across multiple nodes in the network. Sometimes, this term is confused with parallel computing.
- While there is some overlap in the definition, the main difference between these systems is that in a parallel computing system, computation is performed by all nodes simultaneously in order to achieve the result; for example, parallel computing platforms are used in weather research and forecasting, simulation, and financial modelling.
- On the other hand, in a distributed system, computation may not happen in parallel and data is replicated across multiple nodes that users view as a single, coherent system.
- Variations of both of these models are used to achieve fault tolerance and speed. In the parallel system model, there is still a central authority that has control over all nodes and governs processing. This means that the system is still centralized in nature.
- The main differences between distributed and parallel computing lie in computation timing and control. Parallel systems involve simultaneous computation across nodes, whereas distributed systems involve data replication and may not perform computations simultaneously.
- The critical difference between a decentralized system and distributed system is that in a distributed system, there is still a central authority that governs the entire system, whereas in a decentralized system, no such authority exists.
- A decentralized system is a type of network where nodes are not dependent on a single master node; instead, control is distributed among many nodes. This is analogous to a model where each department in an organization is in charge of its own database server, thus taking away the power from the central server and distributing it to the sub-departments, who manage their own databases.
- A significant innovation in the decentralized paradigm that has given rise to this new era of decentralization of applications is decentralized consensus. This mechanism came into play with Bitcoin, and it enables a user to agree on something via a consensus algorithm without the need for a central, trusted third party, intermediary, or service provider.



Different types of network/system

- The key difference between a decentralized system and distributed system is that in a distributed system, there still exists a central authority that governs the entire system, whereas in a decentralized system, no such authority exists.
- A decentralized system is a type of network whereby nodes are not dependent on a single master node; instead, control is distributed among many nodes. For example, this is analogous to a model where each department in an organization has its own database server that they are in charge of, thus taking away the power from the central server and distributing it to the sub-departments that manage their own databases.
- A real innovation in the decentralized paradigm that has started this new era of decentralization applications is decentralized consensus, which was introduced with bitcoin. This enables a user to agree on something via a consensus algorithm without the need for a central trusted third party, intermediary, or service provider.

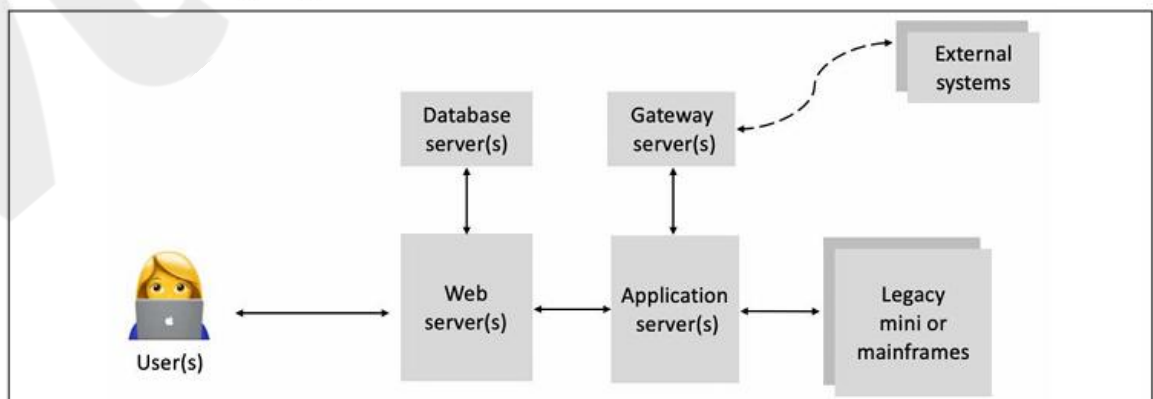


Figure 2.3: A traditional distributed system comprises many servers performing different roles

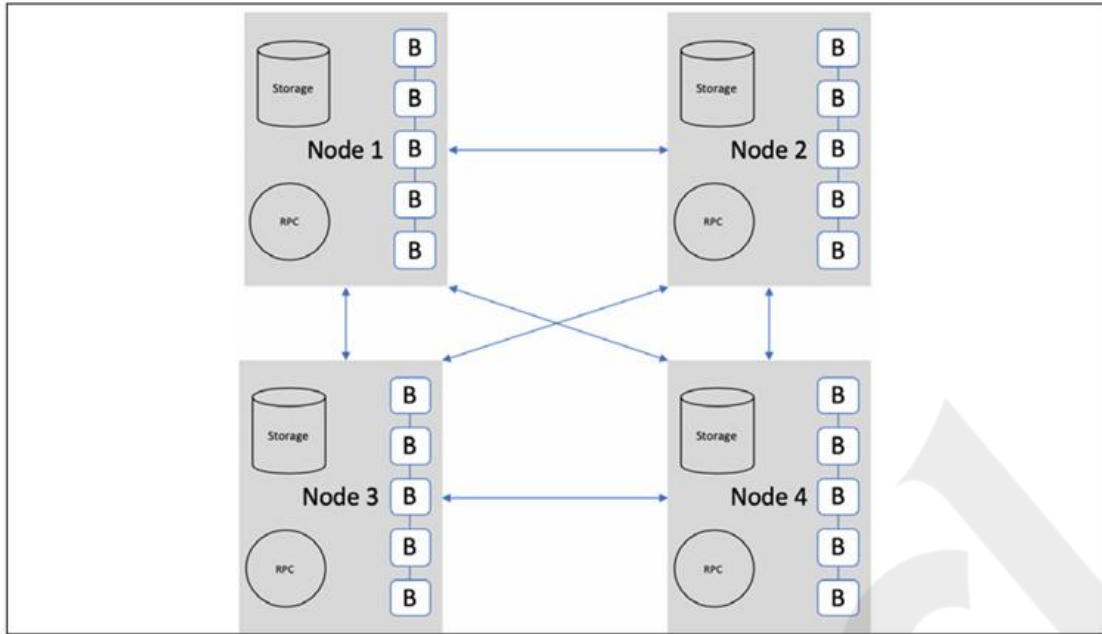


Figure 2.4: A blockchain-based decentralized system (notice the direct P2P connections and the exact replicas of blocks)

A comparison between centralized and decentralized systems (networks/applications) is shown in the following table:

Feature	Centralized	Decentralized
Ownership	Service provider	All users
Architecture	Client/server	Distributed, different topologies
Security	Basic	More secure
High availability	No	Yes
Fault tolerance	Basic, single point of failure	Highly tolerant, as service is replicated
Collusion resistance	Basic, because it's under the control of a group or even single individual	Highly resistant, as consensus algorithms ensure defense against adversaries
Application architecture	Single application	Application replicated across all nodes on the network
Trust	Consumers have to trust the service provider	No mutual trust required
Cost for consumer	Higher	Lower

Methods of decentralization

There are **two** methods that can be used to achieve decentralization.

- **Disintermediation**
- **Contest-driven decentralization**

1. Disintermediation:

- This can be explained with the help of an example. Imagine you want to send money to your friend in another country.
- You go to a bank that will transfer your money to the bank in the country of your choice for a fee. In this case, the bank keeps a central database that is updated, confirming that you have sent the money.
- With blockchain technology, it is possible to send this money directly to your friend without the need for a bank. All you need is the address of your friend on the blockchain. This way, the intermediary is no longer required and decentralization is achieved by disintermediation.
- However, it is debatable how practical decentralization is in the financial sector by disintermediation due to heavy regulatory and compliance requirements. Nevertheless, this model can be used not only in finance but also in many other different industries.
- In the health industry, where patients, instead of relying on a trusted third party (such as the hospital record system) can be in full control of their own identity and their data that they can share directly with only those entities that they trust.
- As a general solution, blockchain can serve as a decentralized health record management system where health records can be exchanged securely and directly between different entities (hospitals, pharmaceutical companies, patients) globally without any central authority.

2. Contest-driven decentralization

- In the method involving competition, different service providers compete with each other in order to be selected for the provision of services by the system. This paradigm does not achieve complete decentralization. However, to a certain degree, it ensures that an intermediary or service provider is not monopolizing the service.
- In the context of blockchain technology, a system can be envisioned in which smart contracts can choose an external data provider from a large number of providers based on their reputation, previous score, reviews, and quality of service. This method will not result in full decentralization, but it allows smart contracts to make a free choice based on the criteria just mentioned. This way, an environment of competition is cultivated among service providers where they compete with each other to become the data provider of choice.
- In the following diagram, varying levels of decentralization are shown. On the left side, the conventional approach is shown where a central system is in control; on the right side, complete disintermediation is achieved, as intermediaries are entirely removed. Competing intermediaries or service providers are shown in the center.
- At that level, intermediaries or service providers are selected based on reputation or voting, thus achieving partial decentralization.

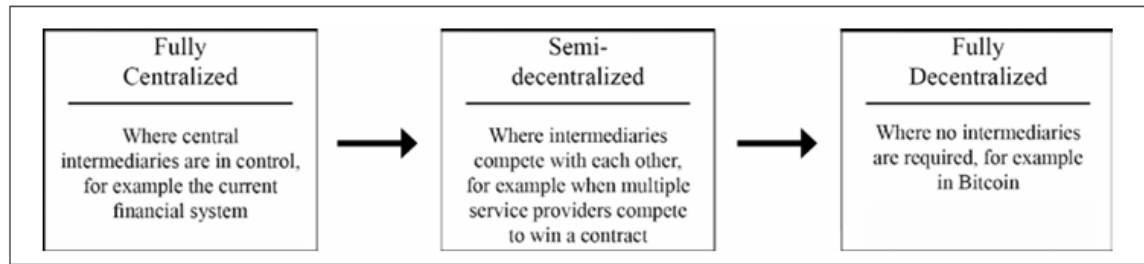


Figure 2.5: Scale of decentralization

Question	Yes/No	Recommended solution
Is high data throughput required?	Yes	Use a traditional database.
	No	A central database might still be useful if other requirements are met. For example, if users trust each other, then perhaps there is no need for a blockchain. However, if they don't or trust cannot be established for any reason, blockchain can be helpful.
Are updates centrally controlled?	Yes	Use a traditional database.
	No	You may investigate how a public/private blockchain can help.
Do users trust each other?	Yes	Use a traditional database.
	No	Use a public blockchain.
Are users anonymous?	Yes	Use a public blockchain.
	No	Use a private blockchain.
Is consensus required to be maintained within a consortium?	Yes	Use a private blockchain.
	No	Use a public blockchain.
Is strict data immutability required?	Yes	Use a blockchain.
	No	Use a central/traditional database.

ROUTES TO DECENTRALIZATION

- There are systems that pre-date blockchain and Bitcoin, including BitTorrent and the Gnutella file-sharing system, which to a certain degree could be classified as decentralized, but due to a lack of any incentivization mechanism, participation from the community gradually decreased. There wasn't any incentive to keep the users interested in participating in the growth of the network.
- With the advent of blockchain technology, many initiatives are being taken to leverage this new technology to achieve decentralization. The Bitcoin blockchain is typically the first choice for many, as it has proven to be the most resilient and secure blockchain and has a market cap of nearly \$166 billion at the time of writing.
- Alternatively, other blockchains, such as Ethereum, serve as the tool of choice for many developers for building decentralized applications. Compared to Bitcoin, Ethereum has become a more prominent choice because of the flexibility it allows for programming any business logic into the blockchain by using smart contracts.

HOW TO DECENTRALIZE?

- Arvind Narayanan and others have proposed a framework in their book Bitcoin and Cryptocurrency Technologies that can be used to evaluate the decentralization requirements of a variety of issues in the context of blockchain technology.
- The framework raises four questions whose answers provide a clear understanding of how a system can be decentralized:
 1. What is being decentralized?
 2. What level of decentralization is required?
 3. What blockchain is used?
 4. What security mechanism is used?
- **The first question** simply asks you to identify what system is being decentralized. This can be any system, such as an identity system or a trading system.
- **The second question** asks you to specify the level of decentralization required by examining the scale of decentralization, as discussed earlier. It can be full disintermediation or partial disintermediation.
- **The third question** asks developers to determine which blockchain is suitable for a particular application. It can be Bitcoin blockchain, Ethereum blockchain, or any other blockchain that is deemed fit for the specific application.
- Finally, a fundamental question that needs to be addressed is how the security of a decentralized system will be guaranteed. For example, the security mechanism can be atomicity based, where either the transaction executes in full or does not execute at all. This deterministic approach ensures the integrity of the system. Other mechanisms may include one based on reputation, which allows for varying degrees of trust in a system.

Decentralization framework example

The four questions discussed previously are used to evaluate the decentralization requirements of this application.

The answers to these questions are as follows:

1. Money transfer system
2. Disintermediation
3. Bitcoin
4. Atomicity

The responses indicate that the money transfer system can be decentralized by removing the intermediary, implemented on the Bitcoin blockchain, and that a security guarantee will be provided via atomicity.

Atomicity will ensure that transactions execute successfully in full or do not execute at all. We have chosen the Bitcoin blockchain because it is the longest established blockchain and has stood the test of time. Similarly, this framework can be used for any other system that needs to be evaluated in terms of decentralization. The answers to these four simple questions help clarify what approach to take to decentralize the system. To achieve complete decentralization, it is necessary that the environment around the blockchain also be decentralized.

BLOCKCHAIN AND FULL ECOSYSTEM DECENTRALIZATION

The blockchain is a distributed ledger that runs on top of conventional systems. These elements include storage, communication, and computation.

1. Storage Data can be stored directly in a blockchain, and with this fact it achieves decentralization. However, a significant disadvantage of this approach is that a blockchain is not suitable for storing large amounts of data by design.

- It can store simple transactions and some arbitrary data, but it is certainly not suitable for storing images or large blobs of data, as is the case with traditional database systems.
- A better alternative for storing data is to use **distributed hash tables (DHTs)**. DHTs were used initially in peer-to-peer file sharing software, such as BitTorrent, Napster, Kazaa, and Gnutella. DHT research was made popular by the CAN, Chord, Pastry, and Tapestry projects.
- BitTorrent is the most scalable and fastest network, but the issue with BitTorrent and the others is that there is no incentive for users to keep the files indefinitely. Users generally don't keep files permanently, and if nodes that have data still required by someone leave the network, there is no way to retrieve it except by having the required nodes rejoin the network so that the files once again become available.
- Two primary requirements here are high availability and link stability, which means that data should be available when required and network links also should always be accessible.
- Inter Planetary File System (IPFS) by Juan Benet possesses both of these properties, and its vision is to provide a decentralized World Wide Web by replacing the HTTP protocol.
- IPFS uses Kademlia DHT and Merkle Directed Acyclic Graphs (DAGs) to provide storage and searching functionality, respectively.
- The incentive mechanism for storing data is based on a protocol known as Filecoin, which pays incentives to nodes that store data using the BitSwap mechanism. The BitSwap mechanism lets nodes keep a simple ledger of bytes sent or bytes received in a one-to-one relationship.
- Also, a Git-based version control mechanism is used in IPFS to provide structure and control over the versioning of data. There are other alternatives for data storage, such as Ethereum Swarm, Storj, and MaidSafe.
- Ethereum has its own decentralized and distributed ecosystem that uses Swarm for storage and the Whisper protocol for communication. MaidSafe aims to provide a decentralized World Wide Web.
- BigChainDB is another storage layer decentralization project aimed at providing a scalable, fast, and linearly scalable decentralized database as opposed to a traditional filesystem. BigChainDB complements decentralized processing platforms and filesystems such as Ethereum and IPFS.

2. Communication

- The Internet (the communication layer in blockchain) is considered to be decentralized. This belief is correct to some extent, as the original vision of the Internet was to develop a decentralized communications system. Services such as email and online storage are now all based on a paradigm where the service provider is in control, and users trust such providers to grant them access to the service as requested. This model is based on the unconditional trust of a central authority (the service provider) where users are not in control of their data. Even user passwords are stored on trusted third-party systems.
- An alternative is to use mesh networks. Even though they are limited in functionality when compared to the Internet, they still provide a decentralized alternative where nodes can talk directly to each other without a central hub such as an ISP. An example of a mesh network is Firechat, which allows iPhone users to communicate with each other directly in a peer-to-peer fashion without an Internet connection
- Imagine a network that allows users to be in control of their communication; no one can shut it down for any reason. This could be the next step toward decentralizing communication networks in the blockchain ecosystem. It must be noted that this model may only be vital in a jurisdiction where the Internet is censored and controlled by the government.
- The original vision of the Internet was to build a decentralized network; however, over the years, with the advent of large-scale service providers such as Google, Amazon, and eBay, control is shifting toward these big players. For example, email is a decentralized system at its core; that is, anyone can run an email server with minimal effort and can start sending and receiving emails. There are better alternatives available. For example, Gmail and Outlook already provide managed services for end users, so there is a natural inclination toward selecting one of these large centralized services as they are more convenient and free to use. This is one example that shows how the Internet has moved toward centralization. Free services, however, are offered at the cost of exposing valuable personal data, and many users are unaware of this fact.
- Blockchain has revived the vision of decentralization across the world, and now concerted efforts are being made to harness this technology and take advantage of the benefits that it can provide.

3. COMPUTING POWER AND DECENTRALIZATION

- Decentralization of computing or processing power is achieved by a blockchain technology such as Ethereum, where smart contracts with embedded business logic can run on the blockchain network.
- Other blockchain technologies also provide similar processing-layer platforms, where business logic can run over the network in a decentralized manner

- The following diagram shows an overview of a decentralized ecosystem. In the bottom layer, the Internet or mesh networks provide a decentralized communication layer.
- In the next layer up, a storage layer uses technologies such as IPFS and BigChainDB to enable decentralization. Finally, in the next level up, you can see that the blockchain serves as a decentralized processing (computation) layer.
- Blockchain can, in a limited way, provide a storage layer too, but that severely hampers the speed and capacity of the system. Therefore, other solutions such as IPFS and BigChainDB are more suitable for storing large amounts of data in a decentralized way.
- The Identity and Wealth layers are shown at the top level. Identity on the Internet is a vast topic, and systems such as bitAuth and OpenID provide authentication and identification services with varying degrees of decentralization and security assumptions:

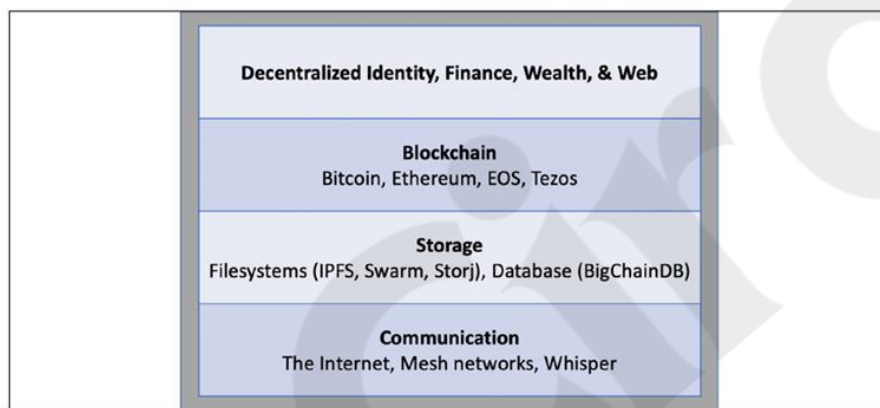


Figure 2.6: Decentralized ecosystem

- The blockchain is capable of providing solutions to various issues relating to decentralization. A concept relevant to identity known as **Zooko's Triangle** requires that the naming system in a network protocol is secure, decentralized, and able to provide human-meaningful and memorable names to the users. Conjecture has it that a system can have only two of these properties simultaneously.
- Nevertheless, with the advent of blockchain in the form of Namecoin, this problem was resolved. It is now possible to achieve security, decentralization, and human-meaningful names with the Namecoin blockchain. However, this is not a panacea, and it comes with many challenges, such as reliance on users to store and maintain private keys securely. This opens up other general questions about the suitability of decentralization to a particular problem. Decentralization may not be appropriate for every scenario. Centralized systems with well established reputations tend to work better in many cases. For example, email platforms from reputable companies such as Google or Microsoft would provide a better service than a scenario where individual email servers are hosted by users on the Internet

Pertinent terminology:

The following concepts are worth citing in the context of decentralization. The terminology introduced here is often used in the literature concerning decentralization and its applications.

Smart contracts

- A smart contract is a software program that usually runs on a blockchain. Smart contracts do not necessarily need a blockchain to run; however, due to the security benefits that blockchain technology provides, blockchain has become a standard decentralized execution platform for smart contracts.
- A smart contract usually contains some business logic and a limited amount of data. The business logic is executed if specific criteria are met. Actors or participants in the blockchain use these smart contracts, or they run autonomously on behalf of the network participants.

Autonomous agents

- An Autonomous Agent (AA) is an artificially intelligent software entity that acts on the behalf of its owner to achieve some desirable goals without requiring any or minimal intervention from its owner.

Decentralized organizations DOs are software programs that run on a blockchain and are based on the idea of actual organizations with people and protocols.

- Once a DO is added to the blockchain in the form of a smart contract or a set of smart contracts, it becomes decentralized and parties interact with each other based on the code defined within the DO software.

Decentralized autonomous organizations DAOs

- Decentralized autonomous organization (DAO) is also a computer program that runs on top of a blockchain, and embedded within it are governance and business logic rules. DAOs and DOs are fundamentally the same thing. The main difference, however, is that DAOs are autonomous, which means that they are fully automated and contain artificially intelligent logic. DOs, on the other hand, lack this feature and rely on human input to execute business logic.
- **Ethereum blockchain led the way with the introduction of DAOs.** In a DAO, the code is considered the governing entity rather than people or paper contracts. However, a human curator maintains this code and acts as a proposal evaluator for the community. DAOs are capable of hiring external contractors if enough input is received from the token holders (participants)
- The most famous DAO project is **The DAO**, which raised \$168 million in its crowdfunding phase. The DAO project was designed to be a venture capital fund aimed at providing a decentralized business model with no single entity as owner. Unfortunately, this project was hacked due to a bug in the DAO code, and millions of dollars' worth of ether currency (ETH) was siphoned out of the project and into a child DAO created by hackers. A major network change (hard

fork) was required on the Ethereum blockchain to reverse the impact of the hack and initiate the recovery of the funds. This incident opened up the debate on the security, quality, and need for thorough testing of the code in smart contracts in order to ensure their integrity and adequate control. There are other projects underway, especially in academia, that are seeking to formalize smart contract coding and testing.

Decentralized autonomous corporations DACs

- Decentralized autonomous corporations (DACs) are similar to DAOs in concept, though considered to be a subset of them. The definitions of DACs and DAOs may sometimes overlap, but the general distinction is that **DAOs are usually considered to be nonprofit, whereas DACs can earn a profit** via shares offered to the participants and to whom they can pay dividends. DACs can run a business automatically without human intervention based on the logic programmed into them.

Decentralized autonomous societies (DASes)

- Decentralized autonomous societies (DASes) are a concept whereby an entire society can function on a blockchain with the help of multiple, complex smart contracts and a combination of DAOs and decentralized applications (DApps) running autonomously.
- This model does not necessarily translate to a free-for-all approach, nor is it based on an entirely libertarian ideology; instead, many services that a government commonly offers can be delivered via blockchains, such as government identity card systems, passports, and records of deeds, marriages, and births.
- Another theory is that, if a government is corrupt and central systems do not provide the levels of trust that a society needs, then that society can start its own virtual one on a blockchain that is driven by decentralized consensus and transparency. This concept might look like a libertarian's or cypherpunk's dream, but it is entirely possible on a blockchain.

DECENTRALIZED APPLICATIONS(DAPPS)

DApps at a fundamental level are software programs that execute using either of the following methods. They are categorized as Type 1, Type 2, or Type 3 DApps:

1.Type 1: Run on their own dedicated blockchain, for example, standard smart contract based DApps running on Ethereum.

- If required, they make use of a native token, for example, ETH on Ethereum blockchain. For example, Ethlance is a DApp that makes use of ETH to provide a job market.

2.Type 2: Use an existing established blockchain. that is, make use of Type 1 blockchain and bear custom protocols and tokens, for example, smart contract based tokenization DApps running Ethereum blockchain.

- An example is **DAI**, which is built on top of Ethereum blockchain, but contains its own stable coins and mechanism of distribution and control.
- Another example is Golem, which has its own token GNT and a transaction framework built on top of Ethereum blockchain to provide a decentralized marketplace for

computing power where users share their computing power with each other in a peer-to-peer network.

- A prime example of Type 2 DApps is the OMNI network, which is a software layer built on top of Bitcoin to support trading of custom digital assets and digital currencies. More information on the OMNI network can be found at <https://www.omnilayer.org>
- **3. Type 3:** Use the protocols of Type 2 DApps; for example, the SAFE Network uses the OMNI network protocol.
- Another example to understand the difference between different types of DApps is the USDT token (Tethers).
- The original USDT uses the OMNI layer (a Type 2 DApp) on top of the Bitcoin network. USDT is also available on Ethereum using ERC20 tokens.
- This example shows that a USDT can be considered a Type 3 DApp, where the OMNI layer protocol (a Type 2 DApp) is used, which is itself built on Bitcoin (a Type 1 DApp).
- Also, from an Ethereum point of view USDT can also be considered a Type 3 DApp in that it makes use of the Type 1 DApp Ethereum blockchain using the ERC 20 standard, which was built to operate on Ethereum.

What are the requirements of DAPP?

For an application to be considered decentralized, it must meet the following criteria. This definition was provided in a whitepaper by **Johnston et al. in 2015, The General Theory of Decentralized Applications, DApps:**

1. The DApp should be fully open source and autonomous, and no single entity should be in control of a majority of its tokens. All changes to the application must be consensus driven based on the feedback given by the community.
2. Data and records of operations of the application must be cryptographically secured and stored on a public, decentralized blockchain to avoid any central points of failure.
3. A cryptographic token must be used by the application to provide access for and incentivize those who contribute value to the applications, for example, miners in Bitcoin.
4. The tokens (if applicable) must be generated by the decentralized application using consensus and an applicable cryptographic algorithm. This generation of tokens acts as a proof of the value to contributors (for example, miners).

Generally, DApps now provide all sorts of different services, including but not limited to financial applications, gaming, social media, and health.

Operations of a DApp

Establishment of consensus by a DApp can be achieved using consensus algorithms such as PoW and Proof of Stake (PoS). So far, only PoW has been found to be incredibly resistant to attacks, as is evident from the success of and trust people have put in the Bitcoin network.

Furthermore, a DApp can distribute tokens (coins) via mining, fundraising, and development.

DESIGN OF DAPP/DECENTRALISED APPLICATIONS:

A DApp—pronounced Dee-App, or now more commonly rhyming with app—is a software application that runs on a decentralized network such as a distributed ledger. They have recently become very popular due to the development of various decentralized platforms such as Ethereum, EOS, and Tezos.

- Traditional apps commonly consist of a user interface and usually a web server or an application server and a backend database. This is a common client/server architecture. This is visualized in the following diagram:

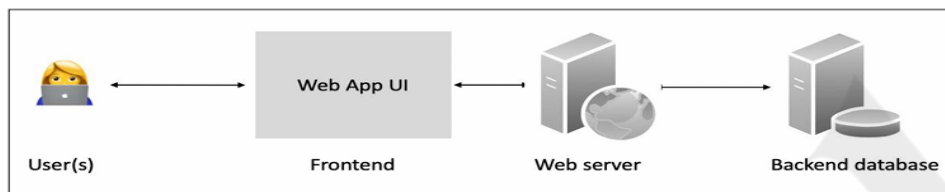


Figure 2.7: Traditional application architecture (generic client/server)

- A DApp on the other hand has a blockchain as a backend and can be visualized as depicted in the following diagram. The key element that plays a vital role in the creation of a DApp is a smart contract that runs on the blockchain and has business logic embedded within it:

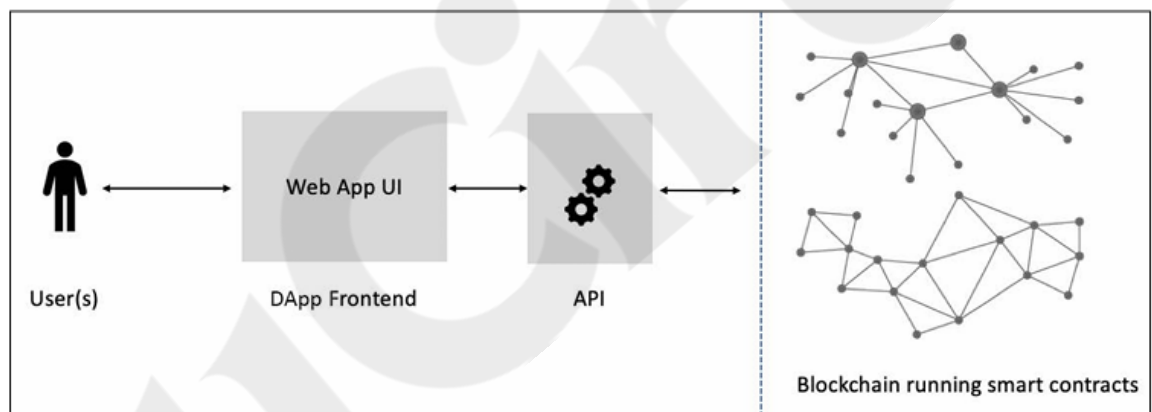


Figure 2.8: Generic DApp architecture

- Note that the frontend in either a DApp or app architecture can either be a thick client, a mobile app, or a web frontend (a web user interface). However, it is usually a web frontend commonly written using a JavaScript framework such as React or Angular.
- The following comparison table highlights the key properties of and differences between these different types of decentralized entities.

Entity	Autonomous?	Software?	Owned?	Capital?	Legal status	Cost
DO	No	No	Yes	Yes	Yes	High
DAO	Yes	Yes	No	Yes	Unsettled	Low
DAC	Yes	Yes	Yes	Yes	Unsettled	Low
DAS	Yes	Yes	No	Possible	Unsettled	Low
DApp	Yes	Yes	Yes	Optional tokens	Unsettled	Use case dependent

DApp examples

- **KYC-Chain** This application provides the facility to manage Know Your Customer (KYC) data securely and conveniently based on smart contracts.
- **OpenBazaar** This is a decentralized peer-to-peer network that enables commercial activities directly between sellers and buyers instead of relying on a central party, such as eBay or Amazon. It should be noted that this system is not built on top of a blockchain; instead, distributed hash tables are used in a peer-to-peer network to enable direct communication and data sharing among peers. It makes use of Bitcoin and various other cryptocurrencies as a payment method.
- **Lazooz** This is the decentralized equivalent of Uber. It allows peer-to-peer ride sharing and users to be incentivized by proof of movement, and they can earn Zooz coins.

PLATFORMS FOR DECENTRALIZATION

Any blockchain network, such as Bitcoin, Ethereum, Hyperledger Fabric, or Quorum, can be used to provide a decentralization service. Many organizations around the world have introduced platforms that promise to make distributed application development easy, accessible, and secure. Some of these platforms are described as follows

Ethereum

- Ethereum tops the list as being the first blockchain to introduce a Turing-complete language and the concept of a virtual machine. This is in stark contrast to the limited scripting language in Bitcoin and many other cryptocurrencies.
- With the availability of its Turing-complete language, **Solidity**, endless possibilities have opened for the development of decentralized applications. This blockchain was first proposed in 2013 by Vitalik Buterin, and it provides a public blockchain to develop smart contracts and decentralized applications. **Currency tokens on Ethereum are called ethers.**

MaidSafe

- This is a project for the decentralized Internet introduced in 2006. This is not a blockchain, but a decentralized and autonomous network.
- MaidSafe provides a **SAFE (Secure Access for Everyone) network that** is made up of unused computing resources, such as storage, processing power, and the data connections of its users.
- The files on the network are divided into small chunks of data, which are encrypted and distributed randomly throughout the network. This data can only be retrieved by its respective owner.

- One key innovation of MaidSafe is that duplicate files are automatically rejected on the network, which helps reduce the need for additional computing resources needed to manage the load. It uses Safecoin as a token to incentivize its contributors.

Lisk

- Lisk is a blockchain application development and cryptocurrency platform. It allows developers to use JavaScript to build decentralized applications and host them in their respective sidechains.
- Lisk uses the **Delegated Proof of Stake (DPOS) mechanism** for consensus, whereby 101 nodes can be elected to secure the network and propose blocks. It uses the Node.js and JavaScript backend, while the frontend allows the use of standard technologies, such as CSS3, HTML5, and JavaScript.
- Lisk uses **LSK coin** as a currency on the blockchain. Another derivative of Lisk is Rise, which is a Lisk-based DApp and digital currency platform. It offers greater focus on the security of the system.

EOS

- This is a blockchain protocol launched in January 2018, with its own cryptocurrency called EOS. EOS raised an incredible 4 billion USD in 2018 through its Initial Coin Offering (ICO).
- The key purpose behind EOS is, as stated by its founders, to build a decentralized operating system. Its throughput is significantly higher (approx. 3,996 transactions per second (TPS)) than other common blockchain platforms, such as Bitcoin (approx. 7 TPS) and Ethereum (approx. 15 TPS).

INNOVATIVE TRENDS IN BLOCKCHAIN TECHNOLOGY

1. Decentralized web:

- Decentralized web is a term that's used to describe a vision of the web where no central authority or set of authorities will be in control.
- The original intention of the Internet was indeed decentralized, and the development of open protocols such as HTTP, SMTP, and DNS meant that any individual could use these protocols freely, and immediately become part of the Internet. This is still true; however, with the emergence of a layer above these protocols called the Web layer introduced a more service-oriented infrastructure, which inevitably led to large profit seeking companies taking over.
- This is evident from the rise of Facebook, Google, Twitter, and Amazon, which of course provide excellent user services but at the cost of a more controlled, centralized, and closed system.

- Once intended and developed as decentralized, open and free protocols are now being dominated by powerful commercial entities around the world, which has resulted in major concerns around privacy and data protection.
- These types of business models do work well and are quite popular due to the high level of standardization and services provided, but they pose a threat to privacy and decentralization due to the dominance of only a handful of entities on the entire Internet.
- With blockchain, it is envisioned that this situation will change as it will allow development of the decentralized Internet, or the decentralized web, or Web 3 for short, which was the original intention of the Internet.

The evolution of the Web over the last few decades by dividing the major developments into three key stages, Web 1, Web 2, and Web 3

- 1. Web 1** This is the original World Wide Web, which was developed in 1989. This was the era when static web pages were hosted on servers and usually only allowed read actions from a user's point of view.
- 2. Web 2** This is the era when more service-oriented and web-hosted applications started to emerge. E-commerce websites, social networking, social media, blogs, multimedia sharing, mash-ups, and web applications are the main features of this period.
 - The current Web is Web 2, and even though we have a richer and more interactive Internet, all of these services are still centralized. Web 2 has generated massive economic value and provides services that are essential for day to-day business, personal use, social interactions, and almost every walk of life, but privacy concerns, the need for trusted third parties, and data breaches are genuine issues that need to be addressed.
 - Common examples of centralized Web 2 services include Twitter, Facebook, Google Docs, and email services such Gmail and Hotmail.
- 3. Web 3** This is the vision of the decentralized internet or web that will revolutionize the way we use the internet today. This is the era that will be fully user-centric and decentralized without any single authority or a large organization or internet company in control.
 - Some examples of Web 3 are as follows:
 - Steemit: This is a social media platform based on the Steem blockchain and STEEM cryptocurrency. This cryptocurrency is awarded to contributors for the content they have shared, and the more votes they get, the more tokens they earn. More information is available at <https://steemit.com>
 - Status: This is a decentralized multipurpose communication platform providing secure and private communication. More information is available at <https://status.im>.

- IPFS: This is a peer-to-peer hypermedia/storage protocol that allows storage and sharing of data in a decentralized fashion across a peer-to-peer network. More information is available at <https://ipfs.io>.

Decentralized identity:

- Another trend that has gained massive popularity recently is decentralized identity. Identity is a sensitive and difficult problem to solve.
- Currently, due to the dominance of large Internet companies and other similar organizations, the identity of a user is not in control of the identity holder and this often leads to privacy issues.
- Decentralized identity gives control of identity credentials back to identity holders and enables them to control when and how they share their credentials and with whom.
- A prime example of such an initiative is that Microsoft has built a decentralized identity network called **Identity Overlay Network (ION)** on top of Bitcoin blockchain.
- This infrastructure is based on work done for decentralized identity at W3C and the Decentralized Identity Foundation.

Similar initiatives have also been taken by IBM and other organizations around the world.

Decentralized finance (DeFi)

- DeFi could be the killer app of the decentralized revolution that everyone has been waiting for.
- Traditionally, finance is a business that is almost impossible to do without the involvement of a trusted third party. It can either be a bank or some other financial firm; consumers have to trust a central authority to do business on their behalf and provide the services.
- There are rules, policies, procedures, and strict regulations that govern this ecosystem. This control and management is of paramount importance for the integrity of the entire financial ecosystem. Still, since there is always a central party that is required in every single transaction consumers do, this approach has some disadvantages.
- Some main disadvantages are listed as follows:
 - **Access barrier:** Access to financial services and banking requires a rigorous onboarding process involving KYC and other relevant checks and procedures. Even though this is extremely important for the integrity of the existing financial system, it can become a barrier at times for millions of unbanked people all around the world, especially in third world countries.
 - **High cost:** Financial services can be costly in some scenarios, especially investment related activities, and could be seen as a barrier toward entering the financial services industry.

- **Transparency issues:** There are concerns around transparency and trust due to the proprietary nature of the financial industry.

- **Siloed:** Most current financial industry solutions are proprietary and are owned by their respective organization. This results in interoperability issues where systems from one organization are unable to talk with another.

Challenges: DeFi comes with a number of advantages, such as inclusion, easy access, and cheaper services; however, it has its own challenges. These drawbacks must be addressed for further adoption of this novel paradigm of financial services:

- **Underdeveloped ecosystem:** This ecosystem is still developing and requires more effort to improve usability and adoption.

- **Too technical:** As some users may find understanding and handling all financial transactions and jargon a bit daunting, adopting a DeFi platform may take some time and educational effort.

- **Lack of regulation:** This is a genuine and clear concern because without the existence of any regulatory framework, this ecosystem can be used for illegal activities. Moreover, trusting the code alone, instead of traditional paper contracts with established financial institutions, is seen as a major problem, especially by consumers who are used to traditional financial systems.

- **Human error:** On a blockchain, human errors can result in serious implications. Especially on DeFi systems or cryptocurrency blockchains, any human negligence can result in serious implications, such as financial loss. Moreover, due to the lack of regulation (relative to traditional finance), such an issue can have an even greater detrimental impact.

- DeFi applications are used in a broad spectrum of use cases, including loans, decentralized exchanges (DEXs), derivatives, payments, insurance, and assets. All of these use cases have one thing in common, which is that there is no central authority in control and participants (that is, users) on DeFi platforms conduct business directly with each other without the involvement of any intermediaries.

Cryptography

You will also be introduced to applications of cryptographic algorithms so that you can gain hands-on experience in the practical implementation of cryptographic functions.

Working with the OpenSSL command line

On the Ubuntu Linux distribution, OpenSSL is usually already available. However, it can be installed using the following commands:

```
$ sudo apt-get install openssl
```

Examples in this chapter have been developed using OpenSSL version 1.0.2t. It is available at <https://packages.ubuntu.com/xenial/openssl>. You are encouraged to use this specific version, as all examples in the chapter have been developed and tested with it. The OpenSSL version can be checked using the following command:

```
$ openssl version
```

If you see the following output,

```
OpenSSL 1.0.2t 10 Sep 2019
```

then you are all set to run the examples provided in this chapter. If you are running a version other than 1.0.2t, the examples may still work but that is not guaranteed, especially as older versions lack the features used in the examples and newer versions may not be backward compatible with version 1.0.2t.

Introduction:

- **Cryptography** is the science of making information secure in the presence of adversaries. It does so under the assumption that limitless resources are available to adversaries.
- **Ciphers** are algorithms used to encrypt or decrypt data so that if intercepted by an adversary, the data is meaningless to them without decryption, which requires a secret key.
- Cryptography is primarily used to provide a confidentiality service. On its own, it cannot be considered a complete solution, rather it serves as a crucial building block within a more extensive security system to address a security problem.
- For example, securing a blockchain ecosystem requires many different cryptographic primitives, such as hash functions, symmetric key cryptography, digital signatures, and public key cryptography.
- In addition to a confidentiality service, cryptography also provides other security services such as integrity, authentication (entity authentication and data origin authentication), and non repudiation. Additionally, accountability is also provided, which is a requirement in many security systems.

A generic cryptographic model is shown in the following diagram:

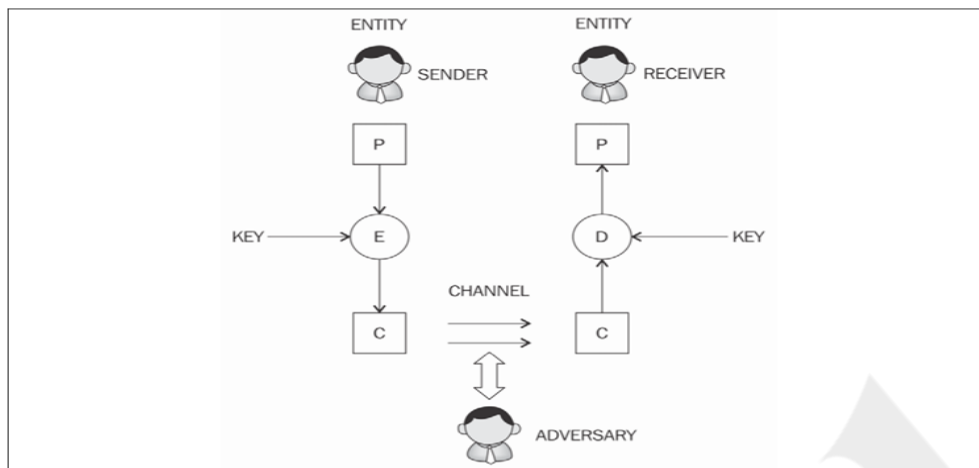


Figure 3.1: A model of the generic encryption and decryption model

In the preceding diagram, P, E, C, and D represent plaintext, encryption, ciphertext, and decryption, respectively. This model utilizes concepts such as entities, senders, receivers, adversaries, keys, and channels.

These concepts are explained as follows:

- Entity: Either a person or system that sends, receives, or performs operations on data
- Sender: This is an entity that transmits the data
- Receiver: This is an entity that takes delivery of the data
- Adversary: This is an entity that tries to circumvent the security service
- Key: A key is data that is used to encrypt the plaintext and also to decrypt the ciphertext
- Channel: A channel provides a medium of communication between entities.

➤ Confidentiality

- Confidentiality is the assurance that information is only available to authorized entities.

➤ Integrity

- Integrity is the assurance that information is modifiable only by authorized entities.

➤ Authentication

- Authentication provides assurance about the identity of an entity or the validity of a message. There are two types of authentication mechanisms, namely, entity authentication and data origin authentication,

1.Entity authentication

- Entity authentication is the assurance that an entity is currently involved and active in a communication session. Traditionally, users are issued a username and password that is used to gain access to the various platforms with which they are working. This practice is known

as single-factor authentication, as there is only one factor involved, namely, something you know; that is, the password and username.

- This type of authentication is not very secure for a variety of reasons, for example, password leakage; therefore, additional factors are now commonly used to provide better security. The use of additional techniques for user identification is known as multi-factor authentication (or two factor authentication if only two methods are used).
- The first method uses something you have, such as a hardware token or a smart card. In this case, a user can use a hardware token in addition to login credentials to gain access to a system. This mechanism protects the user by requiring two factors of authentication. A user who has access to the hardware token and knows the login credentials will be able to access the system. Both factors should be available to gain access to the system, thus making this method a two-factor authentication mechanism. If the hardware token is inaccessible (for example, lost or stolen) or the login password is forgotten by the user, access to the system will be denied. The hardware token won't be of any use on its own unless the login password (something you know) is also known and used in conjunction with the hardware token.
- The second method uses something you are, which uses biometric features to identify the user. With this method, a user's fingerprint, retina, iris, or hand geometry is used to provide an additional factor for authentication. This way, it can be ensured that the user was indeed present during the authentication process, as biometric features are unique to every individual. However, careful implementation is required to guarantee a high level of security, as some research has suggested that biometric systems can be circumvented under specific conditions.

2. Data origin authentication Also known as message authentication, data origin authentication is an assurance that the source of the information is indeed verified. Data origin authentication guarantees data integrity because, if a source is corroborated, then the data must not have been altered. Various methods, such as Message Authentication Codes (MACs) and digital signatures, are most commonly used.

- **Non-repudiation** Non-repudiation is the assurance that an entity cannot deny a previous commitment or action by providing incontrovertible evidence. This is a security service that offers definitive proof that a particular activity has occurred. This property is essential in debatable situations whereby an entity has denied the actions performed, for example, the placement of an order on an e-commerce system. This service produces cryptographic evidence in

electronic transactions so that in case of disputes, it can be used as confirmation of an action

- The non-repudiation protocol usually runs in a communications network, and it is used to provide evidence that an action has been taken by an entity (originator or recipient) on the network. In this context, two communications models can be used to transfer messages from originator A to recipient B:
 - A message is sent directly from originator A to recipient B; and
 - A message is sent to a delivery agent from originator A, which then delivers the message to recipient B
- The primary requirements of a non-repudiation protocol are fairness, effectiveness, and timeliness.
- In many scenarios, there are multiple participants involved in a transaction, as opposed to only two parties. For example, in electronic trading systems, there can be many entities, including clearing agents, brokers, and traders, who can be involved in a single transaction. In this case, two-party non-repudiation protocols are not appropriate. To address this problem, multi-party non-repudiation (MPNR) protocols have been developed. MPNR refers to non-repudiation protocols that run between multiple parties instead of the traditional two parties.

➤ **Accountability**

- Accountability is the assurance that actions affecting security can be traced back to the responsible party. This is usually provided by logging and audit mechanisms in systems where a detailed audit is required due to the nature of the business,
- for example, in electronic trading systems. Detailed logs are vital to trace an entity's actions, such as when a trade is placed in an audit record with the date and timestamp and the entity's identity is generated and saved in the log file. This log file can optionally be encrypted and be part of the database or a standalone ASCII text log file on a system.

➤ **CRYPTOGRAPHIC PRIMITIVES**

Cryptographic primitives are the basic building blocks of a security protocol or system.

- A security protocol is a set of steps taken to achieve the required security goals by utilizing appropriate security mechanisms. Various types of security protocols are in use, such as authentication protocols, non-repudiation protocols, and key management protocols

The taxonomy of cryptographic primitives can be visualized as shown here:

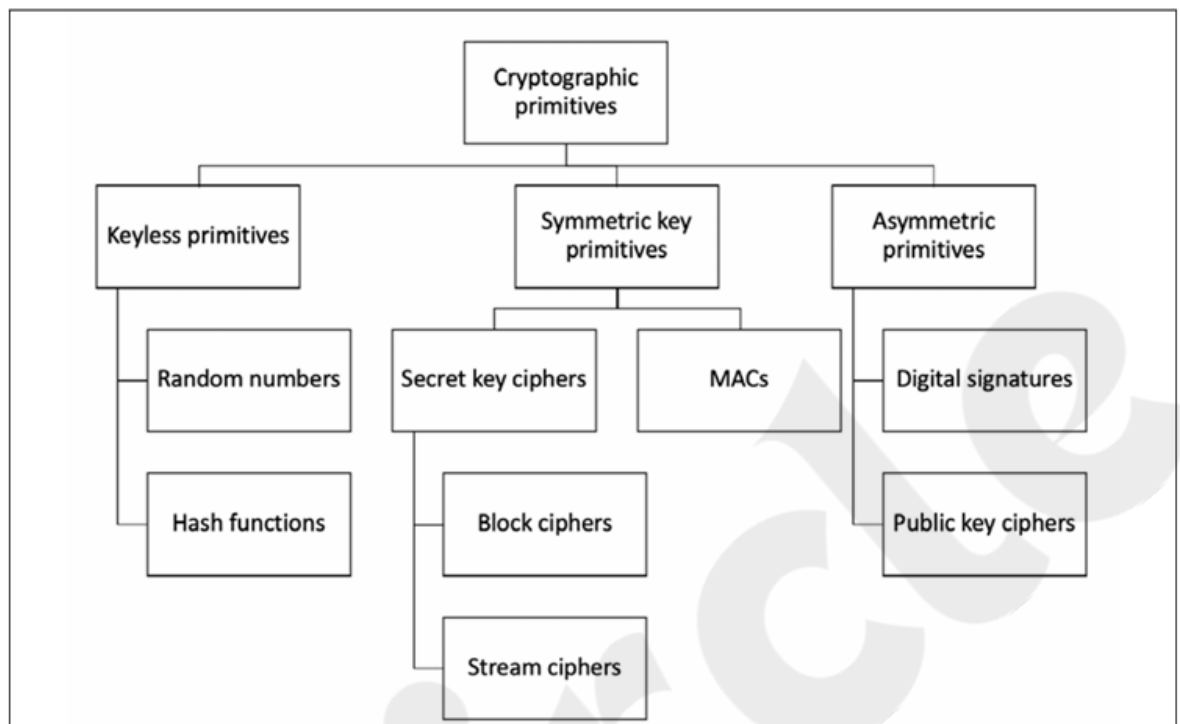


Figure 3.2: Cryptographic primitives

As shown in the preceding cryptographic primitive taxonomy diagram, cryptography is mainly divided into three categories: keyless primitives, symmetric key primitives, or cryptography, and asymmetric key primitives, or cryptography.

1. Keyless primitives

- There are two keyless primitives, namely, **random numbers** and **hash functions**.
 - a. **Random numbers** Randomness provides an indispensable element for the security of the cryptographic protocols. It is used for the generation of keys and in encryption algorithms.
- Randomness ensures that operations of a cryptographic algorithm do not become predictable enough to allow cryptanalysts to predict the outputs and operations of the algorithm, which will make the algorithm insecure. It is quite a feat to generate suitable randomness with a high degree of uncertainty, but there are methods that ensure an adequate level of randomness is generated for use in cryptographic algorithms.
- There are two categories of source of randomness, namely,) **Random Number Generators (RNGs) and Pseudorandom Number Generators (PRNGs)**
 - i. **RNGs** RNGs are software or hardware systems that make use of the randomness available in the real world, that is, the analog world, where uncertainty in the environment produces randomness.
- This can be temperature variations, thermal noises from various electronic components, or acoustic noise. This is called real randomness. Other sources

are based on the fact that a running computer system generates some randomness from the running processes, such as keystrokes or disk movements. These types of sources of randomness are not very practical due to the difficulty of acquiring this data or not having enough entropy. Also, these sources are not always available and could be available only for a limited time.

- *The measure of randomness is called **entropy**.*
- ii. **PRNGs** PRNGs are deterministic functions that work on the principle of using a random initial value called a seed to produce a random looking set of elements. PRNGs are commonly used to generate keys for encryption algorithms.
- A common example of a PRNG is the Blum-Blum Shub (BBS). PRNGs are a better alternative to RNGs due to their reliability and deterministic nature.

b. Hash functions

- Hash functions are used to create fixed-length digests of arbitrarily long input strings. Hash functions are keyless, and they provide a data integrity service. They are usually built using iterated and dedicated hash function construction techniques.
- Various families of hash functions are available, such as MD, SHA1, SHA-2, SHA-3, RIPEMD, and Whirlpool. Hash functions are commonly used for digital signatures and message authentication codes, such as HMACs. They have three security properties, namely, pre-image resistance, second pre-image resistance, and collision resistance. Hash functions are also typically used to provide data integrity services. These can be used both as one-way functions and to construct other cryptographic primitives, such as MACs and digital signatures. Some applications use hash functions as a means for generating PRNGs. There are two practical and three security properties of hash functions that must be met depending on the level of integrity required.
- **Compression of arbitrary messages into fixed-length digests** This property relates to the fact that a hash function must be able to take an input text of any length and output a fixed-length compressed message. Hash functions produce a compressed output in various bit sizes, usually between 128-bits and 512-bits.
- **Easy to compute** Hash functions are efficient and fast one-way functions. It is required that hash functions be very quick to compute regardless of the message size. The efficiency may decrease if the message is too big, but the function should still be fast enough for practical use.
- The following **are the security properties of the hash function**:
 - i. **Pre-image resistance** This property can be explained by using the simple equation: $h(x) = y$ Here, h is the hash function, x is the input, and y is the hash. The first security property requires that y cannot be reverse-computed to x .

is considered a pre-image of y , hence the name pre image resistance. This is also called a one-way property.

ii. pre-image resistance The second pre-image resistance property requires that given x and $h(x)$, it is almost impossible to find any other message m , where $m \neq x$ and hash of m = hash of x or $h(m) = h(x)$. This property is also known as **weak collision resistance**

iii. Collision resistance The collision resistance property requires that two different input messages should not hash to the same output. In other words, $h(x) \neq h(z)$. This property is also known as **strong collision resistance**.

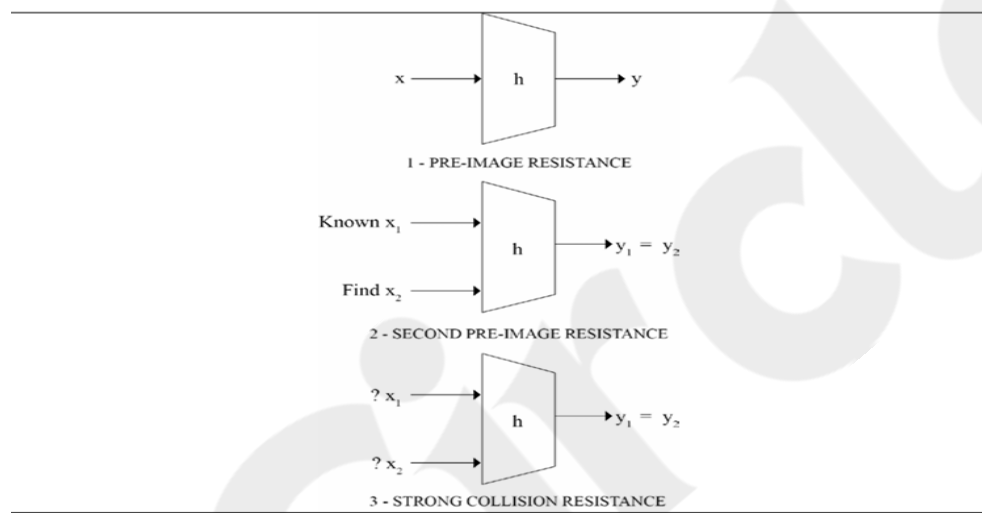


Figure 3.3: Three security properties of hash functions

- Due to their very nature, hash functions will always have some collisions. This is a situation where two different messages hash to the same output. However, they should be computationally impractical to find. A concept known as the avalanche effect is desirable in all cryptographic hash functions. **The avalanche effect** specifies that a small change, even a single character change in the input text, will result in an entirely different hash output.
- Hash functions are usually designed by following an **iterated hash functions approach**.
- With this method, the input message is compressed in multiple rounds on a block-by-block basis in order to produce the compressed output. A popular type of iterated hash function is the **Merkle-Damgard construction**. This construction is based on the idea of dividing the input data into equal block sizes and then feeding them through the compression functions in an iterative manner. The collision resistance of the property of compression functions ensures that the hash output is also collision-resistant. Compression functions can be built using block ciphers. In addition to Merkle-Damgard, there are various other constructions of compression functions proposed by researchers, for example, Miyaguchi-Preneel and Davies-Meyer.

- **Multiple categories of hash functions** are introduced in the below section. **Message Digest Message Digest (MD) functions** were prevalent in the early 1990s. MD4 and MD5 fall into this category. Both MD functions were found to be insecure and are not recommended for use anymore. MD5 is a 128-bit hash function that was commonly used for file integrity checks.
- **Secure Hash Algorithms** The following list describes the most common Secure Hash Algorithms (SHAs):
- **SHA-0:** This is a 160-bit function introduced by the U.S. National Institute of Standards and Technology (NIST) in 1993.
- **SHA-1:** SHA-1 was introduced in 1995 by NIST as a replacement for SHA-0. This is also a 160-bit hash function. SHA-1 is used commonly in SSL and TLS implementations. It should be noted that SHA-1 is now considered insecure, and it is being deprecated by certificate authorities. Its usage is discouraged in any new implementations.
- **SHA-2:** This category includes four functions defined by the number of bits of the hash: SHA-224, SHA-256, SHA-384, and SHA-512. SHA-3: This is the latest family of SHA functions. SHA3-224, SHA3-256, SHA3-384, and SHA3-512 are members of this family. SHA3 is a NIST-standardized version of Keccak.
- Keccak uses a new approach called sponge construction instead of the commonly used Merkle Damgard transformation.
- **RIPEMD:** RIPEMD is the acronym for RACE Integrity Primitives Evaluation Message Digest. It is based on the design ideas used to build MD4. There are multiple versions of RIPEMD, including 128-bit, 160-bit, 256-bit, and 320-bit.
- **Whirlpool:** This is based on a modified version of the Rijndael cipher known as W. It uses the Miyaguchi-Preneel compression function, which is a type of one-way function used for the compression of two fixed-length inputs into a single fixed-length output. It is a single block length compression function. Hash functions have many practical applications ranging from simple file integrity checks and password storage to use in cryptographic protocols and algorithms. They are used in hash tables, distributed hash tables, bloom filters, virus fingerprinting, peer-to-peer (P2P) file-sharing, and many other applications.
- Hash functions play a vital role in blockchain. The Proof-of-Work (PoW) function in particular uses SHA-256 twice in order to verify the computational effort spent by miners. RIPEMD 160 is used to produce Bitcoin addresses.

DESIGN OF SECURE HASH ALGORITHMS (SHA)

Design of SHA-256

- SHA-256 has an input message size limit of $2^{64} - 1$ bits. The block size is 512 bits, and it has a word size of 32 bits. The output is a 256-bit digest. The compression function processes a 512-bit message block and a 256-bit intermediate hash value.

- There are two main components of this function: **the compression function and a message schedule.**
- The algorithm works as follows, in nine steps:

Pre-processing

1. Padding of the message is used to adjust the length of a block to 512 bits if it is smaller than the required block size of 512 bits.
2. Parsing the message into message blocks, which ensures that the message and its padding is divided into equal blocks of 512 bits.
3. Setting up the initial hash value, which consists of the eight 32-bit words obtained by taking the first 32 bits of the fractional parts of the square roots of the first eight prime numbers. These initial values are fixed and chosen to initialize the process. They provide a level of confidence that no backdoor exists in the algorithm.

Hash computation

4. Each message block is then processed in a sequence, and it requires 64 rounds to compute the full hash output. Each round uses slightly different constants to ensure that no two rounds are the same.
5. The message schedule is prepared.
6. Eight working variables are initialized.
7. The compression function runs 64 times.
8. The intermediate hash value is calculated.
9. Finally, after repeating steps 5 through 8 until all blocks (chunks of data) in the input message are processed, the output hash is produced by concatenating intermediate hash values

At a high level, SHA-256 can be visualized in the following diagram:

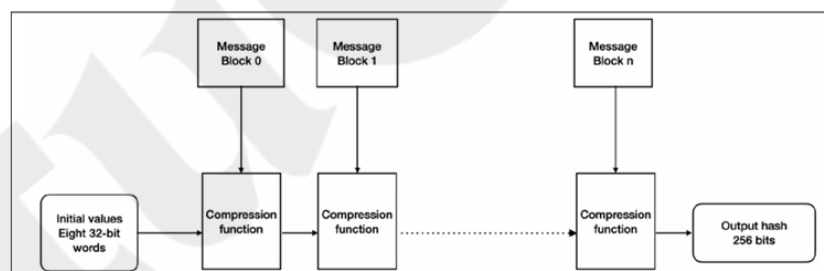


Figure 3.4: SHA-256 high level overview

SHA-256 is a **Merkle Damgard construction** that takes the input message and divides it into equal blocks (chunks of data) of 512 bits. Initial values (or initial hash values) or the initialization vector are composed of eight 32bit words (256 bits) that are fed into the compression function with the first message. Subsequent blocks are fed into the compression function until all blocks are processed and finally, the output hash is produced. The compression function of SHA-256 is shown in the following diagram.

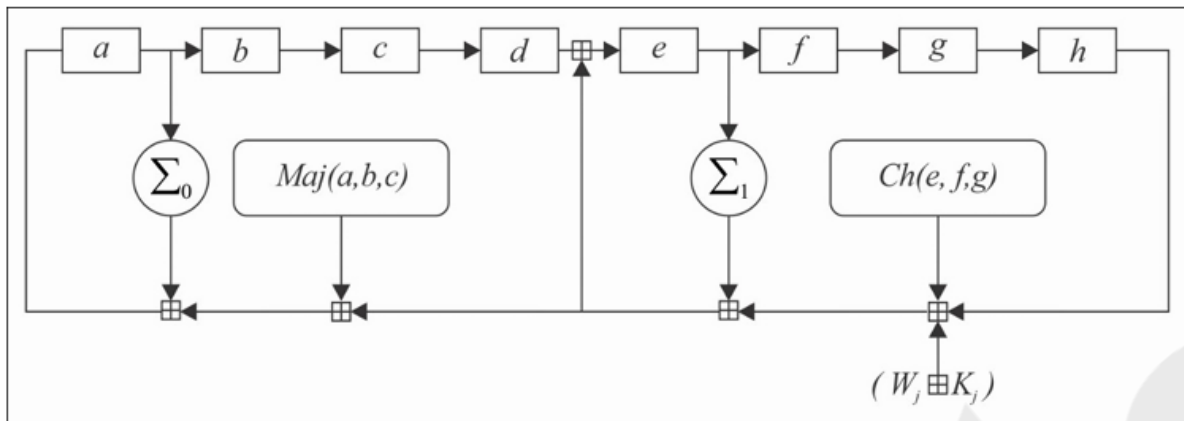


Figure 3.5: One round of an SHA-256 compression function

In the preceding diagram, a , b , c , d , e , f , g , and h are the registers for 8 working variables. Maj and Ch functions are applied bitwise. Σ_0 and Σ_1 perform bitwise rotation. The round constants are W_j and K_j , which are added in the main loop (compressor function) of the hash function, which runs 64 times.

Design of SHA-3 (Keccak)

- The structure of SHA-3 is very different from that of SHA-1 and SHA-2. The key idea behind SHA-3 is based on unkeyed permutations, as opposed to other typical hash function constructions that used keyed permutations.
- Keccak also does not make use of the Merkle Damgard transformation that is commonly used to handle arbitrary-length input messages in hash functions. A newer approach, called **sponge and squeeze construction**, is used in Keccak.
- It is a random permutation model. Different variants of SHA-3 have been standardized, such as SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, and SHAKE256. SHAKE128 and SHAKE256 are extendable-output functions (XOFs), which allow the output to be extended to any desired length.
- The following diagram shows the sponge and squeeze model, which is the basis of SHA-3 or Keccak. Analogous to a sponge, the data (m input data) is first absorbed into the sponge after applying padding. It is then changed into a subset of permutation state using XOR (exclusive OR), and then the output is squeezed out of the sponge function that represents the transformed state. The rate r is the input block size of the sponge function, while capacity c determines the security level.

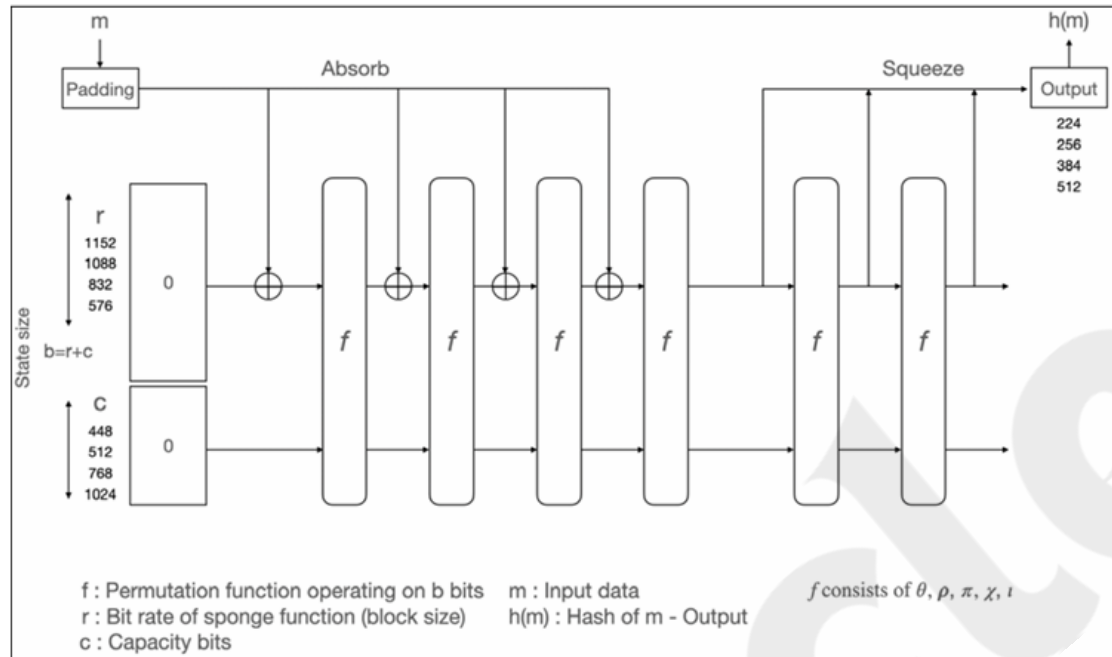


Figure 3.6: The SHA-3 absorbing and squeezing function in SHA3

In the preceding diagram, state size b is calculated by adding bit rate r and capacity bits c . r and c can be any values as long as sizes of $r + c$ are 25, 50, 100, 200, 400, 800, or 1600. The state is a 3-dimensional bit matrix. The initial state is set to 0. The data m is entered into the absorb phase block by block via XOR $\oplus$ after applying padding.

- The following table shows the value of bit rate r (block size) and capacity c required to achieve the desired output hash size under the most efficient setting of $r + c = 1600$:

r (block size)	c (capacity)	Output hash size
1152	448	224
1088	512	256
832	768	384
576	1024	512

The function f is a permutation function. It contains five transformation operations named Theta, Rho, Pi, Chi, and Iota described as follows.

θ - Theta: XOR bits in the state, used for mixing

ρ - Rho: Diffusion function performing rotation of bits

π - Pi: Diffusion function

χ - Chi: XOR each bit, bitwise combine

ι - Iota: Combination with round constants

The details of transformation operations is beyond the scope of this book. However, a high level function of each transformation is described previously. The key idea is to apply these transformations to achieve the **avalanche effect**, which we introduced earlier in this chapter. These five operations combined form a **round**. In the SHA-3 standard, the number of rounds is 24 to achieve the desired level of security.

OpenSSL example of hash functions

The following command will produce a hash of 256 bits of Hello messages using the SHA-256 algorithm:

```
$ echo -n 'Hello' | openssl dgst -sha256
(stdin)= 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Note that even a small change in the text, such as changing the case of the letter H, results in a big change in the output hash.



This phenomenon is known as the avalanche effect, where if the input is slightly changed, the output changes drastically so that cryptanalysts cannot make predictions on the input text by analyzing the output.

Now we run the following command to see the avalanche effect in action:

```
$ echo -n 'hello' | openssl dgst -sha256
(stdin)= 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
```

Note that both outputs are completely different:

```
Hello: 18:5f:8d:b3:22:71:fe:25:f5:61:a6:fc:93:8b:2e:26:43:06:ec:30:4e:
da:51:80:07: d1:76:48:26:38:19:69

hello: 2c:f2:4d:ba:5f:b0:a3:0e:26:e8:3b:2a:c5:b9:e2:9e:1b:16:1e:5c:1f:a7:42:
5e:73:
04:33:62:93:8b:98:24
```

Usually, hash functions do not use a key. Nevertheless, if they are used with a key, then they can be used to create another cryptographic construct called **message authentication codes (MACs)**.

We will see how MACs and HMACs work after we've introduced symmetric key cryptography, because these constructions make use of keys for encryption.

SYMMETRIC CRYPTOGRAPHY

- Symmetric cryptography refers to a type of cryptography where the key that is used to encrypt the data is the same one that is used for decrypting the data. Thus, it is also known as **shared key cryptography**.
- The key must be established or agreed upon before the data exchange occurs between the communicating parties. This is the reason it is also called **secret key cryptography**. A key, more precisely, a cryptographic key, is required for the encryption and decryption process and must be kept secret, hence it is called a secret key. There are many different types of keys depending upon the protocol.
- In symmetric key systems, only one key is required, which is shared between the sender and the receiver. It is used for both encryption and decryption, hence the name symmetric key cryptography or shared key cryptography, or just symmetric cryptography.

- Other types of keys are public keys and private keys, which are generated in pairs for **public key cryptography or asymmetric cryptography**. Public keys are used for encrypting the plaintext, whereas private keys are used for decryption and are expected to be kept secret by the receiver.
- Keys can also be ephemeral (temporary) or static. Ephemeral keys are intended to be used only for a short period of time, such as in a single session between the participants, whereas static keys are intended for long-term usage. Another type of key is called the master key, which is used for the protection, encryption, decryption, and generation of other keys.
- **There are different methods to generate keys.** These methods are listed as follows:
 1. **Random**, where a random number generator is used to generate a random set of bytes that can be used as a key.
 2. **Key derivation-based**, where a single key or multiple keys are derived from a password. A key derivation function (KDF) is used for this purpose, taking the password as input, and converting that into a key. Commonly used key derivation functions are Password Based Key Derivation Function 1 (PBKDF1), PBKDF2, Argon 2, and Scrypt.
- Using a key agreement protocol, two or more participants run a protocol that produces a key and that key is then shared between participants. In key agreement schemes, all participants contribute equally in the effort to generate the shared secret key. The most commonly used key agreement protocol is the Diffie-Hellman key exchange protocol.
- In encryption schemes, there are also some random numbers that play a vital role in the operation of the encryption process. These concepts are presented as follows:
 - **The nonce:** This is a number that can be used only once in a cryptographic protocol. It must not be reused. Nonces can be generated from a large pool of random numbers or they can also be sequential. The most common use of nonces is to prevent replay attacks in cryptographic protocols.
 - **The initial value or initialization vector (IV)** is a random number, which is basically a nonce, but it must be chosen in an unpredictable manner. This means that it cannot be sequential. IVs are used extensively in encryption algorithms to provide increased security.
 - **The salt:** Salt is a cryptographically strong random value that is typically used in hash functions to provide defense against dictionary or rainbow attacks. Using dictionary attacks, hashing-based password schemes can be broken by trying hashes of millions of words from a dictionary in a brute-force manner and matching it with the hashed password. If a salt is used, then a dictionary attack becomes difficult to run because a random salt makes each password unique, and secondly, the attacker will then have to run a separate dictionary attack for random salts, which is quite unfeasible.

Message authentication codes (MACs)

- Message authentication codes (MACs) are sometimes called **keyed hash functions**, and they can be used to provide message integrity and authentication. More specifically, they are used to provide data origin authentication. These are symmetric

cryptographic primitives that use a shared key between the sender and the receiver. MACs can be constructed using block ciphers or hash functions.

- **Hash-based MACs (HMACs)** Similar to the hash function, hash-based MACs (HMACs) produce a fixed-length output and take an arbitrarily long message as the input. In this scheme, the sender signs a message using the MAC and the receiver verifies it using the shared key. The key is hashed with the message using either of the two methods known as secret prefix or secret suffix. With the secret prefix method, the key is concatenated with the message; that is, the key comes first and the message comes afterward, whereas with the secret suffix method, the key comes after the message, as shown in the following equations:

Secret prefix: $M = \text{MAC}_K(x) = h(k || x)$

Secret suffix: $M = \text{MAC}_K(x) = h(x || k)$

There are pros and cons to both methods. Some attacks on both schemes have occurred. There are HMAC constructions schemes that use various techniques, such as ipad (inner padding) and opad (outer padding) that have been proposed by cryptographic researchers. These are considered secure with some assumptions:

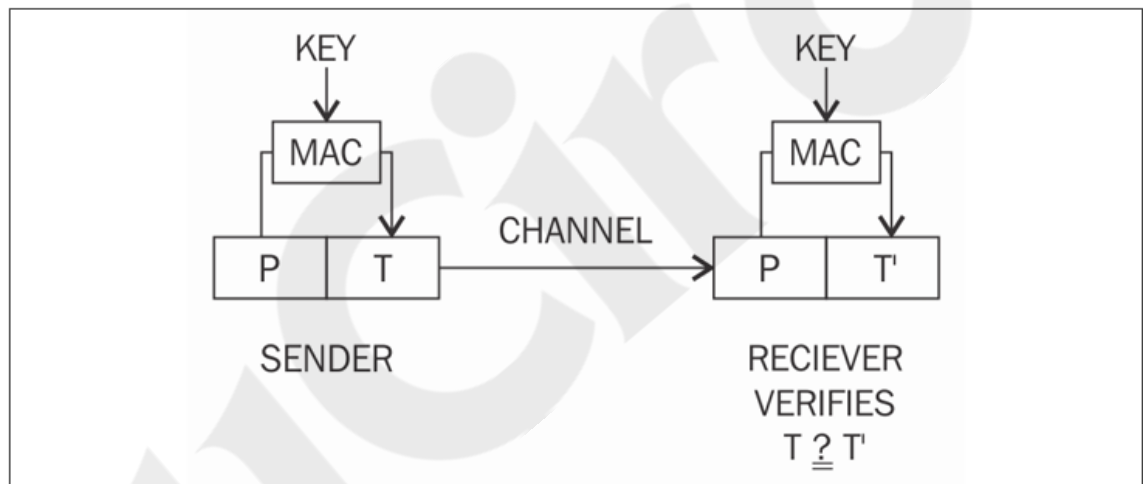


Figure 3.7: Operation of a MAC function

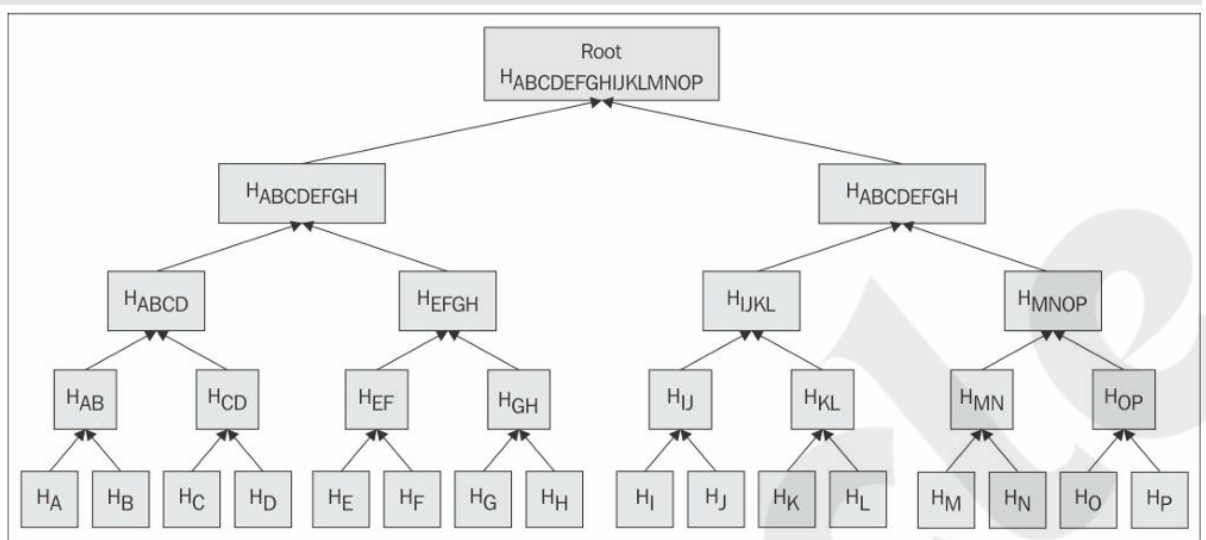
There are various powerful applications of hash functions used in peer-to-peer networks and blockchain technologies, such as Merkle trees, Patricia trees, and distributed hash tables.

Various applications of Hash functions [contd...]

Merkle trees

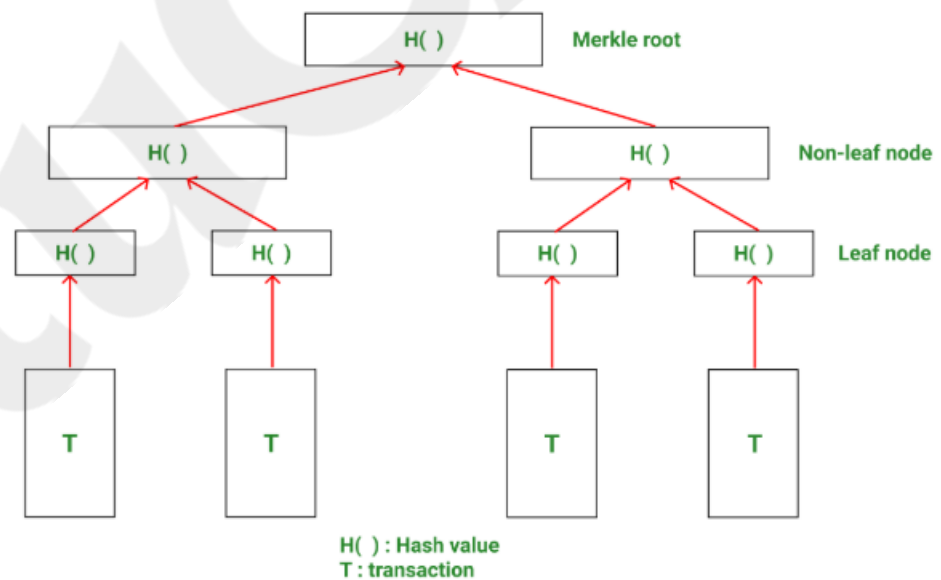
The concept of Merkle tree was introduced by Ralph Merkle. A visualization of Merkle tree is shown here, which makes it easy to understand. Merkle trees allow secure and efficient verification of large data sets. It is a binary tree in which first, the inputs are placed at the leaves (node with no children), and then values of pairs of child nodes are hashed together in order to produce a value for the parent node (internal node) until a single hash value known as Merkle root is achieved.

An example of a Merkle tree:



A Merkle tree

Merkle Tree Structure:



Structure of Merkle tree

1. A blockchain can potentially have thousands of blocks with thousands of transactions in each block. Therefore, memory space and computing power are two main challenges.

2. It would be optimal to use as little data as possible for verifying transactions, which can reduce CPU processing and provide better security, and this is exactly what Merkle trees offer.

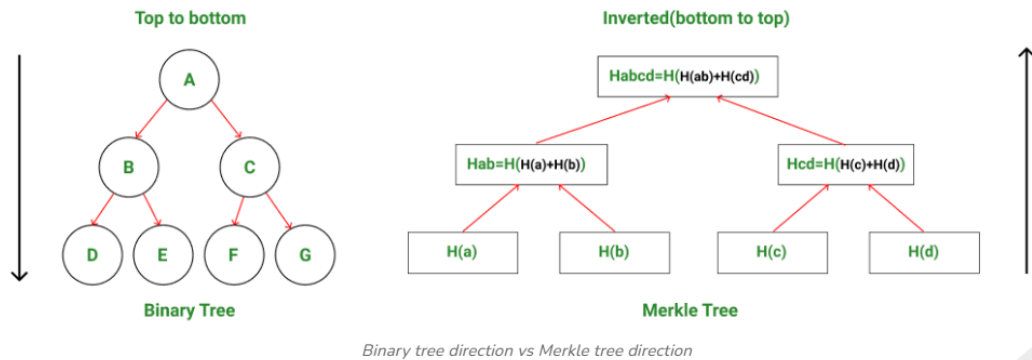
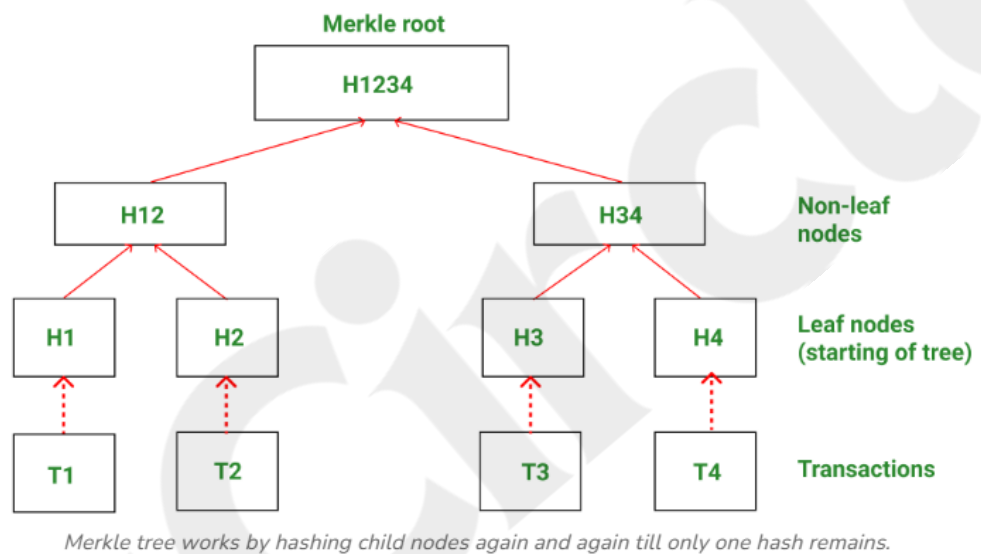
3. In a Merkle tree, transactions are grouped into pairs. The hash is computed for each pair and this is stored in the parent node. Now the parent nodes are grouped into pairs and their hash is stored one level up in the tree. This continues till the root of the tree.

The different types of nodes in a Merkle tree are:

- **Root node:** The root of the Merkle tree is known as the Merkle root and this Merkle root is stored in the header of the block.
 - **Leaf node:** The leaf nodes contain the hash values of transaction data. Each transaction in the block has its data hashed and then this hash value (also known as transaction ID) is stored in leaf nodes.
 - **Non-leaf node:** The non-leaf nodes contain the hash value of their respective children. These are also called intermediate nodes because they contain the intermediate hash values and the hash process continues till the root of the tree.
4. Bitcoin uses the SHA-256 hash function to hash transaction data continuously till the Merkle root is obtained.
5. Further, a Merkle tree is binary in nature. This means that the number of leaf nodes needs to be even for the Merkle tree to be constructed properly. In case there is an odd number of leaf nodes, the tree duplicates the last hash and makes the number of leaf nodes even.

How do Merkle trees Work?

- A Merkle tree is constructed from the leaf nodes level all the way up to the Merkle root level by grouping nodes in pairs and calculating the hash of each pair of nodes in that particular level. This hash value is propagated to the next level. This is a bottom-to-up type of construction where the hash values are flowing from down to up direction.
- Hence, by comparing the Merkle tree structure to a regular binary tree data structure, one can observe that Merkle trees are actually inverted down.[refer above merkle tree example]
-

**Example2:**

Consider a block having 4 transactions- T1, T2, T3, T4. These four transactions have to be stored in the Merkle tree and this is done by the following steps-

Step 1: The hash of each transaction is computed.

$$H1 = \text{Hash}(T1).$$

Step 2: The hashes computed are stored in leaf nodes of the Merkle tree.

Step 3: Now non-leaf nodes will be formed. In order to form these nodes, leaf nodes will be paired together from left to right, and the hash of these pairs will be calculated. Firstly, hash of H1 and H2 will be computed to form H12. Similarly, H34 is computed. Values H12 and H34 are parent nodes of H1, H2, and H3, H4 respectively. These are non-leaf nodes.

$$H12 = \text{Hash}(H1 + H2)$$

$$H34 = \text{Hash}(H3 + H4)$$

Step 4: Finally H1234 is computed by pairing H12 and H34. H1234 is the only hash remaining. This means we have reached the root node and therefore H1234 is the Merkle root.

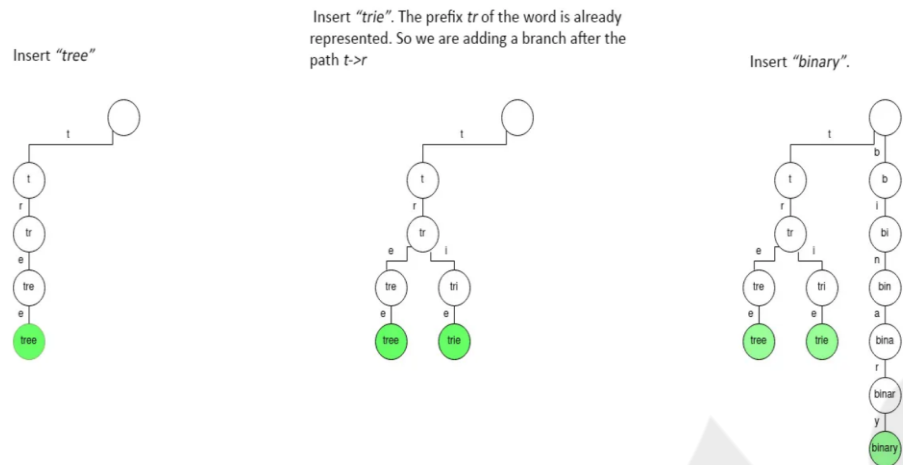
$$H1234 = \text{Hash}(H12 + H34)$$

Why Merkle Trees are Important For Blockchain?

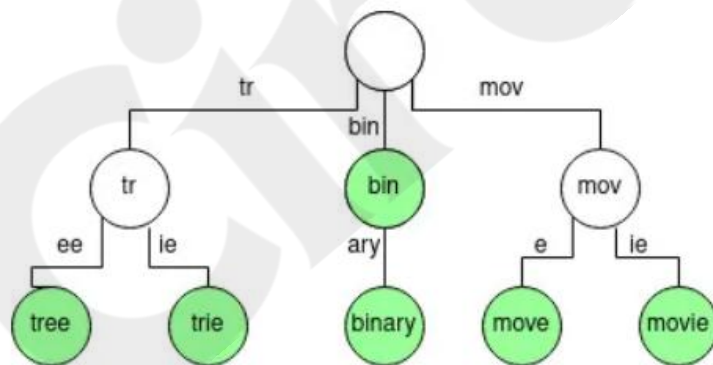
- In a centralized network, data can be accessed from one single copy. This means that nodes do not have to take the responsibility of storing their own copies of data and data can be retrieved quickly.
- However, the situation is not so simple in a distributed system.
- Let us consider a scenario where blockchain does not have Merkle trees. In this case, every node in the network will have to keep a record of every single transaction that has occurred because there is no central copy of the information.
- This means that a huge amount of information will have to be stored on every node and every node will have its own copy of the ledger. If a node wants to validate a past transaction, requests will have to be sent to all nodes, requesting their copy of the ledger. Then the user will have to compare its own copy with the copies obtained from several nodes.
- Any mismatch could compromise the security of the blockchain. Further on, such verification requests will require huge amounts of data to be sent over the network, and the computer performing this verification will need a lot of processing power for comparing different versions of ledgers.
- Without the Merkle tree, the data itself has to be transferred all over the network for verification.
- Merkle trees allow comparison and verification of transactions with viable computational power and bandwidth. Only a small amount of information needs to be sent, hence compensating for the huge volumes of ledger data that had to be exchanged previously.

PATRICIA TREES /TRIES

- In order to understand Patricia trees, first, you will be introduced to the concept of a trie.
- A trie or a digital tree is an ordered tree data structure used to store a dataset. Practical Algorithm to Retrieve Information Coded in Alphanumeric (Patricia), also known as Radix tree, is a compact representation of a trie in which a node that is the only child of a parent is merged with its parent.
- The trie (pronounced “try”) is widely used for storing string data sets. Though it was first introduced by Rene de la Briandais in 1959, the term “trie” was later coined by Edward Fredkin, derived from the middle syllable of the word “retrieval.”
- Merkle-Patricia tree, based on the definitions of Patricia and Merkle, is a tree that has a root node that contains the hash value of the entire data structure.



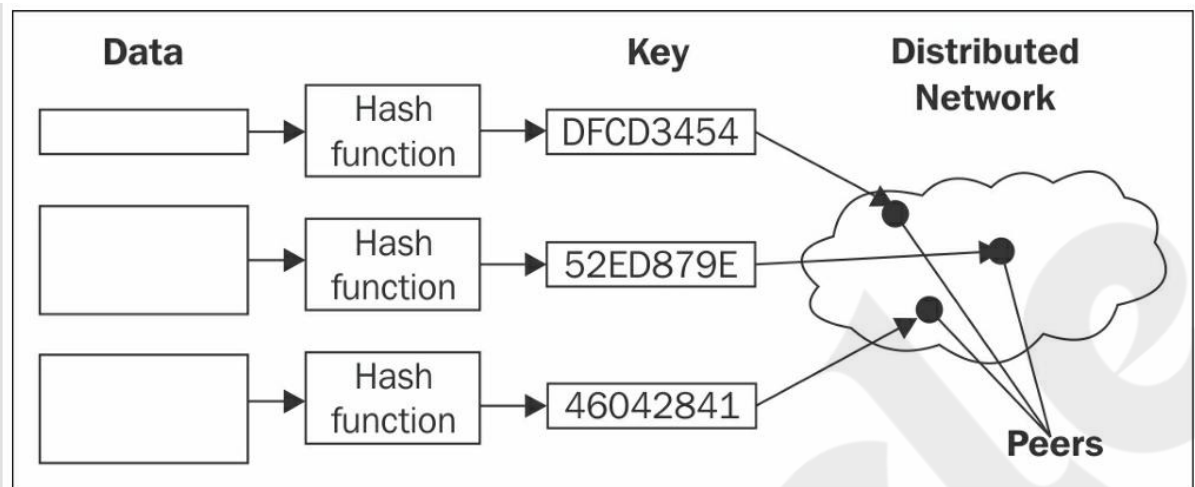
In a Patricia Trie, if a parent node has only a single child node, they will be combined. This is done to compress long paths without any branches into single edges. Finally, we get a trie where every node has at least two branches as shown below.



DISTRIBUTED HASH TABLES (DHTS)

- A hash table is a data structure that is used to map keys to values. Internally, a hash function is used to calculate an index into an array of buckets, from which the required value can be found. Buckets have records stored in them using a hash key and are organized in a particular order.
- With the definition provided earlier in mind, one can think of the distributed hash table as a data structure where data is spread across various nodes and nodes are equivalent to buckets in a peer-to-peer network.
- The following diagram visually shows how a DHT works. The example shows that data is passed through a hash function, which results in generating a compact key. This key is then linked with the data (values) on the peer-to-peer network. When users on the network request the data (via the filename), the filename can be hashed again to produce the same key and any node on the network can then be requested to find

the corresponding data. DHTs provides decentralization, fault tolerance, and scalability



Distributed hash tables

Symmetric cryptography

There are two types of symmetric ciphers: stream ciphers and block ciphers. Data Encryption Standard (DES) and Advanced Encryption Standard (AES) are typical examples of block ciphers, whereas RC4 and A5 are commonly used stream ciphers.

Stream ciphers Stream ciphers are encryption algorithms that apply encryption algorithms on a bit-by-bit basis (one bit at a time) to plaintext using a keystream.

- There are two types of stream ciphers: synchronous stream ciphers and asynchronous stream ciphers.
- Synchronous stream ciphers are those where the keystream is dependent only on the key.
- Asynchronous stream ciphers have a keystream that is also dependent on the encrypted data.
- In stream ciphers, encryption and decryption are the same function because they are simple modulo 2 additions or XOR operations. The fundamental requirement in stream ciphers is the security and randomness of keystreams. Various techniques ranging from PRNGs to true hardware RNGs have been developed to generate random numbers, and it is vital that all key generators be cryptographically secure.

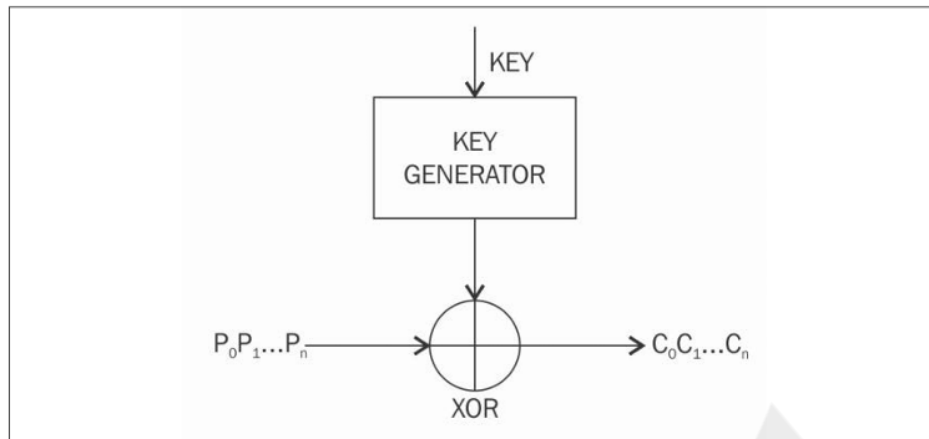


Figure 3.8: Operation of a stream cipher

Block ciphers

- Block ciphers are encryption algorithms that break up the text to be encrypted (plaintext) into blocks of a fixed length and apply the encryption block by block. Block ciphers are generally built using a design strategy known as a **Feistel cipher**. Recent block ciphers such as AES (Rijndael) have been built using a combination of substitution and permutation called a **Substitution-Permutation Network (SPN)**
- **Feistel ciphers** are based on the Feistel network, which is a structure developed by Horst Feistel. This structure is based on the idea of combining multiple rounds of repeated operations to achieve desirable cryptographic properties known as confusion and diffusion. Feistel networks operate by dividing data into two blocks (left and right) and processing these blocks via keyed round functions in iterations to provide sufficient pseudorandom permutations.
- **Confusion** adds complexity to the relationship between the encrypted text and plaintext. This is achieved by substitution. In practice, **A** in plaintext is replaced by **X** in encrypted text. In modern cryptographic algorithms, substitution is performed using lookup tables **called S-boxes**.
- The **diffusion property** spreads the plaintext statistically over the encrypted data. This ensures that even if a single bit is changed in the input text, it results in changing at least half (on average) of the bits in the ciphertext. Confusion is required to make finding the encryption key very difficult, even if many encrypted and decrypted data pairs are created using the same key. In practice, this is achieved by **transposition or permutation**.
- A key advantage of using a Feistel cipher is that encryption and decryption operations are almost identical and only require a reversal of the encryption process to achieve decryption. **DES** is a prime example of Feistel-based ciphers

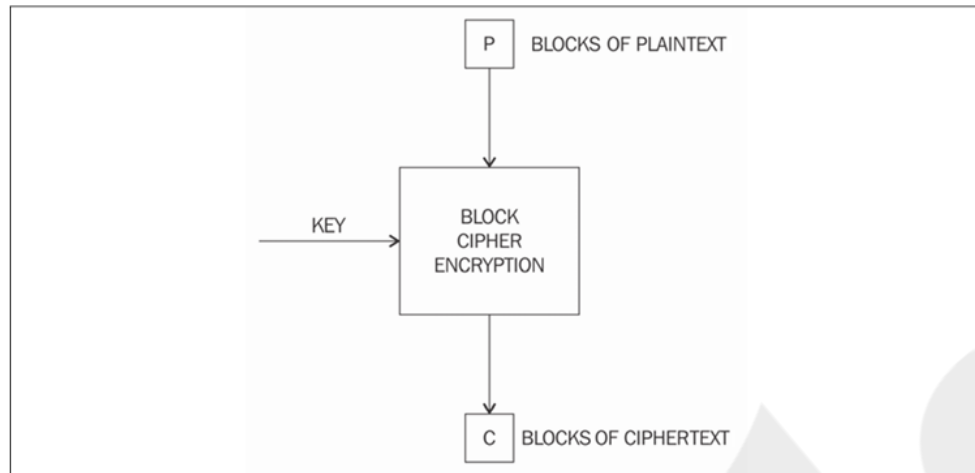


Figure 3.9: Simplified operation of a block cipher

- **Various modes of operation for block ciphers** are Electronic Code Book (ECB), Cipher Block Chaining (CBC), Output Feedback (OFB) mode, and Counter (CTR) mode. These modes are used to specify the way in which an encryption function is applied to the plaintext.

i. Block encryption mode

In block encryption mode, the plaintext is divided into blocks of fixed length depending on the type of cipher used. Then the encryption function is applied to each block.

ii. Electronic codebook

Electronic codebook (ECB) is a basic mode of operation in which the encrypted data is produced as a result of applying the encryption algorithm to each block of plaintext, one by one. This is the most straightforward mode, but it should not be used in practice as it is insecure and can reveal information:

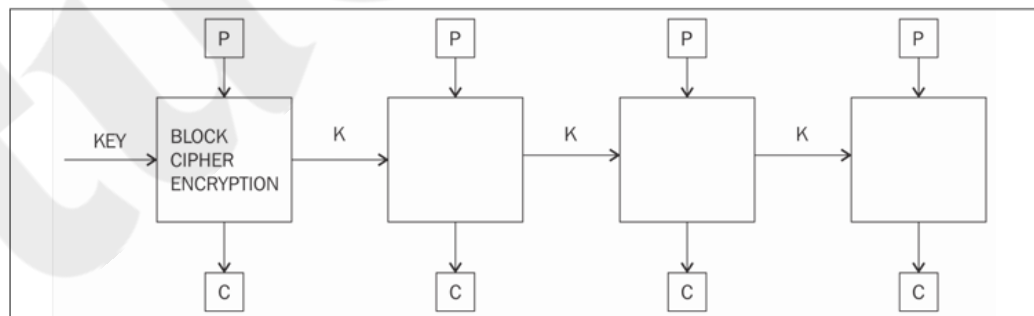


Figure 3.10: Electronic codebook mode for block ciphers

The preceding diagram shows that we have plain text P provided as an input to the block cipher encryption function along with a key, and ciphertext C is produced as output.

- **Cipher block chaining** In cipher block chaining (CBC) mode, each block of plaintext is XORed with the previously encrypted block. CBC mode uses the IV to encrypt the first block. It is recommended that the IV be randomly chosen:

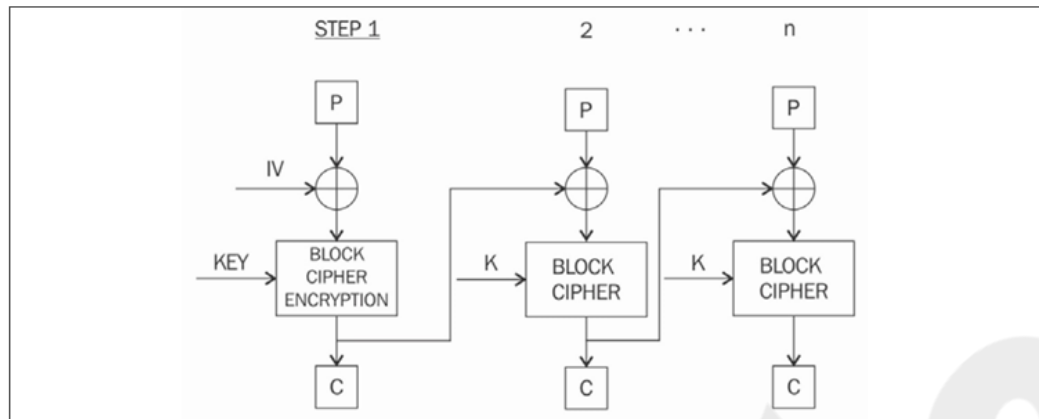


Figure 3.11: Cipher block chaining mode

Counter mode The counter (CTR) mode effectively uses a block cipher as a stream cipher. In this case, a unique nonce is supplied that is concatenated with the counter value to produce a keystream:

CTR mode, as shown in the preceding diagram, works by utilizing a nonce (N) and a counter (C) that feed into the block cipher encryption function. The block cipher encryption function takes the secret key (KEY) as input and produces a keystream (a stream of pseudorandom or random characters), which, when XORed with the plaintext (P), produces the ciphertext (C).

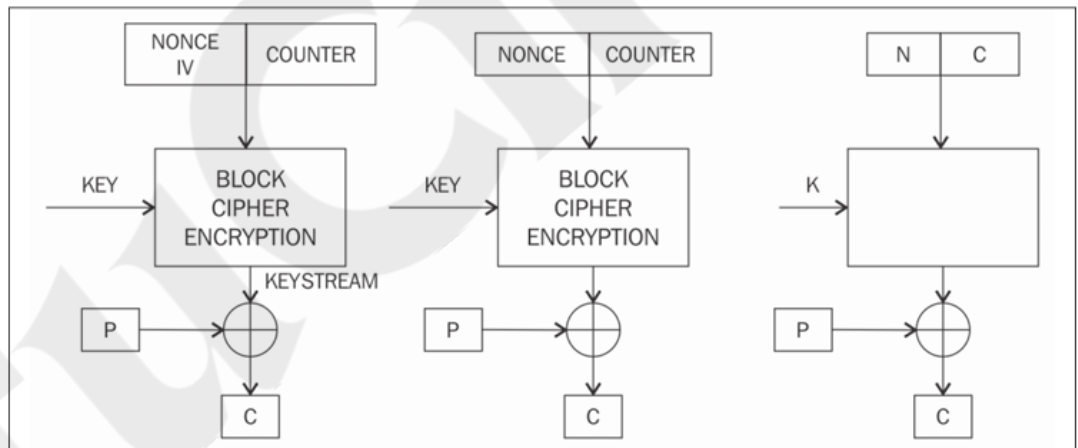


Figure 3.12: Counter mode

Keystream generation mode

In keystream generation mode, the encryption function generates a keystream that is used in stream ciphers. Specifically, the keystream is usually XORed with the plaintext stream to produce encrypted text.

Message authentication mode In message authentication mode, a message authentication code (MAC) is produced from an encryption function. A MAC is a cryptographic checksum that provides an integrity service. The most common method to generate a MAC using block ciphers is CBC-MAC, where a part of the last block of the chain is used as a MAC.

In other words, block ciphers are used in the cipher block chaining mode (CBC mode) to generate a MAC. A MAC can be used to check if a message has been

modified by an unauthorized entity. This can be achieved by encrypting the message with a key using the MAC function. The resulting message and the MAC of the message, once received by the receiver, are checked by encrypting the message received, again with the key, and comparing it with the MAC received from the sender. If they both match, then it means that the message has not been modified by some unauthorized entity, thus an integrity service is provided. If they don't match, then it means that the message has been altered by some unauthorized entity during transmission. Any block cipher, for example, AES in CBC mode, can be used to generate a MAC. The MAC of the message is, in fact, the output of the last round of the CBC operation. The length of the MAC output is the same as the block length of the block cipher used to generate the MAC. It should also be noted that MACs work like digital signatures. However, they cannot provide a non repudiation service due to their symmetric nature.

Cryptographic hash mode Hash functions are primarily used to compress a message to a fixed-length digest. In cryptographic hash mode, block ciphers are used as a compression function to produce a hash of plaintext. There are also other modes, such as Cipher Feedback (CFB) mode, Galois Counter (GCM) mode, and Output Feedback (OFB) mode, which are also used in various scenarios

Data Encryption Standard (DES)

- DES was introduced by NIST as a standard algorithm for encryption, and it was in widespread use during the 1980s and 1990s. However, it did not prove to be very resistant to brute force attacks, due to advances in technology and cryptography research. In July 1998, for example, the Electronic Frontier Foundation (EFF) broke DES using a special-purpose machine called EFF DES cracker (or Deep Crack).
- DES uses a key of only 56 bits, which raised some concerns. This problem was addressed with the introduction of Triple DES (3DES), which proposed the use of a 168-bit key by means of three 56-bit keys and the same number of executions of the DES algorithm, thus making brute force attacks almost impossible. However, other limitations, such as slow performance and 64 bit block size, were not desirable.

The basic idea is shown below:

- As mentioned above, that DES uses a 56-bit key. Actually, The initial key consists of 64 bits. However, before the DES process even starts, every 8th bit of the key is discarded to produce a 56-bit key. That is bit positions 8, 16, 24, 32, 40, 48, 56, and 64 are discarded.

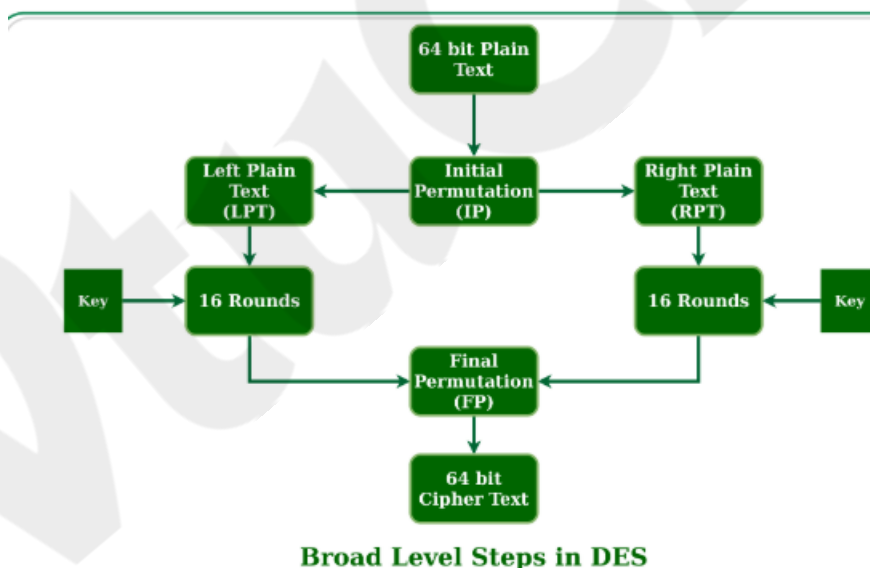
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64

Figure - discarding of every 8th bit of original key

Thus, the discarding of every 8th bit of the key produces a **56-bit key** from the original **64-bit key**.

DES is based on the two fundamental attributes of cryptography: **substitution (also called confusion) and transposition (also called diffusion)**. DES consists of 16 steps, each of which is called a round. Each round performs the steps of substitution and transposition. Let us now discuss the broad-level steps in DES.

- In the first step, the 64-bit plain text block is handed over to an initial Permutation (IP) function.
- The initial permutation is performed on plain text.
- Next, the initial permutation (IP) produces two halves of the permuted block; saying Left Plain Text (LPT) and Right Plain Text (RPT).
- Now each LPT and RPT go through 16 rounds of the encryption process.
- In the end, LPT and RPT are rejoined and a Final Permutation (FP) is performed on the combined block
- The result of this process produces 64-bit ciphertext.



Initial Permutation (IP)

As we have noted, the initial permutation (IP) happens only once and it happens before the first round. It suggests how the transposition in IP should proceed, as shown in the figure. For example, it says that the IP replaces the first bit of the original plain text block with the 58th bit of the original plain

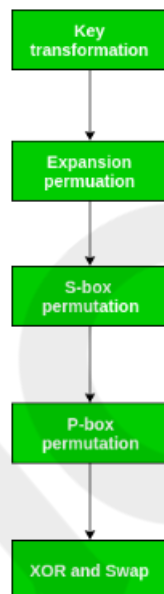
text, the second bit with the 50th bit of the original plain text block, and so on.

This is nothing but jugglery of bit positions of the original plain text block. the same rule applies to all the other bit positions shown in the figure.

58	50	42	34	26	18	10	2	60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6	64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1	59	51	43	35	27	19	11	3
61	33	45	37	29	21	13	5	63	55	47	39	31	23	15	7

Figure - Initial permutation table

As we have noted after IP is done, the resulting 64-bit permuted text block is divided into two half blocks. Each half-block consists of 32 bits, and each of the 16 rounds, in turn, consists of the broad-level steps outlined in the figure.



Step 1: Key transformation

We have noted initial 64-bit key is transformed into a 56-bit key by discarding every 8th bit of the initial key. Thus, for each a 56-bit key is available. From this 56-bit key, a different 48-bit Sub Key is generated during each round using a process called key transformation. For this, the 56-bit key is divided into two halves, each of 28 bits. These halves are circularly shifted left by one or two positions, depending on the round.

For example: if the round numbers 1, 2, 9, or 16 the shift is done by only one position for other rounds, the circular shift is done by two positions. The number of key bits shifted per round is shown in the figure.

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
#key bits shifted	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Figure - number of key bits shifted per round

After an appropriate shift, 48 of the 56 bits are selected. From the 48 we might obtain 64 or 56 bits based on requirement which helps us to recognize that this model is very versatile and can handle any range of requirements needed or provided. For selecting 48 of the 56 bits the table is shown in the figure given below. For instance, after the shift, bit number 14 moves to the first position, bit number 17 moves to the second position, and so on. If we observe the table, we will realize that it contains only 48-bit positions. Bit number 18 is discarded (we will not find it in the table), like 7 others, to reduce a 56-bit key to a 48-bit key. Since the key transformation process involves permutation as well as a selection of a 48-bit subset of the original 56-bit key it is called **Compression Permutation**.

14	17	11	24	1	5	3	28	15	6	21	10
23	19	12	4	26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40	51	45	33	48
44	49	39	56	34	53	46	42	50	36	29	32

Figure - compression permutation

Because of this compression permutation technique, a different subset of key bits is used in each round. That makes DES not easy to crack.

Step 2: Expansion Permutation

Recall that after the initial permutation, we had two 32-bit plain text areas called Left Plain Text(LPT) and Right Plain Text(RPT). During the expansion permutation, the RPT is expanded from 32 bits to 48 bits. Bits are permuted as well hence called expansion permutation. This happens as the 32-bit RPT is divided into 8 blocks, with each block consisting of 4 bits. Then, each 4-bit block of the previous step is then expanded to a corresponding 6-bit block, i.e., per 4-bit block, 2 more bits are added.

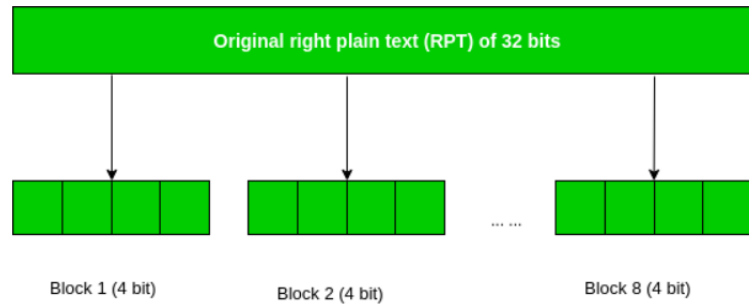


Figure - division of 32 bit RPT into 8 bit blocks

This process results in expansion as well as a permutation of the input bit while creating output. The key transformation process compresses the 56-bit key to 48 bits. Then the expansion permutation process expands the **32-bit RPT to 48-bits**. Now the 48-bit key is XOR with 48-bit RPT and the resulting output is given to the next step, which is the **S-Box substitution**.

Advanced Encryption Standard (AES)

- In 2001, after an open competition, an encryption algorithm named Rijndael invented by cryptographers Joan Daemen and Vincent Rijmen was standardized as AES with minor modifications by NIST. So far, no attack has been found against AES that is more effective than the brute-force method. The original version of Rijndael permits different key and block sizes of 128 bits, 192 bits, and 256 bits. In the AES standard, however, only a 128-bit block size is allowed. However, key sizes of 128 bits, 192 bits, and 256 bits are permissible.

How AES works ??

- During AES algorithm processing, a 4×4 array of bytes known as the state is modified using multiple rounds. Full encryption requires 10 to 14 rounds, depending on the size of the key. The following table shows the key sizes and the required number of rounds:

Key size	Number of rounds required
128-bit	10 rounds
192-bit	12 rounds
256-bit	14 rounds

Once the state is initialized with the input to the cipher, the following four operations are performed sequentially step by step to **encrypt the input**:

- AddRoundKey:** In this step, the state array is XORed with a subkey, which is derived from the master key.
- SubBytes:** This is the substitution step where a lookup table (S-box) is used to replace all bytes of the state array.
- ShiftRows:** This step is used to shift each row to the left, except for the first one, in the state array in a cyclic and incremental manner.

4. **MixColumns:** Finally, all bytes are mixed in a linear fashion (linear transformation), column-wise. This is one round of AES.

In the final round (either the 10th, 12th, or 14th round, depending on the key size), stage 4 is replaced with AddRoundKey to ensure that the first three steps cannot be simply reversed, as shown in the following diagram.

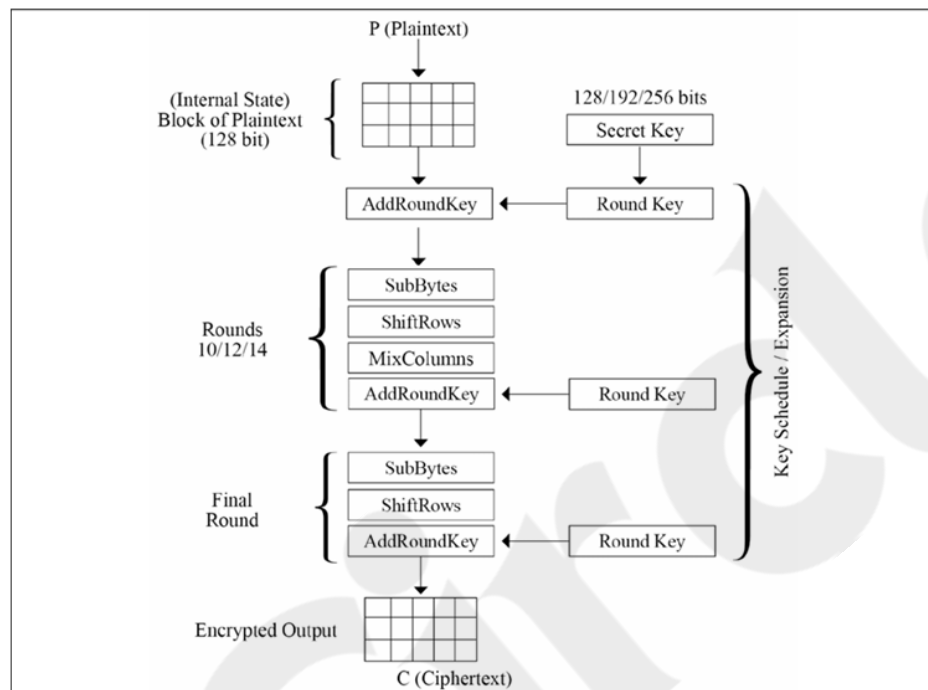


Figure 3.13: AES block diagram, showing the first round of AES encryption. In the final round, the mixing step is not performed

An OpenSSL example of how to encrypt and decrypt using AES

We can use the OpenSSL command-line tool to perform encryption and decryption operations. An example is given here.

First, we create a plain text file to be encrypted:

```
$ echo Datatoencrypt > message.txt
```

Now the file is created, we can run the OpenSSL tool with appropriate parameters to encrypt the file `message.txt` using 256-bit AES in CBC mode:

```
$ openssl enc -aes-256-cbc -in message.txt -out message.bin
enter aes-256-cbc encryption password:
Verifying - enter aes-256-cbc encryption password:
```

Once the operation completes, it will produce a `message.bin` file containing the encrypted data from the `message.txt` file. We can view this file, which shows encrypted contents of the `message.txt` file:

```
$ cat message.bin
Salted__w____s_ŷ_h~?_____
```

Note that `message.bin` is a binary file. Sometimes, it is desirable to encode this binary file in a text format for compatibility/interoperability reasons. A common text encoding format is base64. The following commands can be used to create a base64-encoded message:

```
$ openssl enc -base64 -in message.bin -out message.b64

$ cat message.b64
U2FsdGVkX193uByIcwZf0Z7J1at+4L+Fj8/uzeDAtJE=
```

In order to decrypt an AES-encrypted file, the following commands can be used. An example of `message.bin` from a previous example is used:

```
$ openssl enc -d -aes-256-cbc -in message.bin -out message.dec
enter aes-256-cbc decryption password:

$ cat message.dec
Datatoencrypt
```

Readers may have noticed that no IV has been provided, even though it's required in all block encryption modes of operation except ECB. The reason for this is that OpenSSL automatically derives the IV from the given password. Users can specify the IV using the following switch:

```
-K/-iv , (Initialization Vector) should be provided in Hex.
```

In order to decode from base64, the following commands are used. Follow the `message.b64` file from the previous example:

```
$ openssl enc -d -base64 -in message.b64 -out message.ptx
~/Crypt$ cat message.ptx
Salted__w____s_ŷ_h~?_____:/Crypt$
```

There are many types of ciphers that are supported in OpenSSL. You can explore these options based on the examples provided thus far. A list of supported cipher types is shown as follows:

Cipher Types		
-aes-128-cbc	-aes-128-ccm	-aes-128-cfb
-aes-128-cfb1	-aes-128-cfb8	-aes-128-ctr
-aes-128-ecb	-aes-128-ofb	-aes-192-cbc
-aes-192-ccm	-aes-192-cfb	-aes-192-cfb1
-aes-192-cfb8	-aes-192-ctr	-aes-192-ecb
-aes-192-ofb	-aes-256-cbc	-aes-256-ccm
-aes-256-cfb	-aes-256-cfb1	-aes-256-cfb8
-aes-256-ctr	-aes-256-ecb	-aes-256-ofb
-aes128	-aes192	-aes256
-bf	-bf-cbc	-bf-cfb
-bf-ecb	-bf-ofb	-blowfish
-camellia-128-cbc	-camellia-128-cfb	-camellia-128-cfb1
-camellia-128-cfb8	-camellia-128-ecb	-camellia-128-ofb
-camellia-192-cbc	-camellia-192-cfb	-camellia-192-cfb1
-camellia-192-cfb8	-camellia-192-ecb	-camellia-192-ofb
-camellia-256-cbc	-camellia-256-cfb	-camellia-256-cfb1
-camellia-256-cfb8	-camellia-256-ecb	-camellia-256-ofb
-camellia128	-camellia192	-camellia256
-cast	-cast-cbc	-cast5-cbc
-cast5-cfb	-cast5-ecb	-cast5-ofb
-des	-des-cbc	-des-cfb
-des-cfb1	-des-cfb8	-des-ecb
-des-ede	-des-ede-cbc	-des-ede-cfb
-des-ede-ofb	-des-ede3	-des-ede3-cbc
-des-ede3-cfb	-des-ede3-cfb1	-des-ede3-cfb8
-des-ede3-ofb	-des-ofb	-des3
-desx	-desx-cbc	-id-aes128-CCM
-id-aes128-wrap	-id-aes192-CCM	-id-aes192-wrap
-id-aes256-CCM	-id-aes256-wrap	-id-smime-alg-CMS3DESwrap
-idea	-idea-cbc	-idea-cfb
-idea-ecb	-idea-ofb	-rc2
-rc2-40-cbc	-rc2-64-cbc	-rc2-cbc
-rc2-cfb	-rc2-ecb	-rc2-ofb
-rc4	-rc4-40	-seed
-seed-cbc	-seed-cfb	-seed-ecb
-seed-ofb		

Figure 3.14: Some of the cipher types available in OpenSSL

ASYMMETRIC CRYPTOGRAPHY:

the concepts and practical aspects of public key cryptography, also called asymmetric cryptography or asymmetric key cryptography

MATHEMATICS:

1. Modular arithmetic Also known as clock arithmetic, numbers in modular arithmetic wrap around when they reach a certain fixed number. This fixed number is a positive number called **modulus (sometimes abbreviated to mod)**, and all operations are performed concerning this fixed number.

- Modular arithmetic is analogous to a 12-hour clock; there are numbers from 1 to 12. When 12 is reached, the numbers start from 1 again. Imagine that the time is 9:00 now; 4 hours from now, it will be 1:00 because the numbers wrap around at 12 and start from 1 again.
- In normal addition, this would be $9 + 4 = 13$, but that is not the case on a 12-hour clock; it is 1:00. In other words, this type of arithmetic deals with the remainders after the division operation. For example, $50 \bmod 11$ is 6 because $50 / 11$ leaves a remainder of 6

2. Sets These are collections of distinct objects, for example, $X = \{1, 2, 3, 4, 5\}$. Fields A field is a set in which all its elements form an additive and

multiplicative group. It satisfies specific axioms for addition and multiplication. For all group operations, the distributive law is also applied. The law dictates that the same sum or product will be produced, even if any of the terms or factors are reordered.

3. Finite fields A finite field is one with a finite set of elements. Also known as Galois fields, these structures are of particular importance in cryptography as they can be used to produce accurate and error free results of arithmetic operations. For example, prime finite fields are used in Elliptic Curve Cryptography (ECC) to construct discrete logarithm problems.

4. Prime fields A prime field is a finite one with a prime number of elements. It has specific rules for addition and multiplication, and each non-zero element in the field has an inverse. Addition and multiplication operations are performed modulo p , that is, modulo a prime number.

5. Groups A group is a commutative set with an operation that combines two elements of the set. The group operation is closed and associated with a defined identity element. Additionally, each element in the set has an inverse. Closure (closed) means that if, for example, elements A and B are in the set, then the resultant element after performing an operation on the elements is also in the set. Associative means that the grouping of elements does not affect the result of the operation. Four group axioms must be satisfied for a set to qualify as a group. These group axioms include closure, associativity, an identity element, and an inverse element.

6. Abelian groups An abelian group is formed when the operation on the elements of a set is commutative. The commutative law means that changing the order of the elements does not affect the result of the operation, for example, $A \times B = B \times A$. The key difference is that in an abelian group, closure, associativity, identity element, inverse element, and commutativity abelian group axioms are satisfied, whereas in a group, only the first four axioms are required to be satisfied; that is, closure, associativity, an identity element, and an inverse element

7. Rings If more than one operation can be defined over an abelian group, that group becomes a ring. There are also specific properties that need to be satisfied. A ring must have closure and associative and distributive properties.

8. Cyclic groups A cyclic group is a type of group that can be generated by a single element called the group generator. Order This is the number of elements in a field. It is also known as the cardinality of the field.

This completes a basic introduction to some mathematical concepts involved in cryptography.

Asymmetric Cryptography

- Asymmetric cryptography refers to a type of cryptography where the key that is used to encrypt the data is different from the key that is used to decrypt the data. These keys are called private and public keys, respectively, which why asymmetric cryptography is also known as public key cryptography. It uses both public and private keys to encrypt and decrypt data, respectively. Various

asymmetric cryptography schemes are in use, including RSA (named after its founders, Rivest, Shamir, and Adelman), DSA (Digital Signature Algorithm), and ElGamal encryption. An overview of public-key cryptography is shown in the following diagram:

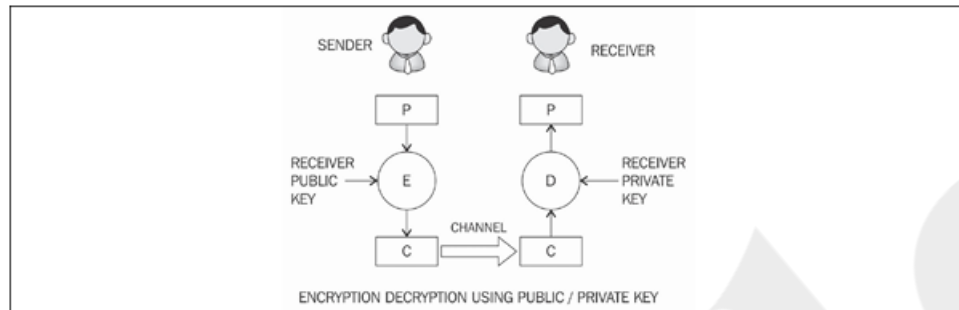


Figure 4.1: Encryption/decryption using public/private keys

- The preceding diagram illustrates how a sender encrypts data P using the recipient's public key and encryption function E , and produces an output encrypted data C , which is then transmitted over the network to the receiver. Once it reaches the receiver, it can be decrypted using the receiver's private key by feeding the C encrypted data into function D , which will output plaintext P . This way, the private key remains on the receiver's side, and there is no need to share keys in order to perform encryption and decryption, which is the case with symmetric encryption.
- The following diagram shows how the receiver uses public key cryptography to verify the integrity of the received message. In this model, the sender signs the data using their private key and transmits the message across to the receiver. Once the message is received, it is verified for integrity by the sender's public key.

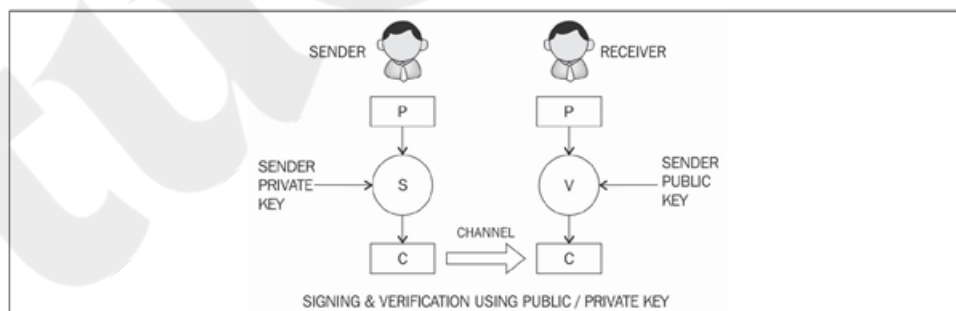


Figure 4.2: Model of a public-key cryptography signature scheme

- The preceding diagram shows that the sender digitally signs the plaintext P with their private key using signing function S , and produces data C , which is sent to the receiver, who verifies C using the sender's public key and function V to ensure the message has indeed come from the sender. Security mechanisms offered by public key cryptosystems include key establishment, digital signatures, identification, encryption, and decryption.
- **Key establishment mechanisms** are concerned with the design of protocols that allow the setting up of keys over an insecure channel. Non-repudiation

services, a very desirable property in many scenarios, can be provided using digital signatures. Sometimes, it is important not only to authenticate a user but also to identify the entity involved in a transaction.

- This can also be achieved by a combination of digital signatures and challenge response protocols. Finally, the encryption mechanism to provide confidentiality can also be obtained using public key cryptosystems, such as RSA, ECC, and ElGamal
- Public key cryptography algorithms are based on various underlying mathematical functions. The three main categories of asymmetric algorithms are described here.

1.Integer factorization:

Integer factorization schemes are based on the fact that large integers are very hard to factor. RSA is a prime example of this type of algorithm.

2. Discrete logarithm:

A discrete logarithm scheme is based on a problem in modular arithmetic. It is easy to calculate the result of a modulo function, but it is computationally impractical to find the exponent of the generator.

In other words, it is extremely difficult to find the input from the result (output). This is called a one-way function. For example, consider the following equation: $3^2 \bmod 10 = 9$ Now, given 9, the result of the preceding equation, finding 2, which is the exponent of the generator 3 in the equation, is extremely hard to determine. This difficult problem is commonly used in the Diffie-Hellman key exchange and digital signature algorithms.

3. Elliptic curves:

The elliptic curves algorithm is based on the discrete logarithm problem discussed previously, but in the context of elliptic curves. An elliptic curve is an algebraic cubic curve over a field, which can be defined by an equation, as shown here.

The curve is non-singular, which means that it has no cusps or self-intersections. It has two variables a and b , as well as a point of infinity:

$y^2 = x^3 + ax + b$ Here, a and b are integers whose values are elements of the field on which the elliptic curve is defined.

Elliptic curves can be defined over reals, rational numbers, complex numbers, or finite fields.

For cryptographic purposes, an elliptic curve over prime finite fields is used instead of real numbers. Additionally, the prime should be greater than 3. Different curves can be generated by varying the values of a and/or b .

The most prominently used cryptosystems based on elliptic curves are the Elliptic Curve Digital Signatures Algorithm (ECDSA) and the Elliptic Curve Diffie-Hellman (ECDH) key exchange.

PUBLIC AND PRIVATE KEYS

- ❖ **A private key**, as the name suggests, is a randomly generated number that is kept secret and held privately by its users.
 - Private keys need to be protected and no unauthorized access should be granted to that key; otherwise, the whole scheme of public key cryptography is jeopardized, as this is the key that is used to decrypt messages.
 - Private keys can be of various lengths, depending on the type and class of algorithms used. For example, in RSA, typically, a key of 1,024 bits or 2,048 bits is used. The 1,024-bit key size is no longer considered secure, and at least a 2,048-bit key size is recommended.
- ❖ **A public key** is freely available and published by the private key owner.
 - Anyone who would then like to send the publisher of the public key an encrypted message can do so, by encrypting the message using the published public key and sending it to the holder of the private key.
 - No one else is able to decrypt the message because the corresponding private key is held securely by the intended recipient.
 - Once the public key-encrypted message is received, the recipient can decrypt the message using the private key. There are a few concerns, however, regarding public keys. These include authenticity and identification of the publisher of the public keys.

The following are the examples of asymmetric key cryptography

RSA

- RSA was invented in 1977 by Ron Rivest, Adi Shamir, and Leonard Adelman, hence the name RSA.
- This type of public key cryptography is based on the integer factorization problem, where the multiplication of two large prime numbers is easy, but it is difficult to factor the product (the result of the multiplication) back to the two original numbers.
- The crux of the work involved with the RSA algorithm happens during the key generation process.
- An RSA key pair is generated by performing the following steps:

1. Modulus generation:

- Select p and q , which are very large prime numbers
- Multiply p and q , $n=p.q$ to generate modulus n

2. Generate the co-prime:

- Assume a number called e .
- e should satisfy a certain condition; that is, it should be greater than 1 and less than $(p-1)(q-1)$.

In other words, e must be a number such that no number other than 1 can be divided into e and $(p-1)(q-1)$. This is called a co-prime, that is, e is the co-prime of $(p-1)(q-1)$

3. Generate the public key:

- The modulus generated in step 1 and co-prime e generated in step 2 is a pair that is a public key. This part is the public part that can be shared with anyone; however, p and q need to be kept secret.

4. Generate the private key:

- The private key is called d here and is calculated from p , q , and e . The private key is basically the inverse of e modulo $(p-1)(q-1)$.

As an equation, it is as follows: $ed = 1 \pmod{(p-1)(q-1)}$ Usually, an extended Euclidean algorithm is used to calculate d . This algorithm takes p , q , and e and calculates d .

The key idea in this scheme is that anyone who knows p and q can easily calculate private key d by applying the extended Euclidean algorithm. However, someone who does not know the value of p and q cannot generate d . This also implies that p and q should be large enough for the modulus n to become extremely difficult (computationally impractical) to factor.

- let's see how encryption and decryption operations are performed using RSA.

RSA uses the following equation to produce ciphertext:

$$C = P^e \pmod n$$

This means that plaintext P is raised to the power of e and then reduced to modulo n .

Decryption in RSA is provided in the following equation:

$$P = C^d \pmod n$$

This means that the receiver who has a public key pair (n, e) can decipher the data by raising C to the value of the private key d , and then reducing to modulo n .

Elliptic curve cryptography

- ECC is based on the discrete logarithm problem founded upon elliptic curves over finite fields (Galois fields).
- The main benefit of ECC over other types of public key algorithms is that it requires a smaller key size, while providing the same level of security as, for example, RSA. Two notable schemes that originate from ECC are ECDH for key exchange and ECDSA for digital signatures. ECC can also be used for encryption, but it is not usually used for this purpose in practice. Instead, key exchange and digital signatures are used more commonly. As ECC needs less space to operate, it is becoming very popular on embedded platforms and in systems where storage resources are limited. By comparison, the same level of security can be achieved with ECC when only using 256-bit operands as compared to 3,072 bits in RSA.
- **Maths behind ECC**

To understand ECC, a basic introduction to the underlying mathematics is necessary. **An elliptic curve is basically a type of polynomial equation known as the Weierstrass equation, which generates a curve over a finite field.** The most commonly used field is where all arithmetic operations are performed modulo a prime number p . Elliptic curve groups consist of points on the curve over a finite field. An elliptic curve is defined in the following equation: The following equation:

$$Y^2 = x^3 + Ax + B \pmod P$$

Here, A and B belong to a finite field $\mathbb{Z}_p$ or $\mathbb{F}_p$ (prime finite field), along with a special value called the point of infinity. The point of infinity ∞ is used to provide identity operations for points on the curve

Basic Group operations on elliptic curves are point addition and point doubling. Point addition is a process where two different points are added, and point doubling means that the same point is added to itself.

Point addition

- Point addition is shown in the following diagram. This is a geometric representation of point addition on elliptic curves.
- In this method, a diagonal line is drawn through the curve that intersects the curve at two points shown below P and Q, which yields a third point between the curve and the line.

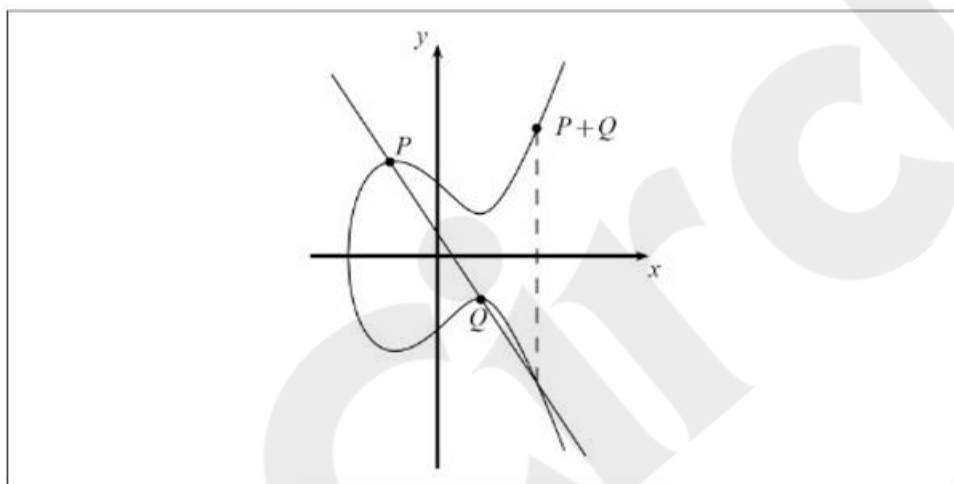


Figure 4.3: Point addition over R

The group operation denoted by the + sign for addition yields the following equation:
 $P + Q = R$

In this case, two points are added to compute the coordinates of the third point on the curve: $P + Q = R$

More precisely, this means that coordinates are added, as shown in the following equation: $(x_1, y_1) + (x_2, y_2) = (x_3, y_3)$

The equation of point addition is as follows: $X^3 = s^2 - x_1 - x_2 \mod p$

$$y_3 = s(x_1 - x_3) - y_1 \mod p$$

Here, we can see the result of the preceding equation:

The group operation denoted by the + sign for addition yields the following equation:
 $P + Q = R$

In this case, two points are added to compute the coordinates of the third point on the curve:
 $P + Q = R$

More precisely, this means that coordinates are added, as shown in the following equation:
 $(x_1, y_1) + (x_2, y_2) = (x_3, y_3)$

The equation of point addition is as follows:

$$X_3 = s^2 - x_1 - x_2 \mod p$$

$$y_3 = s(x_1 - x_3) - y_1 \mod p$$

Here, we can see the result of the preceding equation:

$$S = \frac{(y_2 - y_1)}{(x_2 - x_1)} \mod p$$

S in the preceding equation depicts the line going through P and Q.

An example of point addition is shown here. It was produced using Certicom's online calculator. This example shows the addition and solutions for the equation over finite field F_{23} . This is in contrast to the example shown earlier, which is over real numbers and only shows the curve but provides no solutions to the equation:

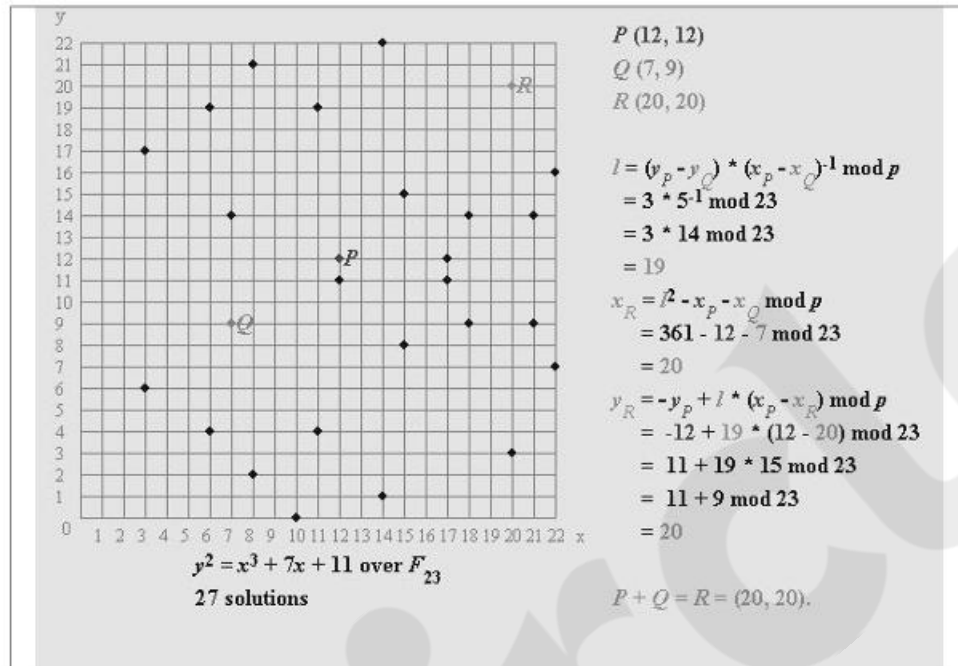


Figure 4.4: Example of point addition

In the preceding example, the graph on the left-hand side shows the points that satisfy this equation:

$$y^2 = x^3 + 7x + 11$$

There are 27 solutions to the equation shown earlier over finite field F_{23} . P and Q are chosen to be added to produce point R . Calculations are shown on the right-hand side, which calculates the third point R . Note that in the preceding graph, in the calculation shown on

As an example, to show how the equation is satisfied by the points shown in the graph, a point (x, y) is picked up where $x = 3$ and $y = 6$. This point can be visualized in the graph shown here. Notice the point at coordinate $(3, 6)$, indicated by an arrow:

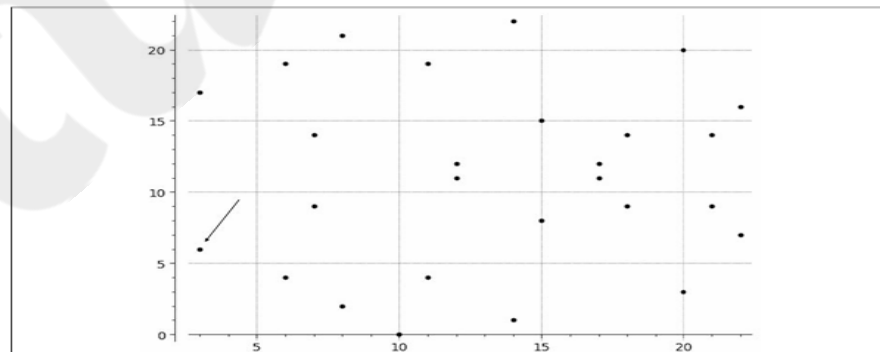


Figure 4.5: Point $(3, 6)$ shown with an arrow (this graph was generated using sagemath)

Using these values shows that the equation is indeed satisfied:

$$\begin{aligned}
 y^2 \bmod 23 &= x^3 + 7x + 11 \bmod 23 \\
 6^2 \bmod 23 &= 3^3 + 7(3) + 11 \bmod 23 \\
 36 \bmod 23 &= 59 \bmod 23 \\
 13 &= 13
 \end{aligned}$$

Point doubling The other group operation on elliptic curves is called point doubling. This is a process where P is added to itself. This is the case when P and Q are at the same point, so effectively the operation becomes adding the point to itself, and is therefore called point doubling. In this method, a tangent line is drawn through the curve, as shown in the following graph. The second point is obtained, which is at the intersection of the tangent line drawn and the curve. This point is then mirrored to yield the result, which is shown as $2P = P + P$

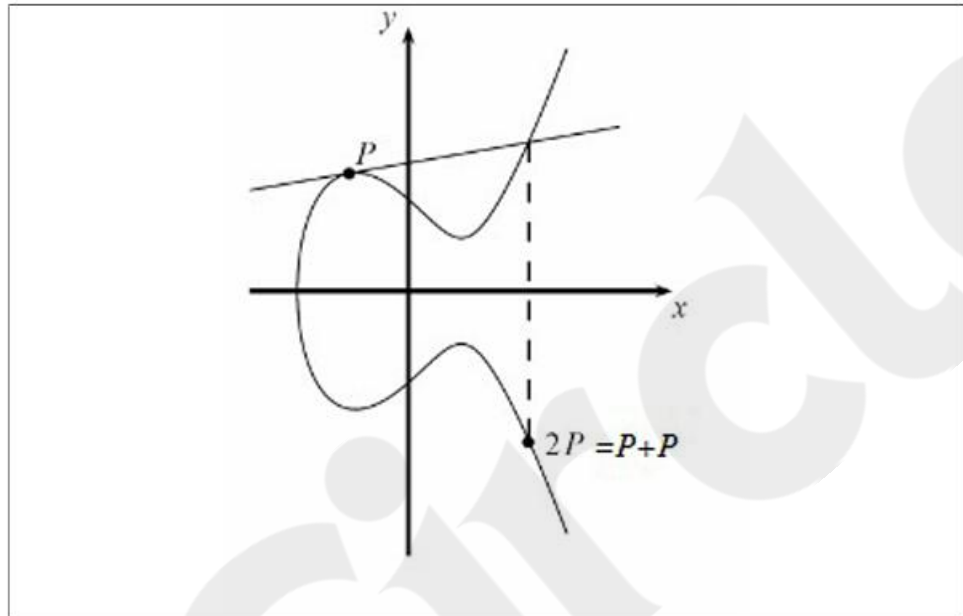


Figure 4.6: Graph representing point doubling

In the case of point doubling, the equation becomes:

$$x_3 = s^2 - x_1 - x_2 \bmod p$$

$$y_3 = s(x_1 - x_3) - y_1 \bmod p$$

$$S = \frac{(y_2 - y_1)}{(x_2 - x_3)} \bmod p$$

Here, S is the slope of the tangent line going through P , which is the line shown at the top in the preceding graph. In the preceding example, the curve is plotted as a simple example, and no solution to the equation is shown.

The following example shows the solutions and point doubling of elliptic curves over finite field F_{23} . The graph on the left-hand side shows the points that satisfy the equation:

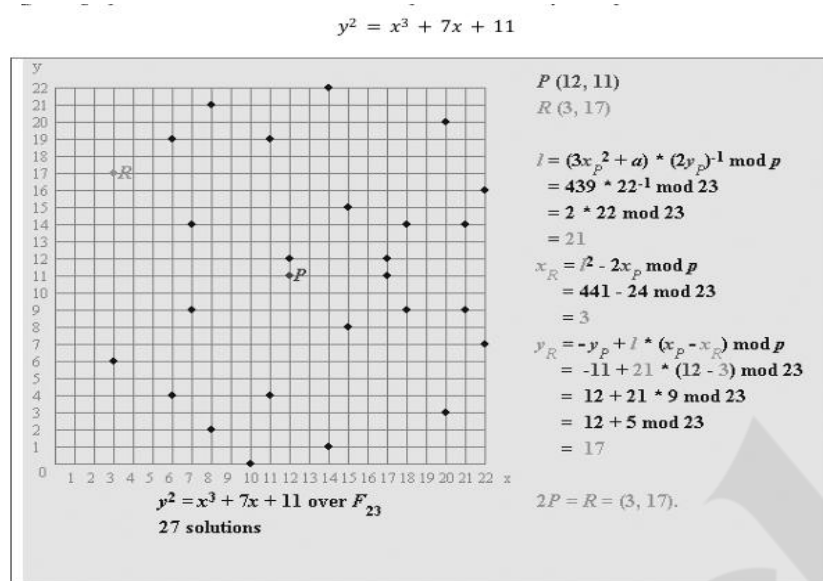


Figure 4.7: Example of point doubling over real numbers

As shown on the right-hand side of the preceding graph, the calculation that finds R after P is added to itself (point doubling). There is no Q shown here because the same point P is used to depict the tangent line going through P . Note that in the calculation shown previously on the right-hand side of the graph, l is used to depict the tangent line going through P .

With this, we have covered basics of ECC. In the next section, an introduction to the discrete logarithm problem based on ECC will be presented. The discrete log problem was introduced earlier in this chapter and can be depicted with a simple equation; that is, given y , find x such that $g^x = y$. Fundamentally, it is a one-way function, which is easy to evaluate but extremely hard (computationally) to invert.

Point multiplication This operation is also called scalar point multiplication. It is used to multiply points on elliptic curves by a given integer. Let's call this integer d and the point P . We get dP by repeatedly adding P , d times. This operation can be described as follows:

$P + P + \dots + P = dP$, where P is added d times. Any point on the curve can be added multiple times to itself. The result of this operation is always another point on the curve. The addition operation is not efficient for large values of d . However, point multiplication operation can be made efficient by utilizing double and add algorithm, which combine point addition and doubling operations to achieve exponential performance gain.

The discrete logarithm problem

In ECC The discrete logarithm problem in ECC is based on the idea that, under certain conditions, all points on an elliptic curve form a cyclic group. On an elliptic curve, the public key is a random multiple of the generator point, whereas the private key is a randomly chosen integer used to generate the multiple.

In other words, a private key is a randomly selected integer, whereas the public key is a point on the curve. The discrete logarithm problem is used to find the private key (an integer) where that integer falls within all points on the elliptic curve. The following equation shows this concept more precisely.

Consider an elliptic curve E , with two elements P and T . The discrete logarithmic problem is to find the integer d , where $1 \leq d \leq \#E$, such that: $P + P + \dots + P = dP = T$. Here, T is the public key (a point on the curve, (x, y)), and d is the private key. In other

words, the public key is a random multiple of the generator P , whereas the private key is the integer that is used to generate the multiple. $\#E$ represents the order of the elliptic curve, which means the number of points that are present in the cyclic group of the elliptic curve.

A cyclic group is formed by a combination of points on the elliptic curve and the point of infinity. The initial starting point P is a public parameter, and the public key T is also published, whereas d , the private key, is kept secret. If d is not known, it is impractical to calculate it by only having the knowledge of T and P , thus creating the hard problem on which ECDLP is built. A key pair is linked with the specific domain parameters of an elliptic curve. Domain parameters include field size, field representation, two elements from the field a and b , two field elements X_g and Y_g , order n of point G , which is calculated as $G = (X_g, Y_g)$, and the cofactor $h = \#E(F_q)/n$.

The elliptic curve domain parameters over $\mathbb{F}_p$ associated with a Koblitz curve `secp256k1` are specified by the sextuple $T = (p, a, b, G, n, h)$ where the finite field $\mathbb{F}_p$ is defined by:

$$p = \text{FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE FFFFC2F}$$

$$= 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$$

The curve $E: y^2 = x^3 + ax + b$ over $\mathbb{F}_p$ is defined by:

$$a = \text{00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000}$$

$$b = \text{00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000007}$$

The base point G in compressed form is:

$$G = \text{02 79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798}$$

and in uncompressed form is:

$$G = \text{04 79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798 483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448 A6855419 9C47D08F FB10D4B8}$$

Finally the order n of G and the cofactor are:

$$n = \text{FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE BAAEDCE6 AF48A03B BFD25E8C D0364141}$$

$$h = \text{01}$$

Figure 4.8: Specification of SECP256K1

An explanation of all of these values in the sextuple follows:

- P is the prime p that specifies the size of the finite field.
- a and b are the coefficients of the elliptic curve equation.
- G is the base point that generates the required subgroup, also known as the generator. The base point can be represented in either compressed or uncompressed form. There is no need to store all points on the curve in a practical implementation. The compressed generator works because the points on the curve can be identified using only the x coordinate and the least significant bit of the y coordinate.
- n is the order of the subgroup.
- h is the cofactor of the subgroup.

We can also use the OpenSSL command line to view these parameters of SECP256K1. This can be seen here:

```
$ openssl ecparam -param_enc explicit -text -noout -name secp256k1
```

This command will show the output as follows:

```
Field Type: prime-field  
Prime:  
    00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:  
    ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:fe:ff:  
    ff:fc:2f  
A:  0  
B:  7 (0x7)  
Generator (uncompressed):  
    04:79:be:66:7e:f9:dc:bb:ac:55:a0:62:95:ce:87:  
    0b:07:02:9b:fc:db:2d:ce:28:d9:59:f2:81:5b:16:  
    f8:17:98:48:3a:da:77:26:a3:c4:65:5d:a4:fb:fc:  
    0e:11:08:a8:fd:17:b4:48:a6:85:54:19:9c:47:d0:  
    8f:fb:10:d4:b8  
Order:  
    00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:  
    ff:fe:ba:ae:dc:e6:af:48:a0:3b:bf:d2:5e:8c:d0:  
    36:41:41  
Cofactor: 1 (0x1)
```

RSA using OpenSSL

The following example illustrates how RSA public and private key pairs can be generated using the OpenSSL command line.

Private key

```
$ openssl genpkey -algorithm RSA -out privatekey.pem -pkeyopt rsa_keygen_bits:1024
.....+++++
.....+++++
```

After executing the command, a file named `privatekey.pem` is produced, which contains the generated private key, as follows:

```
$ cat privatekey.pem
-----BEGIN PRIVATE KEY-----
MIICdgIBADANBgkqhkiG9w0BAQEFAASCAMAwgJcAgEAAoGBAKJOFBzPy2v0d6em Bk/
UGrzdY7TvgDyntYXbfeJId/r+EmtM/F14k2FDTOVwxXaTxiQd+BKuiey/69 91tnrqW/xy/poc
DMvobj8CkQEdNoDnVSA+Nt8f9nRM3iVM94mz3/C/v4Xvoac PyPkr/0jhIV0woCurXGTghg
qIbHRAGMBAEAcgYEAIB3s/N41Jh011TkOSYunWtzT 6isnNkR7g1WrY9H+rG9xxk4P5b1DyE3S
vxBLJA6xgBle8jVQMzm3sKJrJPfZzzT5 NNNnugCxaixrcF1mPzJAP3aqpcSjXkPtv4qgqYeww
gw1A0R3xkQZzBKU+bT02hXV D1ohXu75mDY3xCwqSAECQDUYV04wnSEjEy9tYJ0zaryDacvd/
VG2/U/6qiQGajB eSpSqoEESigbusKku+wVtRYgWWEomL/X58t+K0iEMZZAKEAw6PUR9YLebsm/
SjI i0shV4AKuFdi7e7DYWE5ULbluqP/i28zNjytt48XXIs/KcFykQGeAC6LDHzyccyc
ntDIOQJAVqrE1/wYvV5jkkcXbYlGv5YA+KYD0b9Y/ZRM5UETVKCVXNanf5Cjfw1h MM
hfNxyGwvy2YVK0Nu8oY3xYPi+5QQJAUGcmORE4w6Cs12JUJ5p+zG0s+rG/URhw
B7djTxm7p6b6wR1EWAZDM9MArenj8uXAA1AGCcIsmIdqHfU7lgZ0QJAE9mOdNGW 7qRppgmOE5nuEbxk
DSQI70qkHYbOLUwfcJhZJBrSgqyip6j9/9CbXJRzPgNdwdLeB GgpDKtZs9gLv3A==
-----END PRIVATE KEY-----
```

Public key

As the private key is mathematically linked to the public key, it is also possible to generate or derive the public key from the private key. Using the example of the preceding private key, the public key can be generated as follows:

```
$ openssl rsa -pubout -in privatekey.pem -out publickey.pem
writing RSA key
```

The public key can be viewed using a file reader or any text viewer:

```
$ cat publickey.pem
```

```
-----BEGIN PUBLIC KEY-----
MIGfMA0GCsGqGBIb3DQEBAQUAA4GNADCBiQKBgQCiTThQcz8trznepgZP1Bq8w8u0
74A2J2MQX4hCSHF6/hMjLfxdeJNnw0z1cMV2l08YkIA/gSronsV+vfYrZ66lv8cv 6aHAzL6G4/
EAp4BJ7TnTaFUmjfrdH/Z0TN4lTpeJs9/wv7+L176Gnd8j5K/9I4SF dMKArq1xk4IYKiGx0QIDAQAB
-----END PUBLIC KEY-----
```

In order to see more details of the various components, such as the modulus, prime numbers that are used in the encryption process, or exponents and coefficients of the generated private key, the following command can be used (only part of the output is shown here as the actual output is very long):

```
$ openssl rsa -text -in privatekey.pem
Private-Key: (1024 bit)
modulus:
00:a2:4e:14:1c:cf:cb:6b:ce:77:a7:a6:06:4f:d4:
1a:bc:c3:cb:b4:ef:80:36:27:63:10:5f:88:42:48:
77:fa:fe:13:23:2d:fc:5d:78:93:67:c3:4c:e5:70:
c5:76:97:4f:18:90:80:3f:81:2a:e8:9e:cb:fe:bd:
f6:2b:67:ae:a5:bf:c7:2f:e9:a1:c0:cc:be:86:e3:
f1:00:a7:80:49:ed:39:d3:68:55:26:8d:fa:dd:1f:
f6:74:4c:de:25:4c:f7:89:b3:df:f0:bf:bf:8b:d7:
be:86:9c:3f:23:e4:af:fd:23:84:85:74:c2:80:ae:
ad:71:93:82:18:2a:21:b1:d1
publicExponent: 65537 (0x10001) privateExponent:
00:94:1d:ec:fc:de:25:26:1d:25:d5:39:0e:49:8b:
a7:5a:dc:d3:ea:2b:27:36:44:7b:83:55:ab:63:d1:
fe:ac:6f:71:c7:89:0f:e5:bd:43:c8:4d:d2:bf:10:
4b:24:0e:b1:80:19:5e:f0:95:50:33:39:b7:b0:a2:
6b:24:f1:59:cf:34:f9:34:d3:67:ba:00:b1:6a:2a:
f1:70:5d:66:3f:32:40:3f:76:aa:a5:c4:a3:c4:aa:
53:bf:8a:a0:a9:87:af:c2:05:b5:03:44:77:c4:a4:
19:cc:12:94:f9:b4:ce:da:15:d5:0f:5a:07:c6:ee:
f9:98:36:37:c4:2c:2a:48:01

prime1:
00:d4:61:5d:38:c0:d4:84:8c:4c:bd:b5:82:74:cd:
aa:f2:0c:07:2f:77:f5:46:db:f5:3f:ea:a8:90:19:
a8:c1:79:2a:52:aa:81:04:4a:28:1b:ba:c2:a4:bb:
ec:15:b5:16:20:59:61:28:98:bf:d7:e7:cb:7e:2b:
4d:5e:30:c6:59

prime2:
00:c3:a3:d4:47:d6:0b:79:bb:26:fd:28:e2:88:e4:
a1:57:80:0a:b8:57:62:ee:de:c3:61:61:39:52:56:
f5:ba:a3:ff:8b:6f:33:37:fc:ad:b7:80:57:28:8b:
```

```
3f:29:c1:72:91:01:9e:00:2e:8b:0c:76:72:c9:cc:
9c:9e:d0:c8:39
```

Exploring the public key

Similarly, the public key can be explored using the following commands. Public and private keys are base64-encoded:

```
$ openssl pkey -in publickey.pem -pubin -text
-----BEGIN PUBLIC KEY-----
MIGfMA0GCsGSIb3DQEBAQUAA4GNADCBiQKBgQCiTThQcz8trznepgZP1Bq8w8u0
74A2J2MQX4hCSHF6/hMjLfXdeJNnw0zlcMV2l08YkIA/gSronsV+vfYrZ66lv8cv
6aHAzL6G4/
EAp4BJ7TnTaFUmjfrdH/Z0TN4lTpeJs9/wv7+L176GnD8j5K/9I4SF dMKArq1xk4IYKIGx0QIDAQAB
-----END PUBLIC KEY-----
Public-Key: (1024 bit) Modulus:
00:a2:4e:14:1c:cf:cb:6b:ce:77:a7:a6:06:4f:d4:
1a:bc:c3:cb:b4:ef:80:36:27:63:10:5f:88:42:48:
77:fa:fe:13:23:2d:fc:5d:78:93:67:c3:4c:e5:70:
c5:76:97:4f:18:90:80:3f:81:2a:e8:9e:cb:fe:bd:
f6:2b:67:ae:a5:bf:c7:2f:e9:a1:c0:cc:be:86:e3:
f1:00:a7:80:49:ed:39:d3:68:55:26:8d:fa:dd:1f:
f6:74:4c:de:25:4c:f7:89:b3:df:f0:bf:bf:8b:d7:
be:86:9c:3f:23:e4:af:fd:23:84:85:74:c2:80:ae:
ad:71:93:82:18:2a:21:b1:d1
Exponent: 65537 (0x10001)
```

Now, the public key can be shared openly, and anyone who wants to send you a message can use the public key to encrypt the message and send it to you. You can then use the corresponding private key to decrypt the file.

Encryption and decryption using RSA

Encryption:

```
$ echo datatoencrypt > message.txt

$ openssl rsautl -encrypt -inkey publickey.pem -pubin -in message.txt -out
message.rsa
```

This will produce a file named `message.rsa`, which is in binary format. If you open `message.rsa` in the nano editor or any other text editor of your choice, it will show some scrambled data, as shown in the following screenshot:

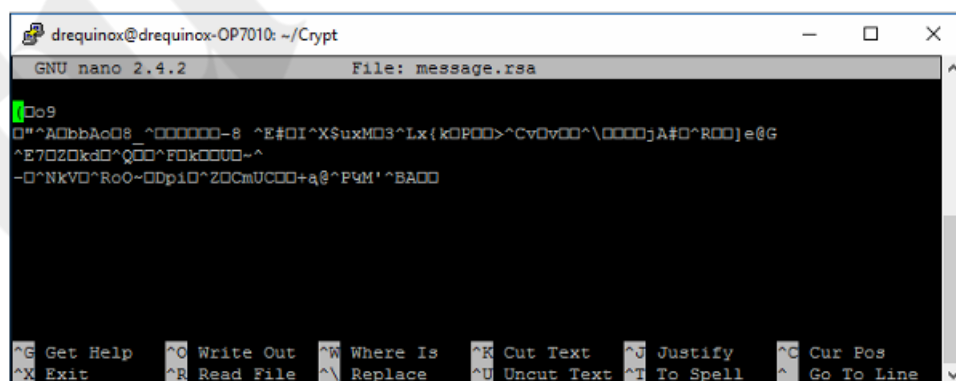


Figure 4.9: message.rsa showing scrambled data

Decryption

In order to decrypt the RSA-encrypted file, the following command can be used:

```
$ openssl rsautl -decrypt -inkey privatekey.pem -in message.rsa -out message.dec
```

Now, if the file is read using cat, decrypted plaintext can be seen, as shown here:

```
$ cat message.dec
datatoencrypt
```

ECC using OpenSSL

OpenSSL provides a very rich library of functions to perform ECC. The following section shows how to use ECC functions in a practical manner in OpenSSL.

ECC private and public key pairs

In this section, first, an example will be presented that demonstrates the creation of a private key using the ECC functions available in the OpenSSL library.

Private key

ECC is based on domain parameters defined by various standards. You can see the list of all available standards defined and the recommended curves available in OpenSSL using the following command (once again, only part of the output is shown here, and it is truncated in the middle):

```
$ openssl ecparam -list_curves
secp112r1 : SECG/WTLS curve over a 112 bit prime field secp112r2 : SECG curve over
a 112 bit prime field secp128r1 : SECG curve over a 128 bit prime field secp128r2
: SECG curve over a 128 bit prime field secp160k1 : SECG curve over a 160 bit
prime field secp160r1 : SECG curve over a 160 bit prime field secp160r2 : SECG/WTLS
curve over a 160 bit prime field secp192k1 : SECG curve over a 192 bit prime field
secp224k1 : SECG curve over a 224 bit prime field secp224r1 : NIST/SECG curve over
a 224 bit prime field secp256k1 : SECG curve over a 256 bit prime field secp384r1 :
NIST/SECG curve over a 384 bit prime field secp521r1 : NIST/SECG curve over a 521
bit prime field
prime192v1: NIST/X9.62/SECG curve over a 192 bit prime field
.
.
.
.
.
brainpoolP384r1: RFC 5639 curve over a 384 bit prime field brainpoolP384t1: RFC
5639 curve over a 384 bit prime field brainpoolP512r1: RFC 5639 curve over a 512
bit prime field brainpoolP512t1: RFC 5639 curve over a 512 bit prime field
```

In the following example, SECP256k1 is used to demonstrate ECC usage.

Private key generation

In this section, a private key based on SECP256k1 will be generated using the OpenSSL command-line tool.

```
$ openssl ecparam -name secp256k1 -genkey -noout -out ec-privatekey.pem
```

This command will produce a file named `ec-privatekey.pem`, which we can view using the command shown here:

```
$ cat ec-privatekey.pem
-----BEGIN EC PRIVATE KEY-----
MHQCAQEEIjHUI9NZA9fUrSxUk/iINq1ghM/ewn/RLNreuR52h/oAcGBSuBBAK
oUQDQgAE0G33mCZ4PKbg5EtwQjk6ucv9Qc9DTr8JdcGXYGxHdzr0Jt1NInaYE0GG
15ul0oczwwKjng==
-----END EC PRIVATE KEY-----
```

The file named `ec-privatekey.pem` now contains the elliptic curve private key that is generated based on the SECP256K1 curve. In order to generate a public key from a private key, issue the following command:

```
$ openssl ec -in ec-privatekey.pem -pubout -out ec-pubkey.pem

read EC key
writing EC key
```

Reading the file produces the following output, displaying the generated public key:

```
$ cat ec-pubkey.pem
-----BEGIN PUBLIC KEY-----
MFYwEAYHKoZIzj0CAQYFK4EEAAoDQgAE0G33mCZ4PKbg5EtwQjk6ucv9Qc9DTr8J
dcGXYGxHdzr0Jt1NInaYE0GGChFMT5pK+wfvSLkY15ul0oczwwKjng==
-----END PUBLIC KEY-----
```

Now, the ec-pubkey.pem file contains the public key derived from ec-privatekey.pem. The private key can be further explored using the following command:

```
$ openssl ec -in ec-privatekey.pem -text -noout
read EC key
Private-Key: (256 bit) priv:
00:91:d4:22:6f:4d:64:08:1f:a5:4a:d2:c5:49:3f:
88:83:6a:d6:08:4c:fd:ec:27:fd:12:cd:ad:eb:91: e7:68:7f
pub:
04:d0:6d:f7:98:26:78:3c:a6:e0:e4:4b:70:42:39:
3a:b9:cb:fd:41:cf:43:4e:bf:09:75:c1:97:60:6c:
47:77:3a:f4:26:dd:4d:22:76:98:13:41:86:0a:11:
4c:4f:9a:4a:fb:07:ef:48:b9:18:97:9b:a5:d2:87:
33:c1:62:a3:9e
ASN1 OID: secp256k1
```

Similarly, the public key can be further explored with the following command:

```
$ openssl ec -in ec-pubkey.pem -pubin -text -noout
read EC key
Private-Key: (256 bit) pub:
04:d0:6d:f7:98:26:78:3c:a6:e0:e4:4b:70:42:39:
3a:b9:cb:fd:41:cf:43:4e:bf:09:75:c1:97:60:6c:
47:77:3a:f4:26:dd:4d:22:76:98:13:41:86:0a:11:
4c:4f:9a:4a:fb:07:ef:48:b9:18:97:9b:a5:d2:87:
33:c1:62:a3:9e
ASN1 OID: secp256k1
```

It is also possible to generate a file with the required parameters—in this case, SECP256K1—and then explore it further to understand the underlying parameters:

```
$ openssl ecparam -name secp256k1 -out secp256k1.pem
$ cat secp256k1.pem
-----BEGIN EC PARAMETERS-----
BgUrgQQACg==
-----END EC PARAMETERS-----
```

The file now contains all the SECP256K1 parameters, and it can be analyzed using the following command:

```
$ openssl ecparam -in secp256k1.pem -text -param_enc explicit -noout
```

This command will produce output similar to the one shown here:

```
Field Type: prime-field Prime:
00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:
ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:fc:2f
A: 0
B: 7 (0x7)
Generator (uncompressed): 04:79:be:66:7e:f9:dc:bb:ac:55:a0:62:95:ce:87:
0b:07:02:9b:fc:db:2d:ce:28:d9:59:f2:81:5b:16:
f8:17:98:48:3a:da:77:26:a3:c4:65:5d:a4:fb:fc:
0e:11:08:a8:fd:17:b4:48:a6:85:54:19:9c:47:d0:
8f:fb:10:d4:b8 Order:
00:ff:ff:ff:ff:ff:ff:ff:ff:ff:ff:
ff:fe:ba:ae:dc:e6:af:48:a0:3b:bf:d2:5e:8c:d0:
36:41:41
Cofactor: 1 (0x1)
```

The preceding example shows the prime number used and the values of A and B, along with the generator, order, and cofactor of the SECP256K1 curve domain parameters.

With the preceding example, our introduction to public key cryptography from an encryption

DIGITAL SIGNATURES

Digital signatures provide a means of associating a message with an entity from which the message has originated. Digital signatures are used to provide data origin authentication and non-repudiation.

- Digital signatures are used in blockchains, where transactions are digitally signed by senders using their private key, before the sender broadcasts the transaction to the network. This digital signing proves that the sender is the rightful owner of the asset; for example, bitcoins. These transactions are verified again by other nodes on the network to ensure that the funds indeed belong to the node (user) who claims to be the owner. We will discuss these concepts in more detail in the chapters dedicated to Bitcoin and Ethereum in this book. Digital signatures are calculated in two steps. As an example, the high-level steps of the RSA digital signature scheme are as follows.

RSA digital signature algorithms

- RSA-based digital signature algorithms are calculated using the two steps listed here. Fundamentally, the idea is to first compute the hash of the data and then sign it with the private key:
 1. Calculate the hash value of the data packet. This will provide the data integrity guarantee, as the hash can be computed at the receiver's end again and matched with the original hash to check whether the data has been modified in transit. Technically, message signing can work without hashing the data first, but that is not considered secure.
 2. Sign the hash value with the signer's private key. As only the signer has the private key, the authenticity of the signature and the signed data is ensured. Digital signatures have some important properties, such as authenticity, unforgeability, and non-reusability:
 - Authenticity means that the digital signatures are verifiable by a receiving party.

- The unforgeability property ensures that only the sender of the message can use the signing functionality using the private key.

Digital signatures must provide protection against forgery. Forgery means an adversary fabricating a valid signature for a message without any access to the legitimate signer's private key. In other words, unforgeability means that no one else can produce the signed message produced by a legitimate sender. This is also called the property of non-repudiation.

- Non-reusability means that the digital signature cannot be separated from a message and used again for another message. In other words, the digital signature is firmly bound to the corresponding message and cannot be simply cut from its original message and attached to another.

The operation of a generic digital signature function is shown in the following diagram:

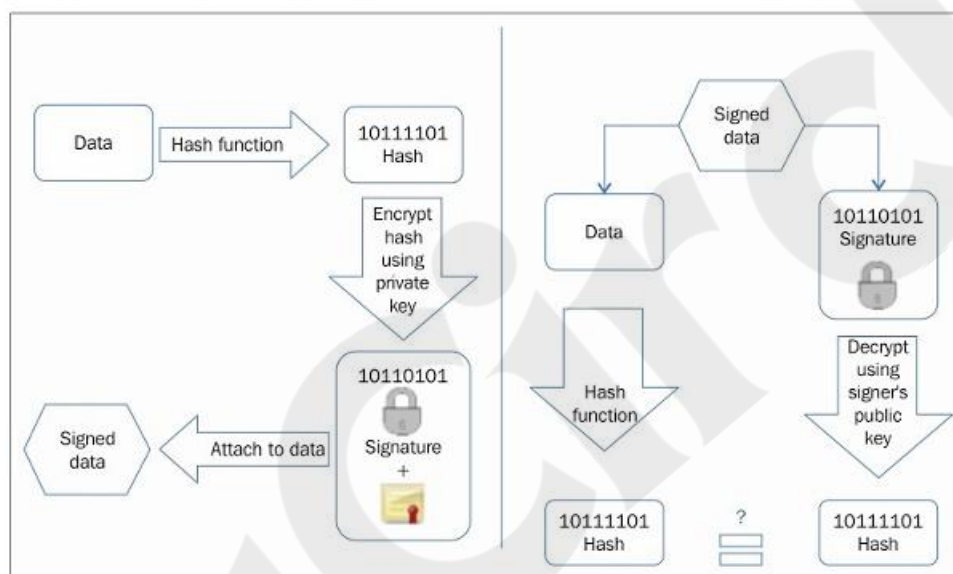


Figure 4.10: Digital signing (left) and verification process (right) (example of RSA digital signatures)

If a sender wants to send an authenticated message to a receiver, there are two methods that can be used: sign then encrypt and encrypt then sign. These two approaches of using digital signatures with encryption are as follows.

Sign then encrypt With this approach, the sender digitally signs the data using the private key, appends the signature to the data, and then encrypts the data and the digital signature using the receiver's public key.

Encrypt then sign With this method, the sender encrypts the data using the receiver's public key and then digitally signs the encrypted data.

The following table shows that ECC is able to provide the same level of cryptographic strength as an RSA-based system with smaller key sizes.

RSA key size (bits)	Elliptic curve key size (bits)
1,024	160
2,048	224
3,072	256
7,680	384
15,360	521

The elliptic curve digital signature algorithm

ECDSA is a DSA based on elliptic curves. The DSA is a standard for digital signatures. It is based on modular exponentiation and the discrete logarithm problem. It is used on the Bitcoin and Ethereum blockchain platforms to validate messages and provide data integrity services. Now, we'll describe how ECDSA works.

In order to sign and verify using the ECDSA scheme, the first key pair needs to be generated:

1. First, define an elliptic curve E with the following:
 - Modulus P
 - Coefficients a and b
 - Generator point A that forms a cyclic group of prime order q
2. An integer d is chosen randomly so that $0 < d < q$.
3. Calculate public key B so that $B = dA$.
 - The public key is a sextuple in the form shown here:

$$K_{pb} = (p, a, b, q, A, B)$$

- The private key is a randomly chosen integer d in step 2:

$$K_{pr} = d$$

Now, the signature can be generated using the private and public key.

4. An ephemeral key K_e is chosen, where $0 < K_e < q$. It should be ensured that K_e is truly random and that no two signatures have the same key; otherwise, the private key can be calculated.
5. Another value R is calculated using $R = K_e A$; that is, by multiplying A (the generator point) and the random ephemeral key.
6. Initialize a variable r with the x coordinate value of point R so that $r = xR$.
7. The signature can be calculated as follows:

$$S = (h(m) + dr)K_e^{-1} \bmod q$$

Here, m is the message for which the signature is being computed, and $h(m)$ is the hash of the message m .

8. Signature verification is carried out by following this process:
 - Auxiliary value w is calculated as $w = s^{-1} \bmod q$
 - Auxiliary value $u1 = w \cdot h(m) \bmod q$
 - Auxiliary value $u2 = w \cdot r \bmod q$
 - Calculate point P , $P = u1A + u2B$
9. Verification is carried out as follows:
 - r, s is accepted as a valid signature if the x -coordinate of point P calculated in Step 4 has the same value as the signature parameter $r \bmod q$; that is:

$$X_p = r \bmod q \text{ means valid signature}$$

$$X_p \neq r \bmod q \text{ means invalid signature}$$

How to generate a digital signature using OpenSSL

The first step is to generate a hash of the message file:

```
$ openssl dgst -sha256 message.txt
SHA256(message.txt)=
eb96d1f89812bf4967d9fb4ead128c3b787272b7be21dd2529278db1128d559c
```

Both hash generation and signing can be done in a single step, as shown here. Note that `privatekey.pem` was generated in the steps provided previously:

```
$ openssl dgst -sha256 -sign privatekey.pem -out signature.bin message.txt
```

Now, let's display the directory showing the relevant files:

```
$ cat signature.bin
```

In order to verify the signature, the following operation can be performed:

```
$ openssl dgst -sha256 -verify publickey.pem -signature signature.bin message.txt
Verified OK
```

Similarly, if some other signature file that is not valid is used, the verification will fail, as shown here:

```
$ openssl dgst -sha256 -verify publickey.pem -signature someothersignature.bin message.txt
Verification Failure
```

Next, an example will be presented that shows how OpenSSL can be used to perform ECDSA-related operations.

ECDSA using OpenSSL

First, the private key is generated using the following commands:

```
$ openssl ecparam -genkey -name secp256k1 -noout -out eccprivatekey.pem
$ cat eccprivatekey.pem
-----BEGIN EC PRIVATE KEY-----
MHQCAQEEIMVmyrNE0s7SYxS/AbXoIwqZqJ+gND9Z2/nQyzcpaPB0AcGBSuBBAAK oUQDQgAEEKKS4E4+
TATIEBX8o2J6PxKkjcoWrXPwNRo/k4Y/CZA4pXvlyTgH5LYm QbU0qUtPM7dAEzOsaoXmetqB+6cM+Q==
-----END EC PRIVATE KEY-----
```

Next, the public key is generated from the private key:

```
$ openssl ec -in eccprivatekey.pem -pubout -out eccpublickey.pem
read EC key
writing EC key
$ cat eccpublickey.pem
-----BEGIN PUBLIC KEY----- MFYwEAYHKoZIzj0CAQYFK4EEAAoDQgAEEKKS4E4+TATIEBX8o2J6Px
KkjcoWrXPw
NRo/k4Y/CZA4pXvlyTgH5LYmQbU0qUtPM7dAEzOsaoXmetqB+6cM+Q==
-----END PUBLIC KEY-----
```

Now, suppose a file named `testsign.txt` needs to be signed and verified. This can be achieved as follows:

1. Create a test file:

```
$ echo testing > testsign.txt

$ cat testsign.txt
testing
```

- Run the following command to generate a signature using a private key for the testsign.txt file:

```
$ openssl dgst -ecdsa-with-SHA1 -sign eccprivatekey.pem testsign.txt >
ecsign.bin
```

- Finally, the command for verification can be run, as shown here:

```
$ openssl dgst -ecdsa-with-SHA1 -verify eccpublickey.pem -signature
ecsign.bin testsign.txt
Verified OK
```

A certificate can also be produced by using the private key generated earlier by using the command shown here:

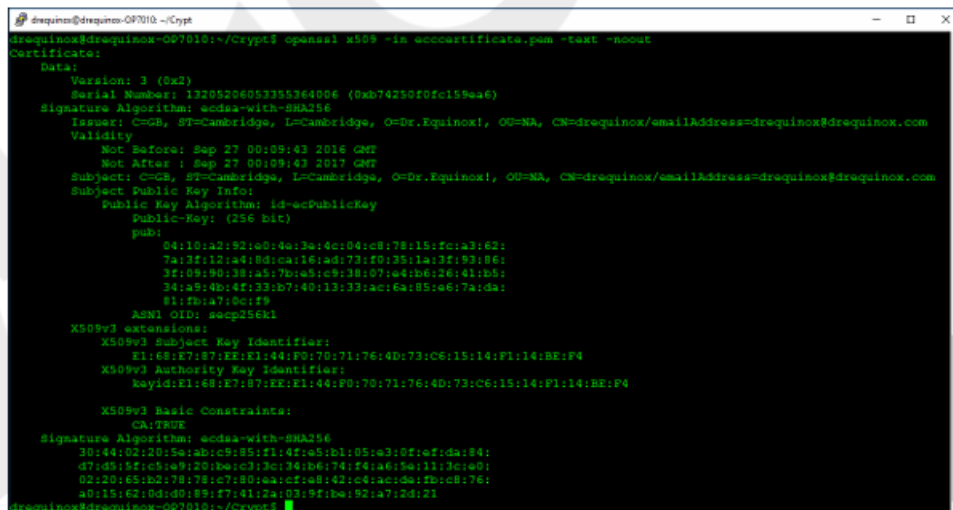
```
$ openssl req -new -key eccprivatekey.pem -x509 -nodes -days 365 - out
eccertificate.pem
```

This command will produce an output similar to the one shown here. Enter the appropriate parameters to generate the certificate:

```
You are about to be asked to enter information that will be incorporated into
your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN. There
are quite a few fields but you can leave some blank
For some fields there will be a default value, If you enter '.', the field will be
left blank.
-----
Country Name (2 letter code) [AU]:GB
State or Province Name (full name) [Some-State]:Cambridge Locality Name (eg,
city) []:Cambridge
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Dr.Equinox!
Organizational Unit Name (eg, section) []:NA
Common Name (e.g. server FQDN or YOUR name) []:drequinox Email Address
[]:drequinox@drequinox.com
```

The certificate can be explored using the following command:

```
$ openssl x509 -in eccertificate.pem -text -noout
```



```
drequinox@drequinox-QP7010: ~/Crypt
drequinox@drequinox-QP7010:~/Crypt$ openssl x509 -in eccertificate.pem -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 13295206053355364006 (0xb74250f0c159aa6)
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: C=GB, ST=Cambridge, L=Cambridge, O=Dr.Equinox!, OU=NA, CN=drequinox/emailAddress=drequinox@drequinox.com
        Validity
            Not Before: Sep 27 00:09:43 2016 GMT
            Not After : Sep 27 00:09:43 2017 GMT
        Subject: C=GB, ST=Cambridge, L=Cambridge, O=Dr.Equinox!, OU=NA, CN=drequinox/emailAddress=drequinox@drequinox.com
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
            Public-Key: (256 bit)
                pub:
                    04:10:a2:92:e0:4e:3e:4c:04:c8:78:15:fc:a3:62:
                    7a:3f:12:a4:8d:ca:16:ad:73:f0:35:1a:3f:93:86:
                    3f:09:90:38:a5:7b:e5:c9:38:07:04:b6:26:41:b5:
                    3a:a9:4b:4f:33:b7:40:13:33:ac:6a:95:a6:7a:da:
                    81:2a:71:0c:19
                ASN1 OID: secp256k1
            X509v3 extensions:
                X509v3 Subject Key Identifier:
                    E1:68:E7:87:EE:E1:44:F0:70:71:76:4D:73:C6:15:14:F1:14:BE:F4
                X509v3 Authority Key Identifier:
                    keyid:E1:68:E7:87:EE:E1:44:F0:70:71:76:4D:73:C6:15:14:F1:14:BE:F4
                X509v3 Basic Constraints:
                    CA:TRUE
        Signature Algorithm: ecdsa-with-SHA256
        30:44:02:20:5e:ab:c9:85:f1:4f:e5:b1:05:e3:0f:ef:da:84:
        d7:d5:5f:c5:a9:20:b6:c3:3c:34:b6:74:f4:a6:5e:11:3c:e0:
        02:20:65:b2:78:78:c7:80:ea:cf:e8:42:c0:ac:da:fb:c8:76:
        a0:16:62:0d:00:18:27:41:2a:03:9f:ba:92:a7:2d:21
drequinox@drequinox-QP7010:~/Crypt$
```

Figure 4.11: An X.509 certificate that uses the ECDSA algorithm with SHA-256

CRYPTOGRAPHIC CONSTRUCTS AND BLOCKCHAIN TECHNOLOGY

Homomorphic encryption Usually, public key cryptosystems, such as RSA, are multiplicative homomorphic or additive homomorphic, such as the Paillier

cryptosystem, and are called **partially homomorphic systems**. Additive Partially Homomorphic Encryptions (PHEs) are suitable for e-voting and banking applications. Until recently, there has been no system that supported both operations, but in 2009, a fully homomorphic system was discovered by Craig Gentry. As these schemes enable the processing of encrypted data without the need for decryption, they have many different potential applications, especially in scenarios where maintaining privacy is required, but data is also mandated to be processed by potentially untrusted parties, for example, cloud computing and online search engines.

Signcryption

Signcryption is a public key cryptography primitive that provides all of the functions of a digital signature and encryption. Yuliang Zheng invented signcryption, and it is now an ISO standard, ISO/IEC 29150:2011. Traditionally, sign then encrypt or encrypt then sign schemes are used to provide unforgeability, authentication, and non-repudiation, but with signcryption, all the services of digital signatures and encryption such as confidentiality are provided at a cost that is less than that of the sign then encrypt scheme.

Signcryption enables $\text{Cost (signature \& encryption)} \ll \text{Cost (signature) + Cost (Encryption)}$ in a single logical step. This means that the cost of applying a digital signature and encrypting a message in the same logical step is lower, compared to a scheme where a message is first signed and then encrypted in two independent steps. This property makes signcryption an efficient scheme

Secret sharing

Secret sharing is the mechanism of distributing a secret among a set of entities. All entities within a set get a unique part of the secret after it is split into multiple parts. The secret can be reconstructed by combining all or some parts (a certain number or threshold) of the secret. The individual secret shares/parts, on their own, do not reveal anything about the secret.

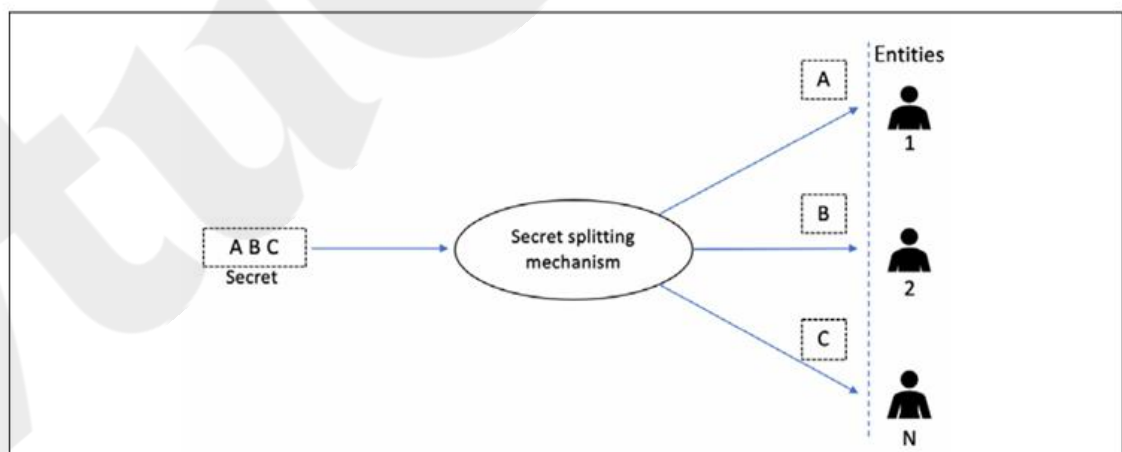


Figure 4.12: Secret sharing scheme

Commitment schemes

Commitment schemes are usually described as a digital cryptographic equivalent of a sealed envelope. A commitment itself does not reveal any information about the actual value inside it. This scheme runs in two phases, namely:

- Commit phase
- Open phase

The commit phase provides two security features; that is, hiding and binding:

- The hiding property is required to ensure that the recipient party cannot find any information or reveal the committed value before the open phase.
- The binding property ensures that once the sender has committed to a value, that value or message cannot be changed.

The open phase (also called the unveil phase) is where the receiver receives some additional information from the sender. This allows the receiver to establish the value hidden (concealed) by the commitment with a proof that the sender has not modified the value after the commitment, that is, that the sender has not deceived during the commit phase.

Commitment schemes are used for building zero-knowledge protocols and range proof protocols. The key idea behind commitment schemes is that they allow someone to secretly commit to a value with the ability to reveal it later. These schemes prevent parties from changing their committed value later, after they've committed to a value. A prime example of a commitment scheme is the Pedersen commitment scheme.

Zero-knowledge proofs

Zero-Knowledge Proofs (ZKPs) were introduced by Goldwasser, Micali, and Rackoff in 1985. These proofs are used to prove the validity of an assertion without revealing any information whatsoever about the assertion. There are three properties of ZKPs that are required: completeness, soundness, and the zero-knowledge property.

Completeness ensures that if a certain assertion is true, then the verifier will be convinced of this claim by the prover.

The soundness property makes sure that if an assertion is false, then no dishonest prover can convince the verifier otherwise.

The zero-knowledge property, as the name implies, is the key property of ZKPs, whereby it is ensured that absolutely nothing is revealed about the assertion except whether it is true or false.

The following diagram shows a variant of this analogy of how the zero-knowledge protocol works. It shows two characters called Peggy and Victor whose roles are Prover and Verifier, respectively. Peggy knows a secret magic word to open the door in a cave, which is shaped like a ring. It has a split entrance and has a magic door in the middle of the ring that blocks the other side. We have also labeled the left and right entrances as A and B. Victor wants to know the secret, but Peggy does not want to reveal it. However, she would like to prove to Victor that she does know the secret:

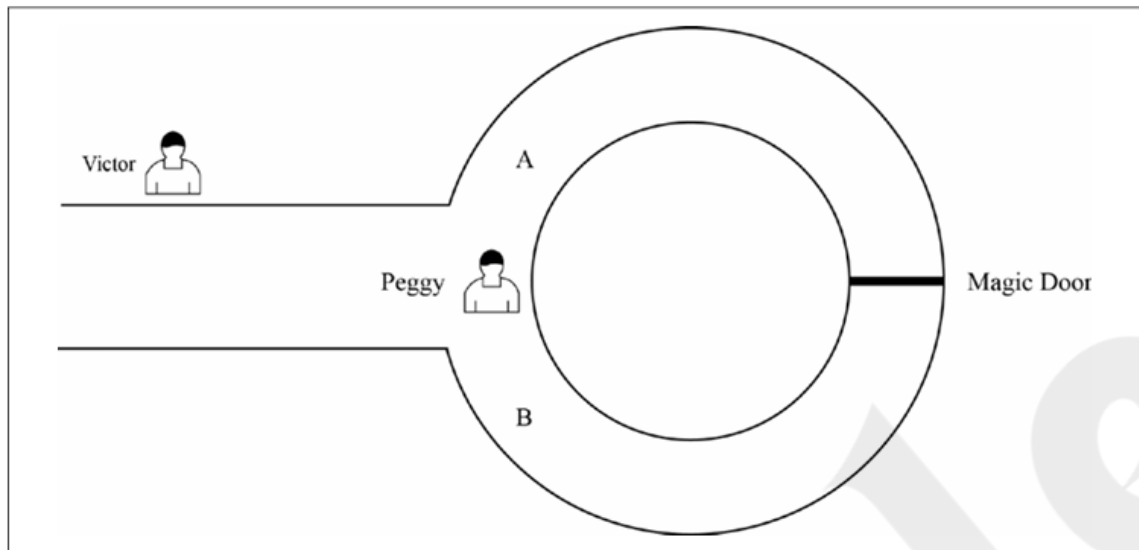


Figure 4.13: Analogy of the zero-knowledge mechanism

Now, there are several steps that are taken by both Peggy and Victor to reach a conclusion regarding whether Peggy knows the secret or not: 1. First, Victor waits outside the main cave entrance and Peggy goes in the cave. 2. Peggy randomly chooses either the A or B entrance to the cave. 3. Now, Victor enters the cave and shouts either A or B randomly, asking Peggy to come out of the exit he named. 4. Victor records which exit Peggy comes out from. Now, suppose Victor asked Peggy to come out from exit A and she came out from exit B. Victor then knows that Peggy does not know the secret. If Peggy comes out of exit A, then there is a 50% chance that she does know the secret, but this also means that she may have got lucky and chose A to enter in the first place, and now has just returned without needing to go through the magic door at all. Now, if this routine is performed several times, and given that Victor is choosing A or B at random, with each run (round) of this routine (protocol), the chances of Peggy getting lucky diminish. If Peggy repeatedly manages to emerge from the entrance that Victor has named, then it is highly probable that Peggy does know the secret to open the magic door.

Note that during this process, Peggy has not revealed the secret at all, but still managed to convince Victor with high probability that she does know the secret. A ZKP comprises the following phases:

1. Witness phase: In this phase, the prover sends proof of the statement and sends it to the verifier.
2. Challenge phase: In this phase, the verifier chooses a question (challenge) and sends it to the prover.
3. Response phase: In this phase, the prover generates an answer and sends it as a response to the verifier. The verifier then checks the answer to ascertain whether the prover really knows the statement. This scheme can also be visualized using the following diagram, which shows how the zero knowledge protocol generally works and what steps the prover and verifier take in order to successfully run the protocol:

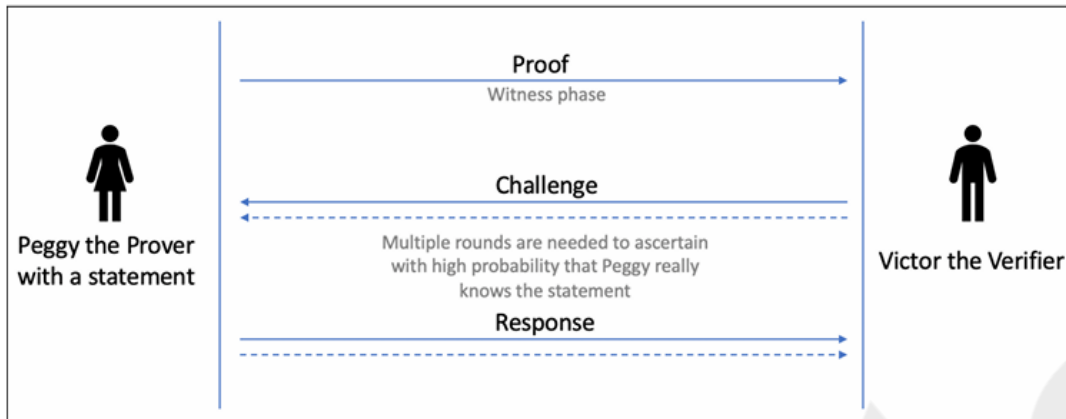


Figure 4.14: A zero-knowledge proof scheme

zk-SNARKs The zk-SNARK is a variant of ZKPs that is more efficient and simpler to use and implement

Zk-SNARKs have several properties, which are described here:

- **Zero knowledge (zk):** This property allows a prover to convince a verifier that an assertion (statement) is true without revealing any information about it. A statement in this context is any computer program that terminates and does not take too long to run, that is, not too many cycles.
- **Succinct (S):** This property allows for a small proof that is very quick to verify and is succinct.
- **Non-interactive (N):** This property allows the prover to prove a statement without any interaction with the verifier.
- **Arguments of Knowledge (ARKs):** These are the arguments to convince the verifier that the prover's assertion is true. Remember that we discussed soundness and completeness earlier. In the case of ARKs, the prover has a computational soundness property, meaning that the prover is computationally bounded, and it is computationally infeasible for it to cheat the verifier. Zk-SNARKs have been implemented in different blockchains.

The most prominent example is Zcash, which uses it for its shielded transactions feature to provide confidentiality and anonymity. Support for cryptographic primitives required for zk-SNARKs has also been introduced in the Ethereum blockchain with the Byzantium release. Zk-SNARK proofs are generated by following a number of different steps. Generating SNARKs for a program (statement) is not a simple process as it requires the program to be converted into a circuit with a very specific format. This specific form is called the Quadratic Arithmetic Program (QAP). The process of proof generation is as follows.

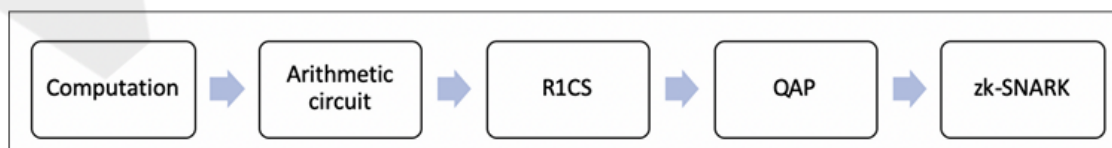


Figure 4.15: zk-SNARK construction

- **Arithmetic circuit:** The first step in zk-SNARK construction is to convert a logical step of a computation into the smallest possible units comprised of basic arithmetic operations. Arithmetic operations include addition, subtraction, multiplication, and division. In the arithmetic circuit, the

computation is presented as wires and gates, representing the flow of inputs and arithmetic operation performed on the inputs.

- **R1CS: R1CS is the abbreviation of Rank 1 Constraint System (R1CS).** Basically, this system is a set of constraints that allows all the steps in the arithmetic circuit to be verified, and also confirms that, at the end of the process, the output is as expected.

- **QAP:** The prover makes use of QAPs to construct proof for the statement. In R1CS, the verifier has to check many different constraints, but with a QAP representation of the circuit, all the different constraints can be bundled into only a single constraint. Finally, QAP is used in the zk-SNARK protocol to prove the assertion through the prover and verifier. Zk-SNARKs are not a silver bullet to privacy problems. Instead, like many other technologies, there are pros and cons.

The biggest criticism of zk-SNARKs is the initial trusted setup. This trusted setup can be influenced and compromised. However, if done correctly, it does work, but there is, however, always a chance that the initial setup was compromised and no one will ever be able to find out. This is the issue that has been addressed in zk-STARKs.

zk-STARKs Zero-knowledge Scalable Transparent ARGuments of Knowledge (zk-STARKs) are a new type of ZKP that has addressed several limitations in zk-SNARKs. This scheme was designed by Eli-Ben Sasson et al.

The key differences between the zk-STARK and zk-SNARK schemes are listed in the following table:

Properties	zk-SNARKs	zk-STARKs
Scalability	Less scalable.	More scalable.
Initial trusted setup	Required.	Not required, has a publicly verifiable mechanism.
Post-quantum resistant	Not resistant to attacks from quantum computers.	Yes, resistant to attacks from quantum computers.
Construction techniques	Zk-SNARKs rely on elliptic curves and pairings.	Zk-STARKs are based on hash functions and concepts from information theory.

Even though this scheme is more efficient, post-quantum resistant, scalable, and transparent, the biggest limitation is its size of proof, which is a few hundred kilobytes compared to zk-SNARKs' 288 bytes. This is seen as a problem in public blockchains where large proofs may pose a problem to scalability and performance.

Zero-knowledge range proofs—ZKRPs ZKRP are used to prove that a number is between a certain range. This can be useful when, for example, someone does not want to reveal their salary but is willing to only reveal the range between which their salary lies. Range proofs can also be used for age checks without requiring the prover to reveal their exact age.

Different types of digital signatures

Blind signatures Blind signatures were introduced by David Chaum in 1982. They are based on public key digital signature schemes, such as RSA. The key idea behind blind signatures is to get the message signed by the signer, without actually revealing the message. This is achieved by disguising or blinding the message before signing it, hence the name blind signatures.

This blind signature can then be verified against the original message, just like a normal digital signature. Blind signatures were introduced as a mechanism to allow the development of digital cash schemes. Specifically, a blind signature can provide unlinkability and anonymity services in a distributed system, such as a blockchain.

The blind signature scheme is also used in building Blindcoin, which is used in the Mixcoin protocol for Bitcoin to hide user addresses from the mixing service. Mixcoin protocol allows anonymous payments in the Bitcoin network, but the user addresses are still visible to the mixing service. Blindcoin is a protocol that addresses this limitation.

Multisignatures In this scheme, a group of entities signs a single message. In other words, multiple unique keys held by their respective owners are used to sign a single message. This scheme was introduced in 1983 by Itakura et al. in their paper A Public-key Cryptosystem Suitable for Digital Multisignatures, vol. 71, Nec Research & Development (1983), pp. 474-480. Multisignatures are also sometimes called multiparty signatures in literature.

In blockchain implementations, multisignatures provide the ability to allow transactions to be signed by multiple users, which results in increased security. This is also called multi-sig and has been implemented in Bitcoin. These schemes can be used in such a way that the requirement of a number of signatures can be set in order to authorize transactions. For example, a 1-of-2 multisignature can represent a joint account where either one of the joint account holders is required to authorize a transaction by signing it. As another variation, for example, a 2-of-2 multisignature can be used in a scenario where both joint account holders' signatures are required to sign the transaction. This concept is also generalized as m of n signatures, where m is the minimum number of required signatures and n is the total number of signatures. This concept can be visualized with the following diagram:

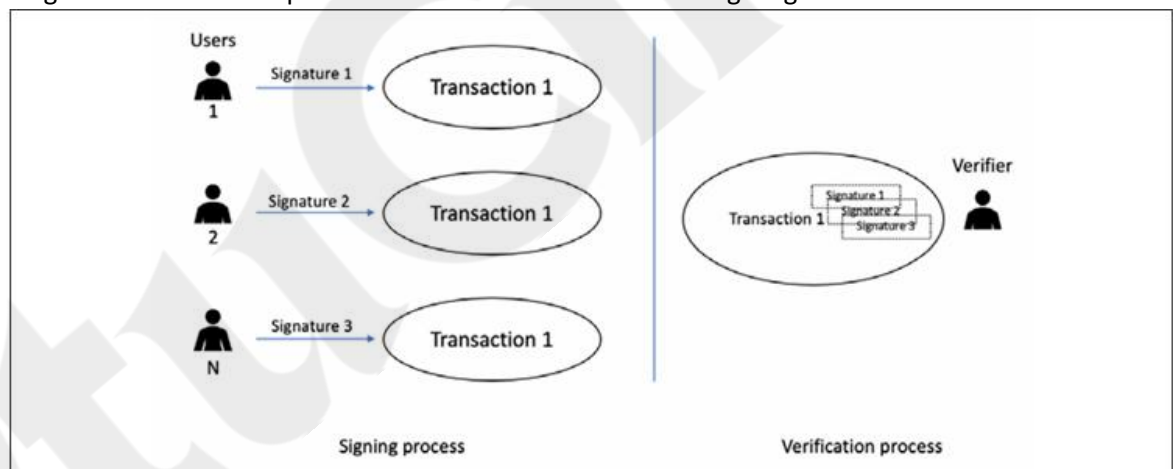


Figure 4.16: Multi signature scheme

The preceding diagram shows the signing process on the left-hand side, where m is the number of different users, and holding m unique signatures signs a single transaction. When the validator or verifier receives it, all the signatures in it need to be individually verified. The Openchain and Multichain blockchains make use of multisignature schemes.

Threshold signatures This scheme does not rely on users to sign the message with their own unique keys; instead, it requires only one public key and one private key, and results in only one digital signature. In multisignature, the resultant message contains digital signatures from all signers and requires verification individually by the verification party, but in threshold signatures, the verifier has to verify only one digital signature. The key idea in the scheme is to split the private key into multiple parts, and each signer keeps their own share of the private

key. The signing process requires each user to use their respective share of the private key to sign the message. The communication between signers is governed by a specific communication protocol. In contrast with multisignatures, the threshold signatures result in a smaller transaction and are quicker to verify. A disadvantage, however, is that in order for threshold signatures to work, all signers must remain online, whereas in multisignatures, the signature can be provided asynchronously; that is, users can provide signatures whenever they are available. From another angle, there could be a scenario in multisignatures where a user can withhold their signature maliciously, which could be disadvantageous. Threshold signatures can also be used to provide anonymity services in a blockchain network:

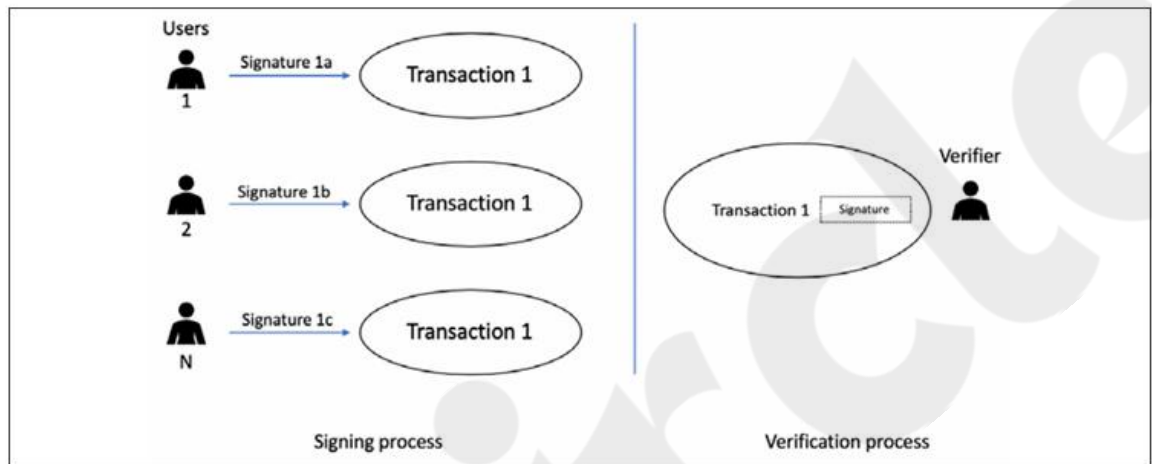


Figure 4.17: Threshold signature scheme

The preceding diagram shows the signing process on the left-hand side, where m number of different users, holding different parts (shares) of a digital signature, sign a single transaction. When the validator or verifier receives it, only one signature needs to be verified.

Aggregate signatures Aggregate signatures are used to reduce the size of digital signatures. This scheme is particularly useful in scenarios where multiple digital signatures are in use. The core idea is to aggregate multiple signatures into a single signature, without increasing the size of the signature of a single message. It is simply a type of digital signature that supports aggregation. The small aggregate signature is enough to provide verification to the verifier that all users did sign their original messages. Aggregate signatures are commonly used to reduce the size of messages in network and security protocols. For example, the size of digital certificate chains in **Public Key Infrastructure (PKI)** can be reduced significantly by compressing all signatures in the chain into a single signature. **Boneh–Lynn–Shacham (BLS)** aggregate signatures is a popular example of the aggregate signature. BLS has also been used in various blockchains, and especially in Ethereum 2.0.

Ring signatures Ring signatures were introduced in 2001 by Ron Rivest, Adi Shamir, and Yael Tauman.

Ring signature schemes allow a mechanism where any member of a group of signers can sign a message on behalf of the entire group. The key requirement here is that the identity of the actual signer who signed the message must remain unknown (computationally infeasible to determine) to an outside observer. It looks equally likely that anyone of the trusted group of signers could have signed the message, but it is not possible to figure out who actually signed the message. Each member of the ring group keeps a public key and a private key. Ring signatures can be used to provide anonymity services. This scheme is used in CryptoNote and Monero.

Encoding schemes Other than cryptographic primitives, binary-to-text encoding schemes are also used in various scenarios. The most common use is to convert binary data into text so that it can either be processed, saved, or transmitted via a protocol that does not support the processing of binary data. For example, images can be stored in a database encoded in base64, which allows a text field to be able to store a picture.

Base64 The base64 encoding scheme is used to encode binary data into printable characters. We can do a quick experiment using the **OpenSSL command line**

```
$ openssl rand 16 -base64  
4ULD5sJtGeoSnogIniHp7g==
```

The preceding command has generated a random sequence of 16 bits and then, using the -base64 switch, it has converted that into a base64 text string. Base64 converts from 8 bits to a 6-bit ASCII representation. This is useful for storage and transmission, especially in cases where binary data handling could lead to incompatibility between systems. This mechanism is a flexible way to represent binary data as ASCII, which can be easily and universally stored and transmitted.

Base58 The base58 scheme was first introduced with Bitcoin and is used to encode integers into alphanumeric strings. The key idea behind this encoding scheme is to avoid non-alphanumeric characters and also those characters that look similar and could lead to ambiguity; for example, a lower-case L (l) may look like the number one (1). This feature is especially useful because Bitcoin addresses must not have any confusion about the character representation; otherwise, it could lead to wrongly sending bitcoins to some non-existent or incorrect address, which is clearly a financial loss. This ingenious encoding scheme avoids this type of situation by ignoring similar looking characters.